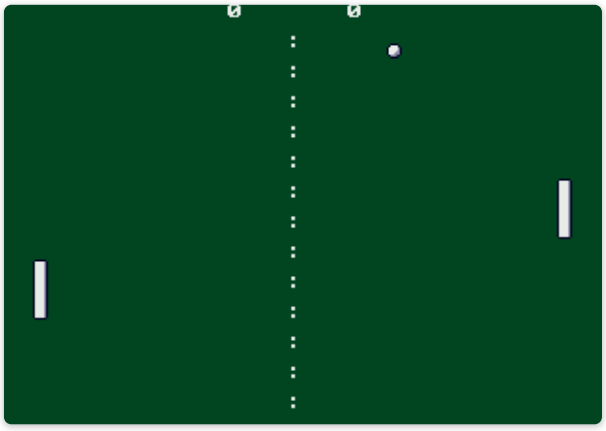
MEGABASIC 1.0

Beginner's Guide

Make your own Sega Mega Drive / Genesis games in BASIC.
No programming experience needed - every program explained line by line.



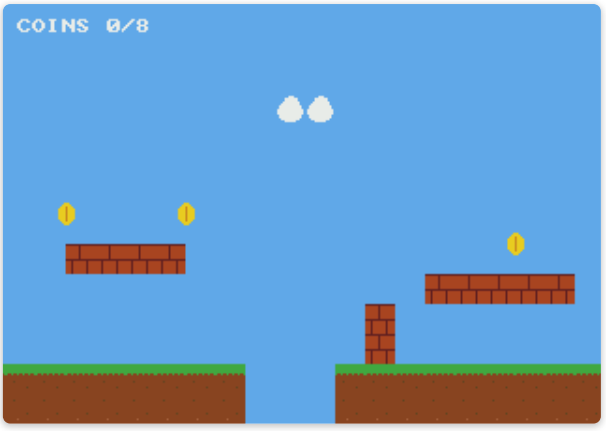
Pong



Breakout



Space Shooter



Coin Quest

Seven complete games inside: Guess the Number · Pong · Snake · Breakout · Space Shooter · Coin Quest · Star Catcher

Contents

Part I — Getting started

1 What is programming?
2 Meet the Mega Drive
3 Setting up your computer
4 Your first program

Part II — Programming basics

5 Variables: the game's memory
6 Maths
7 Making decisions with IF
8 Loops: doing things again and again
9 The game loop
10 Reading the joypad
11 SUBs and FUNCTIONs: your own commands
12 Arrays: many values, one name
13 Your first game: Guess the Number

Part III — Graphics and sound

14 How pictures work on the Mega Drive
15 Your first sprite
16 Animation
17 Many sprites with arrays
18 Collisions and score
19 Smooth movement and gravity
20 Backgrounds and levels with Tiled
21 Scrolling
22 Music and sound effects

Part IV — Six complete games

23 Game: Pong
24 Game: Snake
25 Game: Breakout
26 Game: Space Shooter
27 Game: Coin Quest
28 Game: Star Catcher

Part V — Help

29 When something goes wrong
30 Glossary
31 Command cheat sheet
32 Where to go next

How to use this guide

This book is written for people who have never written a computer program. You do not need to know any maths beyond adding, subtracting, multiplying and dividing. You do need curiosity, patience - and a little time to try things out.

- **Read the chapters in order.** Every chapter uses what you learned in the chapters before it. Part I gets your tools ready, Part II teaches programming, Part III adds pictures and sound, and Part IV builds six complete games of different kinds.
- **Type the programs yourself.** It is tempting to copy and paste, but typing a program makes you notice every word. Every program is also in the `examples` folder in case you get stuck.
- **Read the line-by-line tables.** Each program is shown in a table: the program on the left, an explanation of *every line* on the right. The small numbers on the far left are line numbers - they are only there so we can talk about the lines; you do not type them.
- **Change things and see what happens.** Change a number, rebuild, play. Nothing can break - the worst thing that can happen is an error message, and this guide explains what they mean (chapter 29).
- **Do the exercises.** Short exercises with answers are spread through the book.

Coloured boxes mark special information:

> **Tip** A useful trick that makes your work easier.

> **Good to know** Background knowledge - interesting, but you can skip it on a first reading.

> **Watch out** A common mistake and how to avoid it.

> **Try it** A small experiment to try on your own.

In the program listings, colours help you see the parts of a line: **keywords** are purple, **built-in commands** blue, "text" green, numbers orange and ' *comments* grey. Visual Studio Code colours your programs the same way.

Part I — Getting started

What a program is, what the Mega Drive can do, how to install everything, and your very first program on the screen.

1 What is programming?

A computer does nothing on its own. It follows instructions - very fast, very exactly, and without ever guessing what you meant.

A **program** is a list of instructions. Imagine writing a recipe for a robot that has never cooked: *take a pan, put it on the stove, turn the stove on, wait until the pan is hot*... Each step must be clear, and the order matters. Programming is exactly that, except the robot is a game console and the instructions are written in a **programming language**.

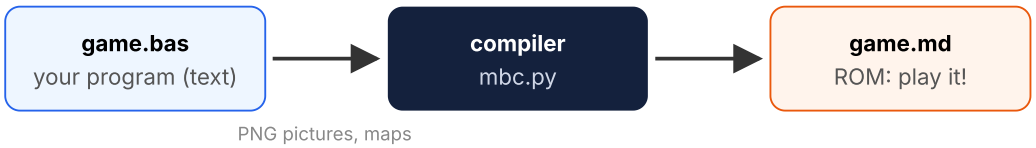
MegaBASIC is such a language. Its instructions are short English words. This is a complete MegaBASIC program:

```
PRINT "HELLO"
```

It has one instruction: `PRINT` means *write on the screen*, and `"HELLO"` is what to write.

From your text to a cartridge

The Mega Drive's processor cannot read English words. It only understands **machine code**: long lists of numbers. A program called a **compiler** translates your text into machine code. The MegaBASIC compiler produces a **ROM file** - the same thing that is stored inside a game cartridge. Its name ends in `.md`.



Your program and pictures go in, a playable ROM comes out.

You can play the ROM in an **emulator** - a program that imitates a Mega Drive on your PC - or copy it to a flash cartridge (EverDrive, Mega EverDrive, MegaSD...) and play it on a real console and television.

Why BASIC?

BASIC was invented in 1964 to teach programming to beginners. Home computers of the 1980s started straight into BASIC, and a whole generation of game developers learned with it. MegaBASIC keeps the friendly style but adds modern features, and because it is *compiled* into machine code it runs fast enough for real action games.

What you will learn

Part	You will be able to...
I. Getting started	install the tools, build and run a program

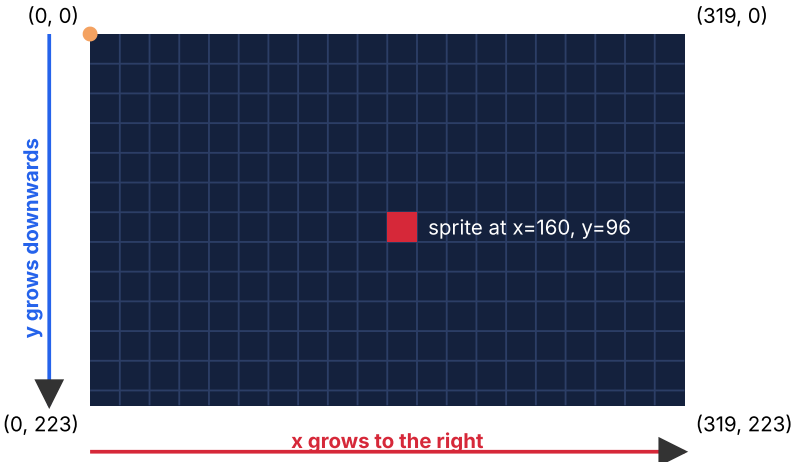
II. Programming basics	use variables, maths, decisions, loops, the joypad, SUBs, FUNCTIONs and arrays - and write a first text game
III. Graphics and sound	show and animate sprites, detect collisions, draw levels with Tiled, scroll them, write music and sound effects
IV. Complete games	build Pong, Snake, Breakout, a Space Shooter, a platform game and Star Catcher
V. Help	fix errors, look up words in the glossary and the command cheat sheet

2 Meet the Mega Drive

Before we program it, let's look at what is inside the console. You don't have to remember any of this - just get a feeling for the parts.

The screen

The picture is made of tiny coloured squares called **pixels**. The Mega Drive's screen is **320 pixels wide and 224 pixels high**. Every pixel has an address made of two numbers: **x** (how far from the left edge) and **y** (how far from the top edge). Counting starts at 0.

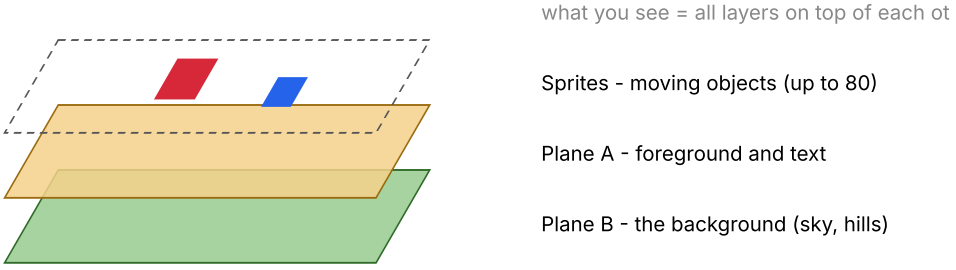


Screen coordinates. Note that y counts **downwards** - unlike in maths lessons.

Text uses a coarser grid: the screen holds **40 columns and 28 rows** of characters, because each character is 8 × 8 pixels (320 / 8 = 40, 224 / 8 = 28).

Tiles, planes and sprites

The Mega Drive does not store the picture pixel by pixel. It builds it from **tiles**: little 8 × 8 pixel pictures, like mosaic stones. There are three kinds of layer:



Layers of a Mega Drive picture. Each plane can scroll on its own.

- **Planes A and B** are large grids of tiles used for backgrounds: landscapes, walls, levels. Each plane can be moved (**scrolled**) smoothly.
- **Sprites** are small pictures that can be placed at any pixel position: the hero, enemies, bullets, stars. Up to 80 at once.
- There is also a **window** plane that never scrolls - handy for showing the score.

Colours and palettes

The console can make 512 different colours, but each tile or sprite may only use the 16 colours of one **palette**. There are 4 palettes, so 64 colours can be on screen at once. In every palette, colour number 0 is **transparent**: you see through it to the layer behind.

Frames

The television picture is redrawn **60 times per second** (50 in Europe on PAL consoles). Each picture is called a **frame**. A game works like a flip-book: every frame, everything moves a little, and the new picture is shown. Moving a sprite 2 pixels every frame makes it travel 120 pixels per second.

Sound

Two sound chips play together: the **YM2612** with 6 'FM' channels for rich instrument sounds, and the **PSG** with 3 square-wave channels and 1 noise channel for beeps and drums. A channel plays one note at a time, so up to 10 notes can sound together.

> **Good to know** All the hardware details - writing to the video chip, the sound chips' registers, interrupts - are handled by MegaBASIC. You just say `SPRITE` or `PLAYMUSIC`.

3 Setting up your computer

We need four things: Python (which runs the compiler), the MegaBASIC files, an editor to write programs in, and an emulator to play them. This takes about 15 minutes, once.

Step 1 - Install Python 3

The MegaBASIC compiler is itself a program, written in the Python language. You do not need to learn Python - you only need it installed.

System	What to do
Windows	Go to **python.org** > Downloads and run the installer. **Important:** on the first screen tick the box **"Add python.exe to PATH"**, then click *Install Now*.
macOS	Download the macOS installer from **python.org** and run it.
Linux	Python 3 is normally installed already. If not: `sudo apt install python3` (Ubuntu/Debian).

To check, open a terminal (Windows: press the Windows key, type `cmd`, press Enter; macOS: Terminal) and type:

Terminal

```
python --version
```

You should see something like `Python 3.12.4`. On macOS/Linux type `python3 --version` instead.

Step 2 - Unpack MegaBASIC

Unzip `MegaBASIC-1.0.zip` into a folder you can find again, e.g. `C:\MegaBASIC` on Windows or `MegaBASIC` in your home folder. Inside:

File / folder	What it is
`mbc.py`	the MegaBASIC compiler
`megabasic/`	the parts of the compiler - you never need to touch these
`examples/guide/`	the small programs from Parts I-III of this guide
`examples/games/`	the games from Part IV
`examples/starcatcher/`	the Star Catcher game
`megabasic-1.0.0.vsix`	the extension for Visual Studio Code
`docs/`	this guide and the Reference Manual

Step 3 - Install Visual Studio Code and the MegaBASIC extension

1. Download Visual Studio Code from **code.visualstudio.com** and install it (it is free).
2. Start it. On the left edge, click the icon made of four squares (**Extensions**), or press `Ctrl+Shift+X`.
3. At the top of the Extensions panel click the **...** (three dots) and choose **Install from VSIX...**
4. Choose the file `megabasic-1.0.0.vsix` from the MegaBASIC folder. After a moment a message says the extension is installed.
5. Choose **File > Open Folder...** and open the MegaBASIC folder (or a new, empty folder for your own games).

From now on, every file ending in `.bas` is coloured, VS Code suggests commands while you type, shows help when you hold the mouse over a word, and checks your program for mistakes whenever you save.

Step 4 - Install an emulator

Pick one free Mega Drive emulator: **BlastEm** (very accurate), **Kega Fusion** or **Gens** (easy), or **RetroArch** with the *Genesis Plus GX* core. Then tell VS Code where it is: **File > Preferences > Settings**, type `megabasic` in the search box, and paste the full path of the emulator program into **Emulator Path** - for example `C:\Emulators\blastem\blastem.exe`.

Step 5 - The two keys you will press all the time

Key	What it does
`Ctrl+Shift+B`	**Build**: saves your program and compiles it into a ROM (`.md` file next to it).
`F5`	**Build and run**: builds, then starts the emulator with your game.
`F1`	Opens the MegaBASIC reference for the word under the cursor.
`Ctrl+Space`	Shows suggestions: commands, your variables...

Building without VS Code

You can write programs in any text editor (Notepad, TextEdit...) and build them in a terminal. Go to the folder with `cd`, then:

Terminal (Windows)

```
cd C:\MegaBASIC
python mbc.py examples\guide\01_hello.bas
```

What you see

```
MegaBASIC 1.0: examples\guide\01_hello.bas -> examples\guide\01_hello.md
  program+data: 11,040 bytes   ROM: 128 KB
  RAM: 1,346 bytes variables, 0 bytes arrays   (0.11s)
```

The compiler tells you where the ROM was written and how much space the program uses. Open the `.md` file in your emulator.

> **Watch out** If the terminal says *python is not recognized*, Python was installed without the PATH option. Run the installer again, choose *Modify* and tick *Add Python to environment variables* - or try `py mbc.py ...` instead.

4 Your first program

Time to make the Mega Drive say hello. In VS Code choose File > New File, save it as hello.bas, and type the program below.

01_hello.bas	
`' My first MegaBASIC program`	A line that starts with an apostrophe `'` is a **comment**. The compiler skips it completely. Comments are notes for people - for you next week, or for a friend reading your program. Use them a lot.
`TITLE "HELLO"`	`TITLE` gives the game a name. The name is stored in the ROM; emulators show it in their title bar or game list. The name is written between double quotes `"`. Text between quotes is called a **string**.
`PRINT "HELLO, MEGA DRIVE!"`	`PRINT` writes text on the screen. The first PRINT starts in the top-left corner (column 0, row 0). After printing, the invisible **cursor** moves to the start of the next row - like pressing Enter on a typewriter.
`PRINT`	A `PRINT` with nothing after it prints nothing and just moves the cursor down: an empty line.
`PRINT "THIS IS MY FIRST PROGRAM."`	This text appears on row 2, below the empty row.
`PRINT AT 10, 12, "TEXT CAN GO ANYWHERE"`	`PRINT AT x, y,` puts text at an exact place: column 10, row 12. Columns go from 0 to 39 (left to right), rows from 0 to 27 (top to bottom). Note the commas between the numbers and before the text.



The program running in an emulator

Build and run

1. Save the file (`Ctrl+S`).
2. Press `F5` (or `Ctrl+Shift+B` and open `hello.md` in your emulator yourself).
3. The emulator starts and shows your text. You have programmed a Mega Drive!

What just happened?

The compiler read your four instructions, checked them, translated them into about 300 bytes of 68000 machine code, and added the **runtime**: a few thousand bytes of ready-made machine code that starts the console, sets up the video chip, contains the font for the letters, the sound driver and everything else your program might need. The console runs your instructions from top to bottom. After the last one the program is finished - the text simply stays on screen.

Upper case, lower case, spaces

- MegaBASIC does not care about upper and lower case in commands: `print`, `Print` and `PRINT` are the same. This guide writes commands in CAPITALS so they stand out.
- Text inside quotes is printed exactly as written. The built-in font has upper and lower case letters, but CAPITALS look most "retro".
- Extra spaces between words do not matter: `PRINT   "HI"` works. Indenting lines (putting spaces in front) also does not matter to the compiler - it is only for people, as you will see in chapter 7.

When you make a mistake

Try it now: change `PRINT` into `PRNT` and build. Instead of a ROM you get a message like:

Error message

```
01_hello.bas:3: error: unknown command or SUB 'PRNT'
```

It tells you the **file**, the **line number** (3) and what is wrong. In VS Code the line is also underlined in red. Fix it and build again. Error messages are not a failure - they are the compiler helping you. Experienced programmers see them every day.

Exercise. Make the program print your name in the middle of the screen.
Answer: Add a line such as `PRINT AT 15, 14, "ALEX"`. The middle column is about 20; start a bit to the left so the name is centred: 20 - (letters / 2).

Exercise. What happens with `PRINT AT 38, 5, "HELLO"`?
Answer: The text is too long for the row: after column 39 it continues on the next row.

Part II — Programming basics

Variables, maths, decisions, loops, the game loop, the joypad, SUBs, FUNCTIONs and arrays. At the end you write your first complete game: Guess the Number.

5 Variables: the game's memory

Every game must remember things: the score, how many lives are left, where the player is. Programs remember numbers in variables.

Think of a **variable** as a box with a name label on it. Inside the box there is one number. You can look at the number, or throw it away and put a new one in. A game might have boxes labelled `score`, `lives`, `playerX`…



Three variables after the program below has run.

02_variables.bas

```
1  ' Variables and arithmetic
2  DIM apples, pears, total
3  apples = 12
4  pears = 7
5  total = apples + pears
6  PRINT "APPLES: "; apples
7  PRINT "PEARS:  "; pears
8  PRINT "TOTAL:  "; total
9  PRINT
10 PRINT "7 * 6   = "; 7 * 6
```

`DIM` (short for *dimension*) creates variables. This line makes three boxes called `apples`, `pears` and `total`. Several names are separated by commas. Every new variable starts with the value 0.

Read this as *apples becomes 12*: the number 12 is put into the box `apples`. The old value (0) is thrown away. This is called an **assignment**. The box name is always on the left of the `=`.

Put 7 into `pears`.

The computer first works out the right side: it looks into `apples` (12) and `pears` (7) and adds them: 19. Then 19 is put into `total`. The boxes `apples` and `pears` are not changed.

PRINT can show several things on one line. Separate them with a semicolon `;`. The text in quotes is printed as it is; the name without quotes is replaced by the number in the box. Result: `APPLES: 12`.

Shows `PEARS: 7`. The extra spaces inside the quotes line the numbers up.

Shows `TOTAL: 19`.

An empty line.

You can calculate directly inside PRINT. `*` is the multiplication sign (computers do not use x). Notice that the part in quotes is printed as text, but `7 * 6` outside the quotes is calculated: 42.

11	`PRINT "100 / 8 = "; 100 / 8; "  REMAINDER "; 100 MOD 8`	`/` divides. MegaBASIC works with **whole numbers** only, so 100 / 8 gives 12 - the .5 is dropped. `MOD` gives what is left over: 100 = 12 × 8 + 4, so `100 MOD 8` is 4.
12	`PRINT "-5 + 2  = "; -5 + 2`	Negative numbers are no problem: -5 + 2 = -3.
13	`total += 100`	`total += 100` means *add 100 to total*. It is a short way of writing `total = total + 100`: take the value of total (19), add 100, put the result (119) back into total.
14	`PRINT "TOTAL AFTER += 100: "; total`	Proves it: shows 119.



The = sign does not mean "equals" here

In maths, `x = x + 1` is nonsense. In programming it is perfectly normal: it means *take x, add 1, and store the result in x*. The `=` in an assignment is an instruction, not a statement of fact. (Inside an `IF` - chapter 7 - the `=` is a question: *is it equal?*)

Choosing names

- A name starts with a letter and may contain letters, digits and the underscore `_`: `score`, `player2Lives`, `enemy_speed`.
- No spaces: `player x` is not allowed, use `playerX` or `player_x`.
- Names cannot be the same as a MegaBASIC word: `print` or `sprite` are taken.
- Upper and lower case are the same: `Score`, `SCORE` and `score` are one variable.
- Choose names that say what is inside. `lives` is much better than `l` or `x7`.

Constants

Some numbers never change while the game runs - the screen width, the player's speed, the number of enemies. Give them a name with `CONST`:

```
CONST SPEED = 3
CONST MAX_ENEMIES = 10
x = x + SPEED
```

If you later want a faster player, you change one line. Constants also take no memory. By custom, constants are written in CAPITALS.

Good to know Every number in MegaBASIC is a whole number between about -2 billion and +2 billion. There are no fractions like 1.5 - you will learn a neat trick for smooth slow movement in chapter 19.

Exercise. Write a program that stores your age in a variable and prints how old you will be in 10 years.
Answer: `DIM age` / `age = 12` / `PRINT "IN 10 YEARS: "; age + 10`

6 Maths

Games calculate all the time: new positions, scores, distances, damage. Here are the tools.

02c_maths.bas	
`' Maths: a little shop calculator`	
`DIM price, amount, cost, coins`	Four variables for our shop.
`price = 15`	One sword costs 15 gold...
`amount = 4`	...and we buy 4.
`cost = price * amount`	Multiply: 15 * 4 = 60, stored in `cost`.
`PRINT "4 SWORDS COST "; cost; " GOLD"`	Shows `4 SWORDS COST 60 GOLD`. Text, a number and more text joined with `;`.
`coins = 100`	Our purse.
`PRINT "YOU HAVE "; coins; " GOLD"`	Shows how much gold we have: 100.
`PRINT "AFTER BUYING: "; coins - cost; " GOLD"`	A calculation inside PRINT: 100 - 60 = 40.
`PRINT`	Empty line.
`PRINT "100 GOLD BUYS "; 100 / price; " SWORDS"`	Whole-number division: 100 / 15 = 6 (6 × 15 = 90). The rest is thrown away...
`PRINT "AND "; 100 MOD price; " GOLD IS LEFT OVER"`	...but MOD tells us the rest: 100 - 90 = 10 gold left over.
`PRINT`	Empty line.
`PRINT "2 + 3 * 4   = "; 2 + 3 * 4`	Like in school maths, `*` and `/` are done before `+` and `-`: 3 * 4 = 12, then 2 + 12 = 14.
`PRINT "(2 + 3) * 4 = "; (2 + 3) * 4`	Brackets are calculated first: 5 * 4 = 20. When in doubt, use brackets.
`PRINT "17 MOD 2    = "; 17 MOD 2; "  (1 MEANS ODD)"`	Any number MOD 2 is 0 for even numbers and 1 for odd numbers - a classic trick.
`PRINT "RND(100)    = "; RND(100)`	`RND(100)` is a **random** whole number from 0 to 99. Games use RND for dice, enemy positions, surprises.



Sign	Meaning	Example	Result
`+`	add	`7 + 2`	9
`-`	subtract	`7 - 2`	5
`*`	multiply	`7 * 2`	14
`/`	divide (whole number)	`7 / 2`	3

`MOD`	remainder of a division	`7 MOD 2`	1
`( )`	do this part first	`(1 + 2) * 3`	9

Useful maths functions

A **function** is a built-in calculation: you give it values in brackets and it gives back an answer.

Function	Gives back	Example
`RND(n)`	random number 0 ... n-1	`RND(6) + 1` = a dice roll 1-6
`ABS(x)`	x without its minus sign	`ABS(-5)` = 5
`MIN(a, b)`	the smaller number	`MIN(3, 8)` = 3
`MAX(a, b)`	the bigger number	`MAX(3, 8)` = 8
`CLAMP(x, low, high)`	x, but kept between low and high	`CLAMP(400, 0, 304)` = 304
`SQR(x)`	square root	`SQR(81)` = 9

> **Tip** `CLAMP` is the game programmer's best friend: `x = CLAMP(x, 0, 304)` stops a player from walking off the screen.

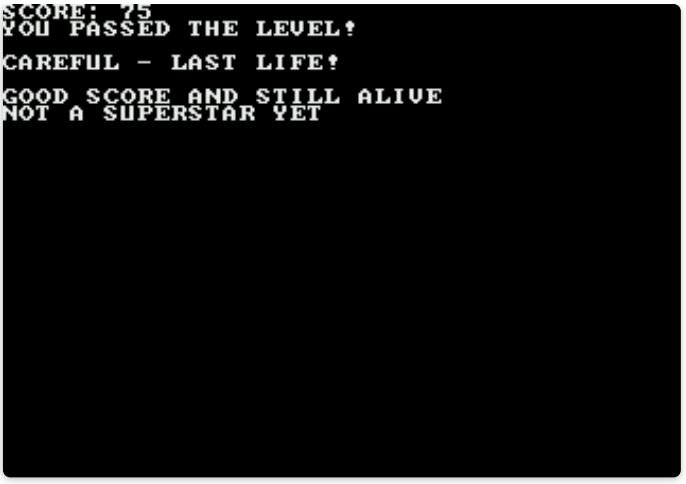
Exercise. A level takes 1 minute and 35 seconds. How do you print that time in seconds?
Answer: `PRINT 1 * 60 + 35` (95 seconds). Or the other way: `PRINT 95 / 60; " MIN "; 95 MOD 60; " SEC"`.

7 Making decisions with IF

A game constantly asks questions: Has the player lost all lives? Did the ball hit the bat? Is the score high enough? IF lets the program choose what to do.

03_if.bas

1	`' Making decisions with IF`	
2	`DIM score, lives`	Two variables...
3	`score = 75`	...with values we can change to test the program.
4	`lives = 1`	One life left - the interesting case for the block IF below.
5	`PRINT "SCORE: "; score`	Show the score.
6	`IF score >= 50 THEN PRINT "YOU PASSED THE LEVEL!"`	The one-line IF: `IF question THEN action`. The question here is *is score bigger than or equal to 50?* If the answer is yes (true), the action after `THEN` is done. If no (false), it is skipped. 75 >= 50, so the text is printed.
7	`IF score < 50 THEN PRINT "TRY AGAIN"`	The opposite question. 75 is not smaller than 50 - this PRINT is skipped.
8	`PRINT`	Empty line.
9	`IF lives = 0 THEN`	The **block IF** for when you need more than one line or several choices. Here `=` is a question: *is lives equal to 0?*
10	`    PRINT "GAME OVER"`	Only runs if lives is 0. The line is indented (moved right) to show that it belongs to the IF.
11	`ELSEIF lives = 1 THEN`	`ELSEIF` = *otherwise, if...*. Asked only if the questions above were all false. lives is 1, so...
12	`    PRINT "CAREFUL - LAST LIFE!"`	...this line runs.
13	`ELSE`	`ELSE` = *in all other cases*. Here: when lives is 2 or more.
14	`    PRINT "LIVES LEFT: "; lives`	Not run this time, because a branch above was already chosen.
15	`END IF`	`END IF` closes the block. Exactly one of the three branches ran; the program continues below.
16	`PRINT`	Empty line.
17	`IF score > 50 AND lives > 0 THEN PRINT "GOOD SCORE AND STILL ALIVE"`	`AND` joins two questions: **both** must be true. 75 > 50 is true and 1 > 0 is true, so the text appears.
18	`IF score > 90 OR lives > 5 THEN PRINT "SUPERSTAR!" ELSE PRINT "NOT A SUPERSTAR YET"`	`OR`: it is enough that **one** of the questions is true. Both are false here, so the part after `ELSE` is used. A one-line IF may have an ELSE too.



Try it Change `score = 75` to 30, and `lives = 1` to 0 or 4, rebuild, and predict what will be printed before you look.

Questions you can ask

Write	Means	Example that is true
`a = b`	a is equal to b	`3 = 3`
`a <> b`	a is not equal to b	`3 <> 4`
`a < b`	a is smaller than b	`2 < 5`
`a > b`	a is bigger than b	`5 > 2`
`a <= b`	smaller or equal	`5 <= 5`
`a >= b`	bigger or equal	`6 >= 5`
`q1 AND q2`	both are true	`1 < 2 AND 3 < 4`
`q1 OR q2`	at least one is true	`1 > 2 OR 3 < 4`
`NOT q`	the opposite	`NOT (1 > 2)`

TRUE and FALSE are numbers

When the computer answers a question, the answer is a number: **TRUE is -1, FALSE is 0**. You can store an answer in a variable and use it later. IF treats every number that is not 0 as true. This is very handy:

```
gameOver = (lives = 0)        ' gameOver becomes TRUE or FALSE
IF gameOver THEN PRINT "BYE"

musicOn = TRUE
IF musicOn THEN PLAYMUSIC song
```

Why indent?

The compiler ignores the spaces at the start of a line, but people need them. Compare:

Hard to read

```
IF a > 1 THEN
IF b > 1 THEN
PRINT "BOTH"
END IF
END IF
```

Easy to read

```
IF a > 1 THEN
    IF b > 1 THEN
        PRINT "BOTH"
    END IF
END IF
```

Indent everything inside a block by 4 spaces (VS Code does it when you press Tab). You will spot a missing `END IF` immediately.

> **Watch out** A very common mistake: forgetting `THEN`, or forgetting `END IF` for a block IF. The compiler says *missing END IF* - it means an IF that starts a block was never closed.

Exercise. Write an IF that prints "HIGH SCORE!" when `score` is bigger than `best`.
Answer: `IF score > best THEN PRINT "HIGH SCORE!"`

8 Loops: doing things again and again

Computers are great at repetition. A loop runs the same lines many times - 10 times, 1000 times, or forever.

04_loops.bas

1	`' Three kinds of loops`	
2	`PRINT "FOR: TIMES TABLE OF 7"`	A heading for the first part.
3	`FOR i = 1 TO 5`	The **FOR loop** counts. The variable `i` (the *counter*) starts at 1. The lines between `FOR` and `NEXT` - the **body** of the loop - run with i = 1, then i = 2, 3, 4, and finally 5. `i` is a normal variable; FOR creates it automatically.
4	`    PRINT "  "; i; " X 7 = "; i * 7`	The body uses the counter: it prints one line of the times table. First round: `1 X 7 = 7`, then `2 X 7 = 14` ...
5	`NEXT`	`NEXT` marks the end of the body. It adds 1 to i and jumps back up to FOR. When i would become 6 - more than 5 - the loop ends and the program continues after NEXT.
6	`PRINT`	Two empty lines...
7	`PRINT "WHILE: DOUBLING"`	...and the heading for the second loop.
8	`n = 1`	Start value for `n`.
9	`WHILE n < 300`	The **WHILE loop** repeats *as long as* its question is true. Before every round it asks: is n smaller than 300?
10	`    PRINT "  "; n;`	Print n. The `;` at the end means *stay on this line*, so all numbers appear side by side.
11	`    n = n * 2`	Double n: 1, 2, 4, 8, 16, 32, 64, 128, 256... When n becomes 512, the question `n < 300` is false and the loop stops.
12	`WEND`	`WEND` (While END) closes the WHILE loop. You may also write `END WHILE`.
13	`PRINT`	Finish the line of numbers...
14	`PRINT`	...and leave an empty line.
15	`PRINT "DO..LOOP UNTIL: COUNTDOWN"`	Heading for the third loop.
16	`c = 5`	Countdown start.
17	`DO`	`DO` starts a loop...
18	`    PRINT "  "; c;`	...print the count...
19	`    c -= 1`	...subtract 1 (short for `c = c - 1`)...
20	`LOOP UNTIL c = 0`	...and repeat *until* c is 0. The question is asked at the end, so the body always runs at least once.
21	`PRINT "  LIFT OFF!"`	Runs after the loop has finished.



Loops inside loops

A loop body can contain another loop. The inner loop then runs completely, for every round of the outer loop. This is how you work through rows and columns:

04b_nested.bas

1	`' Loops inside loops: a multiplication table`	
2	`PRINT "     1  2  3  4  5  6  7  8  9"`	The column headings.
3	`FOR row = 1 TO 9`	The **outer** loop: row goes from 1 to 9.
4	`    PRINT AT 0, row, row`	Print the row number in column 0. `PRINT AT` also accepts variables as positions.
5	`    FOR col = 1 TO 9`	The **inner** loop: for each row, col goes from 1 to 9 - so this body runs 9 × 9 = 81 times in total.
6	`        PRINT AT col * 3 + 2, row, row * col`	Print the product. Each column is 3 characters wide, so the x position is calculated: col 1 at x = 5, col 2 at x = 8...
7	`    NEXT`	End of the inner loop...
8	`NEXT`	...and of the outer loop. Each NEXT belongs to the nearest open FOR. (You may write `NEXT col` and `NEXT row` to make it clearer.)
9	`PRINT AT 0, 12, "A BOX OF STARS:"`	A second example:
10	`FOR y = 14 TO 18`	Rows 14 to 18...
11	`    FOR x = 2 TO 20`	...columns 2 to 20...
12	`        PRINT AT x, y, "*"`	...a star in every cell: a filled rectangle of 19 × 5 stars.
13	`    NEXT`	End inner loop.
14	`NEXT`	End outer loop.



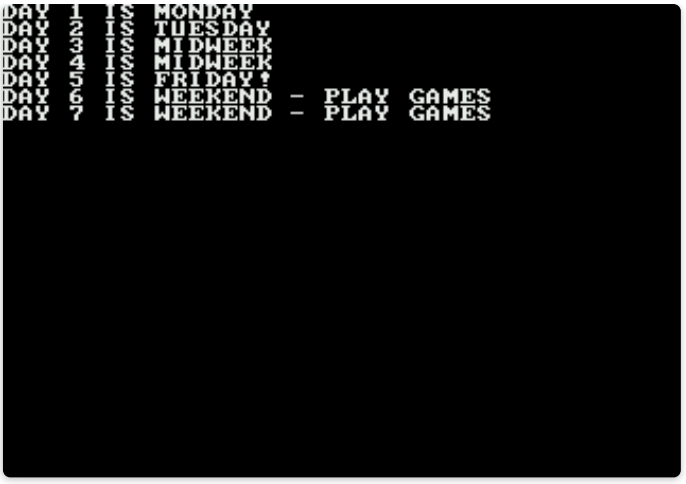
SELECT CASE: choosing from many options

When one value decides between many actions, a long chain of ELSEIFs gets messy. `SELECT CASE` is tidier:

03b_select.bas

```
' SELECT CASE: one value, many choices
FOR day = 1 TO 7
    PRINT "DAY "; day; " IS ";
    SELECT CASE day
        CASE 1
            PRINT "MONDAY"
        CASE 2
            PRINT "TUESDAY"
        CASE 3, 4
            PRINT "MIDWEEK"
        CASE 5
            PRINT "FRIDAY!"
        CASE ELSE
            PRINT "WEEKEND - PLAY GAMES"
    END SELECT
NEXT
```

Go through the days of the week.	
Start a line; the `;` at the end keeps the cursor on the same line.	
`SELECT CASE value`: compare `day` with each CASE below, from top to bottom.	
If day is 1...	
...this line runs, and then the program jumps to END SELECT.	
If day is 2...	
Runs only for day 2.	
A CASE can list several values separated by commas. You can also write ranges (`CASE 3 TO 4`) or comparisons (`CASE IS > 5`).	
Runs for day 3 and for day 4.	
If day is 5...	
...it's Friday!	
`CASE ELSE` catches everything not matched above (6 and 7).	
Runs for 6 and 7.	
End of the SELECT block. Only one CASE ever runs.	
Next day.	



Leaving a loop early

Command	What it does
`EXIT FOR`	leave the FOR loop now (also `EXIT WHILE`, `EXIT DO`)
`CONTINUE FOR`	skip the rest of this round, start the next one
`FOR i = 10 TO 0 STEP -1`	count backwards; `STEP 2` counts in twos

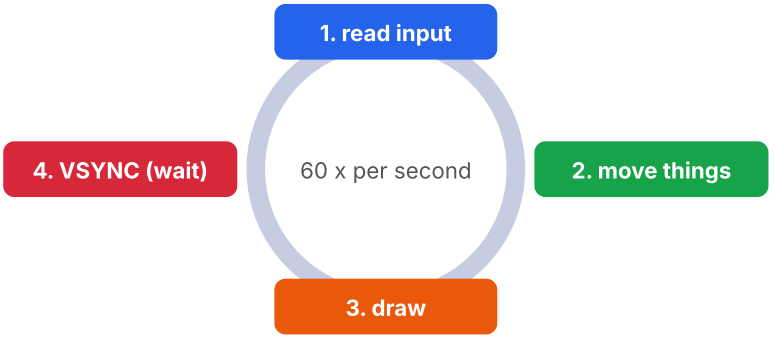
Good to know A loop that never ends is called an **endless loop** . Usually it is a mistake - but the main loop of every game is endless on purpose: it runs until the console is switched off.

Exercise. Print the numbers 10, 20, 30 ... 100 using a FOR loop.
Answer: `FOR n = 10 TO 100 STEP 10` / `PRINT n` / `NEXT`

Exercise. Print a triangle of stars: 1 star on the first row, 2 on the second ... 5 on the fifth.
Answer: `FOR row = 1 TO 5` / `FOR s = 1 TO row` / `PRINT AT s, row, "*"` / `NEXT` / `NEXT`

9 The game loop

Now the most important idea in this book. Every video game - from Pong to the newest console games - is built around one loop that runs once per frame.



The game loop: input, update, draw, wait for the next frame - forever.

05_moving_text.bas

```
1  ' The game loop: a bouncing word
2  DIM x, y, dx, dy
3  x = 5 : y = 5
4  dx = 1 : dy = 1
5  DO
6      PRINT AT x, y, "      "        ' erase the old position
7      x = x + dx
8      y = y + dy
9      IF x <= 0 OR x >= 35 THEN dx = -dx
10     IF y <= 0 OR y >= 27 THEN dy = -dy
11     PRINT AT x, y, "MEGA!"         ' draw at the new position
12     WAIT 4                         ' wait 4 frames (1/15 second)
13 LOOP
```

`x`, `y`: where the word is (column and row). `dx`, `dy`: its **direction** - how much x and y change each step (d stands for *difference*).

Start at column 5, row 5. The colon `:` lets you write two statements on one line.

Move 1 column right and 1 row down each step: diagonally down-right.

The **game loop** begins. There is no UNTIL or WHILE at the end, so it runs forever.

Step 1: erase the word at its current position by printing 5 spaces over it. Without this, the word would leave a trail.

Step 2: move. Add the direction to the position...

...for both x and y.

If the word reaches the left edge (column 0) or the right edge (column 35 - the word is 5 letters, 35 + 5 = 40)...

...or the top or bottom edge, reverse the vertical direction. `dx = -dx` turns 1 into -1 and -1 into 1: the word bounces.

Step 3: draw the word at the new position.

Step 4: wait 4 frames. With 60 frames per second this is 1/15 of a second, so the word moves 15 steps per second.

Back to DO. Erase, move, bounce, draw, wait... forever.



VSYNC - waiting for the next frame

Our moving word used `WAIT 4`. Real games use `VSYNC` (vertical synchronisation), which waits exactly until the next frame begins - `WAIT 1` and `VSYNC` are the same. VSYNC does two important jobs:

1. It makes the game run at a steady speed - one round of the loop per frame, 60 per second.
2. It sends all changes - sprite positions, scrolling, colours - to the video chip at the right moment, in the short pause between two pictures. That is why nothing flickers.

So remember the golden rule: **every game loop calls VSYNC exactly once**.

The shape of every game in this book

```
DO
    ' 1. read the joypad
    ' 2. move the player, the enemies, the bullets
    ' 3. draw: sprites, score
    VSYNC        ' 4. wait for the next frame
LOOP
```

Good to know `TIMER` counts frames since the console was switched on: 60 per second. `TIMER / 60` is the number of seconds. Animations often use `(TIMER >> 3) AND 3`, which counts 0, 1, 2, 3, 0, 1... changing every 8 frames.

10 Reading the joypad

A game needs a player. The Mega Drive joypad has a direction pad (D-pad), buttons A, B, C and START - the 6-button pad adds X, Y, Z and MODE.

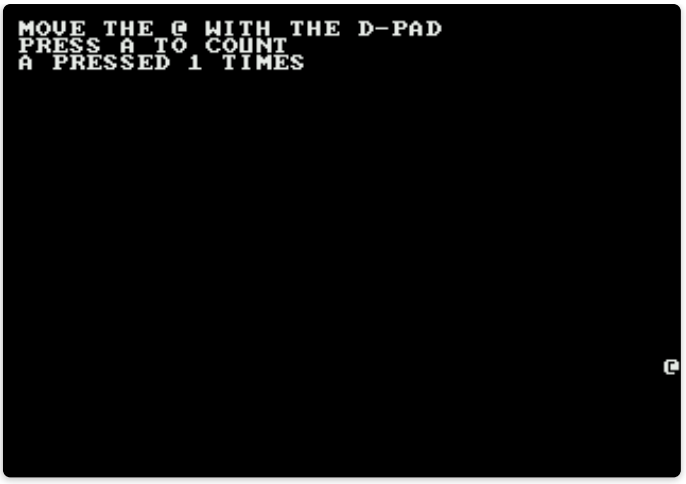
Two functions read the joypad. Both give back TRUE or FALSE:

Function	TRUE when...	Use it for
`BUTTON(BTN_LEFT)`	the button is **held down** right now (every frame while you hold it)	walking, steering, accelerating
`PRESSED(BTN_A)`	the button was **just pressed** (only in the one frame when it goes down)	jumping, firing single shots, menus, START

The button names are built-in constants: `BTN_UP`, `BTN_DOWN`, `BTN_LEFT`, `BTN_RIGHT`, `BTN_A`, `BTN_B`, `BTN_C`, `BTN_START` (and `BTN_X`, `BTN_Y`, `BTN_Z`, `BTN_MODE`).

06_joypad.bas

1	`' Reading the joypad`	
2	`DIM x, y, presses`	The position of the @ character, and a counter for button presses.
3	`x = 20 : y = 14`	Start in the middle of the text screen.
4	`PRINT AT 1, 1, "MOVE THE @ WITH THE D-PAD"`	Instructions at the top of the screen.
5	`PRINT AT 1, 2, "PRESS A TO COUNT"`	Second line of instructions.
6	`DO`	The game loop.
7	`    PRINT AT x, y, " "`	Erase the @ at its current position.
8	`    IF BUTTON(BTN_LEFT) AND x > 0 THEN x -= 1`	If LEFT is held **and** we are not already at the left edge, move one column left. The `x > 0` test keeps the @ on the screen.
9	`    IF BUTTON(BTN_RIGHT) AND x < 39 THEN x += 1`	The same for right (the last column is 39)...
10	`    IF BUTTON(BTN_UP) AND y > 4 THEN y -= 1`	...up (rows 0-3 hold the instructions, so we stop at row 4)...
11	`    IF BUTTON(BTN_DOWN) AND y < 27 THEN y += 1`	...and down.
12	`    IF PRESSED(BTN_A) THEN presses += 1`	PRESSED is true only once per press: holding A down counts as one press. Try changing it to BUTTON and hold A - the counter races!
13	`    PRINT AT x, y, "@"`	Draw the @ at its (maybe new) position.
14	`    PRINT AT 1, 3, "A PRESSED "; presses; " TIMES"`	Show the counter.
15	`    WAIT 3`	Slow down: 3 frames per step, 20 steps per second.
16	`LOOP`	Repeat forever.

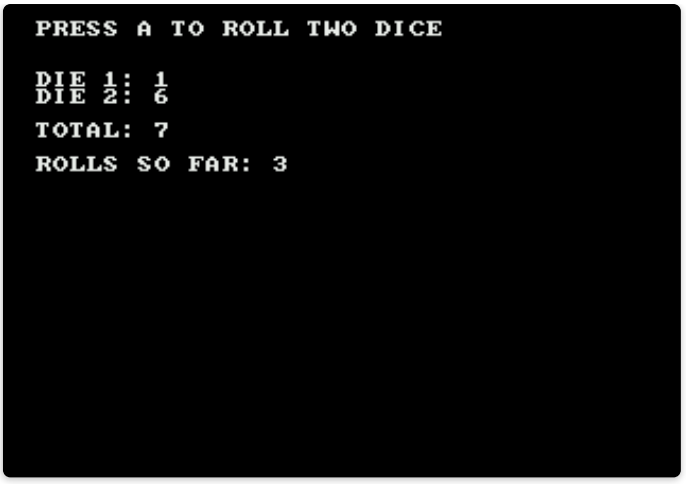


Another example: rolling dice

02b_dice.bas

```
' Rolling dice with random numbers
DIM die1, die2, total, rolls
PRINT AT 2, 1, "PRESS A TO ROLL TWO DICE"
DO
    IF PRESSED(BTN_A) THEN
        RANDOMIZE TIMER
        die1 = RND(6) + 1
        die2 = RND(6) + 1
        total = die1 + die2
        rolls = rolls + 1
        PRINT AT 2, 4, "DIE 1: "; die1
        PRINT AT 2, 5, "DIE 2: "; die2
        PRINT AT 2, 7, "TOTAL: "; total; " "
        PRINT AT 2, 9, "ROLLS SO FAR: "; rolls
    END IF
    VSYNC
LOOP
```

Two dice, their total, and how often we rolled.	
Instructions.	
The game loop.	
Everything until END IF happens once per press of A.	
Computers cannot really throw dice: RND produces numbers from a fixed recipe. `RANDOMIZE TIMER` mixes in the exact frame number at which you pressed the button - something nobody can predict - so the results are different every time.	
`RND(6)` gives 0 to 5. Adding 1 gives 1 to 6, like a real die.	
The second die.	
Add them up.	
Count the roll.	
Show the results...	
The second die's value.	
...the space after the total erases an old second digit (12 followed by 7 would otherwise show 72).	
How many rolls so far.	
End of the actions for one press.	
Wait for the next frame. Without VSYNC, PRESSED would never see a new frame.	
Forever.	



Tip To check two buttons at once use OR: `BUTTON(BTN_A OR BTN_B)` is true if A or B is held. For the second joypad add a 2: `BUTTON(BTN_LEFT, 2)`.

Exercise. Change 06_joypad.bas so that button B clears the screen (`CLS`).
Answer: Add `IF PRESSED(BTN_B) THEN CLS` inside the loop. (The instructions disappear too - print them again after CLS if you like.)

11 SUBs and FUNCTIONs: your own commands

When programs grow, they need structure. SUBs and FUNCTIONs let you invent your own commands, with names you choose.

Imagine you need to draw a frame around text in five places. Instead of writing the same ten lines five times, you write them once, give them a name - `DrawBox` - and from then on `DrawBox` is a new command. That is a **SUB** (subroutine). A **FUNCTION** is the same, but it calculates an answer and gives it back, like `RND` or `ABS`.

09_subs.bas

```
' SUBs and FUNCTIONs
DIM total
```

A **global** variable. Variables created with DIM in the main program can be used everywhere - in the main program and inside all SUBs and FUNCTIONs.

```

SUB DrawBox(x, y, w, h)
```

Here the definition of a new command begins. `DrawBox` is its name. Inside the brackets are its **parameters**: four variables that receive values each time the SUB is used (position x, y; width w; height h).

```
    ' draws a frame with characters
```

A comment describing the SUB. VS Code shows it when you hover over the name anywhere in your program.

```
    FOR i = 0 TO w - 1
```

Draw the top and bottom edge, w characters long. Note: `i` is a **local** variable - it only exists inside this SUB. Even if the main program has its own `i`, they don't disturb each other.

```
        PRINT AT x + i, y, "-"
        PRINT AT x + i, y + h - 1, "-"
```

A dash on the top edge...

...and one on the bottom edge (h rows lower).

```
    NEXT
    FOR i = 1 TO h - 2
```

End of the first FOR loop.

Then the left and right edges, between the corners.

```
        PRINT AT x, y + i, "!"
        PRINT AT x + w - 1, y + i, "!"
```

The left edge...

...and the right edge.

```
    NEXT
END SUB
```

End of the second FOR loop.

End of the definition. Important: defining a SUB does not run it! It is only a recipe that waits until someone calls it.

```

FUNCTION Square(n)
```

A FUNCTION with one parameter `n`.

```
    RETURN n * n
```

`RETURN` gives the answer back to whoever called the function, and leaves the function immediately.

```
END FUNCTION
```

End of the function definition.

```

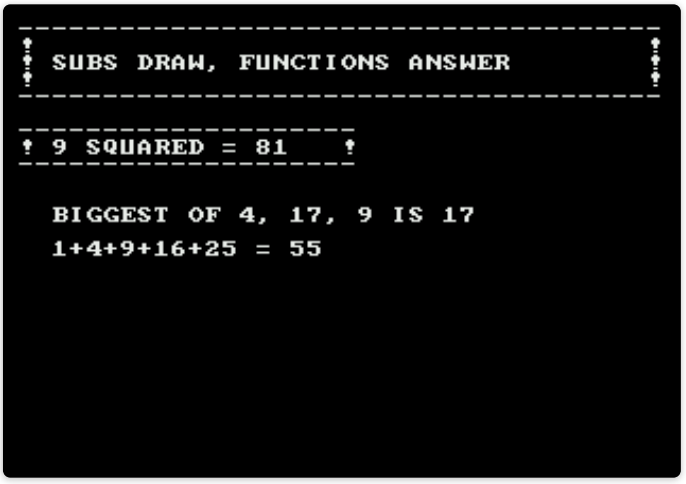
FUNCTION Biggest(a, b, c)
```

A function with three parameters that finds the largest of three numbers.

```
    LOCAL best
```

`LOCAL` declares a variable that belongs only to this function. (Variables first used inside a SUB/FUNCTION are local automatically - LOCAL just makes it clear.)

22	`    best = a`	Start by assuming a is the biggest...
23	`    IF b > best THEN best = b`	...if b is bigger, b is the new best...
24	`    IF c > best THEN best = c`	...and the same for c.
25	`    RETURN best`	Give back the result.
26	`END FUNCTION`	End of Biggest.
27		
28	`DrawBox 1, 1, 38, 5`	**The main program starts here.** We **call** our SUB: its name, then the values for the parameters in the same order. Inside DrawBox, x = 1, y = 1, w = 38, h = 5. You can also write `DrawBox(1, 1, 38, 5)`.
29	`PRINT AT 3, 3, "SUBS DRAW, FUNCTIONS ANSWER"`	Text inside the box.
30	`DrawBox 1, 7, 20, 3`	Use the SUB again, with different values - that is the whole point.
31	`PRINT AT 3, 8, "9 SQUARED = "; Square(9)`	A function is used *inside* an expression, with its values in brackets. `Square(9)` is replaced by its answer, 81.
32	`PRINT AT 3, 12, "BIGGEST OF 4, 17, 9 IS "; Biggest(4, 17, 9)`	Prints 17.
33	`FOR k = 1 TO 5`	Functions can be used in loops and calculations like any built-in function...
34	`    total = total + Square(k)`	...this adds 1 + 4 + 9 + 16 + 25 = 55 into the global `total`.
35	`NEXT`	Next k, until k has been 5.
36	`PRINT AT 3, 14, "1+4+9+16+25 = "; total`	Shows the result.



Global and local - who can see a variable?

Kind	Created by	Visible	Lifetime
global	`DIM` in the main program, or first used in the main program	everywhere	the whole game
local	parameters, `LOCAL`, or first used inside a SUB/FUNCTION	only inside that SUB/FUNCTION	one call; starts at 0 every call

Watch out If a SUB should change the score, the score must be a global: write `DIM score` near the top of the main program. Otherwise the SUB gets its own local `score` that is forgotten when the SUB ends.

Good programs consist mostly of small SUBs with clear names. The main loop of the complete games later in this book is just a list of calls: `MovePlayer`, `MoveEnemies`, `DrawScore` ... Reading it tells you what the game does; the details are inside the SUBs.

Exercise. Write a FUNCTION `Twice(n)` that returns n * 2, and print `Twice(21)`.
Answer: `FUNCTION Twice(n)` / `RETURN n * 2` / `END FUNCTION` / `PRINT Twice(21)`

12 Arrays: many values, one name

A game with 20 enemies needs 20 x positions. Twenty variables x1, x2 ... x20 would be awful. An array is a numbered row of boxes with one name.



The array scores has five boxes, numbered 0 to 4.

12b_arrays_text.bas

```
1  ' Arrays: five scores in one variable
2  DIM scores(4) = {120, 85, 300, 42, 210}
3  DIM best, total, i
4
5  PRINT "THE SCORES:"
6  FOR i = 0 TO 4
7      PRINT "  PLAYER "; i + 1; ": "; scores(i)
8  NEXT
9
10 best = scores(0)
11 total = 0
12 FOR i = 0 TO 4
13     IF scores(i) > best THEN best = scores(i)
14     total = total + scores(i)
15 NEXT
16 PRINT
17 PRINT "BEST SCORE:    "; best
18 PRINT "TOTAL:         "; total
19 PRINT "AVERAGE:       "; total / 5
20
21 scores(3) = 999
22 PRINT "PLAYER 4 NOW:  "; scores(3)
```

`DIM scores(4)` creates an array with boxes numbered **0 to 4** - that is 5 boxes. The list in curly brackets `{ }` fills them with starting values (without it they all start at 0).

Normal variables.

A heading.

Arrays and FOR loops are best friends: i goes through all box numbers 0 to 4.

`scores(i)` means *the box with the number i*. The number in brackets is called the **index**. With i = 2 it is scores(2), which holds 300.

Back for the next player.

To find the best score, start with the first one...

...and start the sum at 0.

Look at every score:

if this one is better than the best so far, remember it;

and add it to the total.

End of the search loop. After it, best holds the largest score.

Empty line.

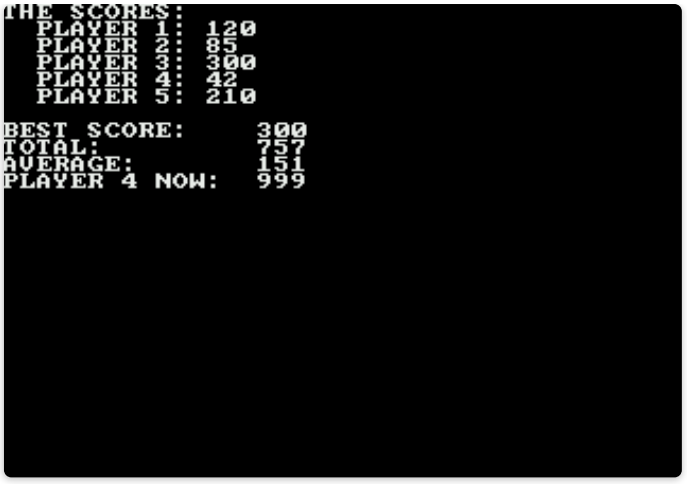
300.

757.

757 / 5 = 151 (whole numbers).

Change one box: the fourth box (index 3) gets 999.

Proof.



Arrays with two dimensions

A board game or a level map is a grid: rows and columns. Give an array two sizes:

```
DIM board(7, 7)            ' an 8 x 8 chess board: board(0,0) ... board(7,7)
board(3, 4) = 1            ' put something at column 3, row 4
IF board(x, y) = 0 THEN PRINT "EMPTY"
```

The Snake game in chapter 24 uses a 2-dimensional array to remember what is in each cell of the playfield.

Smaller boxes

Normal boxes hold big numbers and use 4 bytes of memory each. If you only store small numbers, use `AS BYTE` (0 to 255, 1 byte each) or `AS WORD` (-32768 to 32767, 2 bytes): `DIM level(39, 27) AS BYTE`. The Mega Drive has 64 KB of memory - plenty, but big arrays add up.

> **Watch out** The index must stay inside the array: with `DIM scores(4)`, `scores(5)` does not exist. MegaBASIC does not check this while the game runs (it would slow everything down) - writing outside an array silently overwrites other variables. Double-check your FOR loop limits.

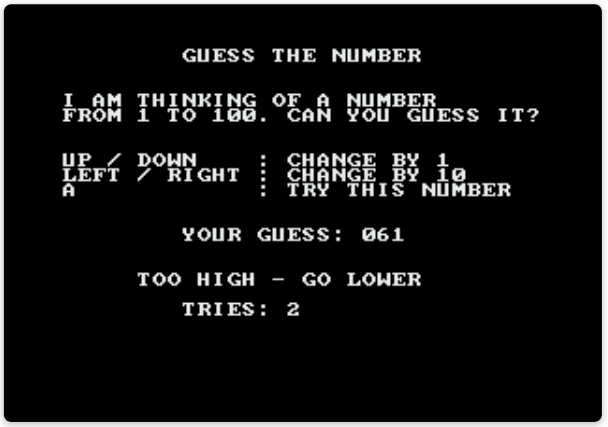
Exercise. Make an array with the 7 days' temperatures and print the coldest.
Answer: Like the best-score loop, but with `<`: `IF t(i) < coldest THEN coldest = t(i)`.

13 Your first game: Guess the Number

Time to put everything together. The console thinks of a secret number from 1 to 100 and you have to find it. It only uses text - but it is a real game.



The welcome screen



Playing: "too high" after the second try

Plan first

Before writing a game, describe it in plain words. This is called **planning**, and even professional developers start this way:

1. Show a welcome screen and wait for START.
2. Pick a secret random number from 1 to 100.
3. Let the player change a guess with the D-pad (UP/DOWN by 1, LEFT/RIGHT by 10) and confirm it with A.
4. After each try say "too high" or "too low" and count the tries.
5. When the guess is right, congratulate, wait, and start again.

Then list what the game must remember - those become the variables: the secret number, the current guess, the number of tries, and whether the round is won.

The program, part 1: sounds and variables

guess.bas

6	`TITLE "GUESS THE NUMBER"`	The game's name for the ROM header.
7		
8	`SFX beep`	`SFX` starts the definition of a **sound effect** called `beep`. Chapter 22 explains sound in detail; for now, just know that the text in quotes describes notes.
9	`    PSG3 "@short v12 o5 l32 c"`	`PSG3` is one of the sound channels. `@short` is a short, clicky sound; `c` is the note C.
10	`END SFX`	End of the sound effect.
11		
12	`SFX wrong`	A second effect: two low notes falling (C, then G below it).
13	`    PSG3 "@short v13 o3 l16 c <g"`	Two short notes: C in octave 3, then `<g` = the G below it.
14	`END SFX`	End of the 'wrong' sound.

15		
16	`SFX win`	A happy rising run of notes for the winner.
17	`    PSG3 "@pluck v15 o5 l16 c e g >c e g >c4"`	`@pluck` fades each note out like a plucked string; the notes climb up two octaves.
18	`END SFX`	End of the 'win' sound.
19		
20	`DIM secret, guess, tries, won`	The four things the game remembers: the secret number, the player's current guess, how many tries so far, and whether the round is won (TRUE/FALSE).

Part 2: the welcome screen

guess.bas

22	`DO`	The **outer loop**: one round of the whole game. When a round ends we come back here and start a new round. It never ends.
23	`    ' ----- welcome screen -----`	
24	`    CLS`	`CLS` (CLear Screen) removes all text from the screen.
25	`    PRINT AT 12, 3, "GUESS THE NUMBER"`	The welcome text. The x positions are chosen so that each line is roughly centred.
26	`    PRINT AT 4, 6, "I AM THINKING OF A NUMBER"`	What the game is about...
27	`    PRINT AT 4, 7, "FROM 1 TO 100. CAN YOU GUESS IT?"`	...in two lines.
28	`    PRINT AT 4, 10, "UP / DOWN    : CHANGE BY 1"`	The instructions: which buttons do what.
29	`    PRINT AT 4, 11, "LEFT / RIGHT : CHANGE BY 10"`	LEFT/RIGHT change the guess by 10.
30	`    PRINT AT 4, 12, "A            : TRY THIS NUMBER"`	A confirms the guess.
31	`    PRINT AT 10, 22, "PRESS START TO PLAY"`	Tell the player how to start.
32	`    DO`	An **inner loop** that just waits...
33	`        VSYNC`	...one frame at a time...
34	`    LOOP UNTIL PRESSED(BTN_START)`	...until START is pressed.

Part 3: a new round

guess.bas

37	`    RANDOMIZE TIMER`	Mix the exact moment START was pressed into the random number generator, so the secret number is different every game.
38	`    secret = RND(100) + 1`	RND(100) gives 0 to 99; plus 1 gives 1 to 100. This is the number to find.
39	`    guess = 50`	The player's guess starts in the middle.
40	`    tries = 0`	No tries yet. Important: these must be reset every round, otherwise the second game would continue counting.
41	`    won = FALSE`	Not won yet.
42	`    PRINT AT 10, 22, "                  "`	Erase the "PRESS START" line by printing spaces over it.

Part 4: the guessing loop

guess.bas

44	`' ----- the guessing loop -----`	
45	`DO`	The **game loop** of one round begins. It ends with `LOOP UNTIL won` at the bottom.
46	`    IF PRESSED(BTN_UP) THEN guess += 1`	UP adds 1 to the guess. We use PRESSED, not BUTTON, so each press changes the number by exactly 1.
47	`    IF PRESSED(BTN_DOWN) THEN guess -= 1`	DOWN subtracts 1...
48	`    IF PRESSED(BTN_RIGHT) THEN guess += 10`	...RIGHT adds 10...
49	`    IF PRESSED(BTN_LEFT) THEN guess -= 10`	...LEFT subtracts 10. Fast for big jumps.
50	`    guess = CLAMP(guess, 1, 100)`	Keep the guess between 1 and 100, whatever the player does.
51	`    PRINT AT 12, 15, "YOUR GUESS: "; PAD$(guess, 3)`	Show the current guess. `PAD$(guess, 3)` prints it with 3 digits (005, 050, 100), so a shorter number never leaves old digits behind.
52		
53	`    IF PRESSED(BTN_A) THEN`	A = "try this number". Everything until the matching END IF happens once per press.
54	`        tries += 1`	Count the try.
55	`        IF guess < secret THEN`	A block IF **inside** another block IF - perfectly fine. Too small?
56	`            PRINT AT 9, 18, "TOO LOW  - GO HIGHER"`	Say so. The spaces at the end make both messages the same length, so one fully covers the other.
57	`            PLAYSFX wrong`	Play the 'wrong' sound effect. It plays by itself while the program continues.
58	`        ELSEIF guess > secret THEN`	Too big?
59	`            PRINT AT 9, 18, "TOO HIGH - GO LOWER "`	The guess was too big.
60	`            PLAYSFX wrong`	Same sound for both kinds of wrong answer.
61	`        ELSE`	Neither smaller nor bigger means: equal! Found it.
62	`            won = TRUE`	Remember that the round is won.
63	`        END IF`	End of the too-low/too-high/correct decision.
64	`        PRINT AT 12, 20, "TRIES: "; tries`	Show the number of tries.
65	`    END IF`	End of 'A was pressed'.
66	`    VSYNC`	One frame done.
67	`LOOP UNTIL won`	Repeat until `won` is TRUE. Because `won` holds TRUE or FALSE, it can be used directly as the question.

Part 5: the winner

guess.bas		
70	`PLAYSFX win`	Fanfare!
71	`PRINT AT 9, 18, "  CORRECT! WELL DONE "`	Overwrite the last hint.
72	`IF tries <= 7 THEN`	With the best strategy you never need more than 7 tries (see below)...
73	`    PRINT AT 7, 24, "AMAZING - A REAL EXPERT!"`	...so 7 or fewer earns praise...
74	`ELSE`	Otherwise (8 tries or more)...
75	`    PRINT AT 7, 24, "TIP: TRY THE MIDDLE NUMBER"`	...otherwise a hint.
76	`END IF`	End of the praise decision.
77	`WAIT 180`	Show the result for 3 seconds (180 frames).

The very last line, `LOOP`, jumps back to the first `DO`: welcome screen, new number, new round.

Good to know The smart strategy is to always guess the middle of what is still possible: 50, then 25 or 75, and so on. Each guess halves the possibilities: 100, 50, 25, 13, 7, 4, 2, 1 - at most 7 tries. Computer scientists call this a **binary search** .

Make it yours

- Change the range to 1 - 1000 (how many tries does the best strategy need now?).
- Give the player only 10 tries, then show "YOU LOSE - IT WAS ...".
- Keep a best score (fewest tries) across rounds in a new variable.
- Make a two-player version: player 1 chooses the secret number with the D-pad while player 2 looks away.

Part III — Graphics and sound

Pictures from PNG files, sprites, animation, collisions, smooth movement, levels drawn in Tiled, scrolling, music and sound effects.

14 How pictures work on the Mega Drive

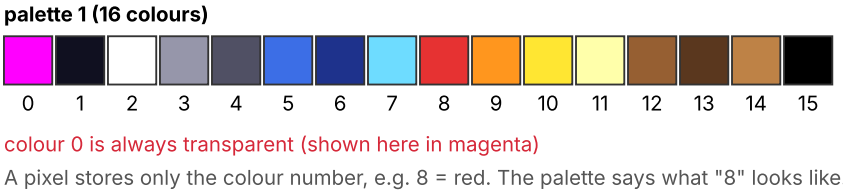
Before we show our first sprite, let's understand how a picture gets from your paint program into the console.

Tiles

Everything you see is made of **tiles**: small squares of 8 × 8 pixels. A 16 × 16 pixel character is made of 4 tiles (2 across, 2 down); a 32 × 32 boss of 16 tiles. The tiles are stored in the video chip's own memory, called **VRAM** (Video RAM). Before a picture can be shown, it must be copied into VRAM - that is what the command `LOADTILES` does.

Palettes and the 16-colour rule

Each tile stores, for each of its 64 pixels, a colour *number* from 0 to 15 - not the colour itself. The actual colours come from a **palette**: a list of 16 colours. Change the palette and the picture changes colour instantly; old games used this for blinking lights, sunsets and "power-up" flashes. The console has 4 palettes (numbered 0 to 3), and each sprite or tile uses one of them.

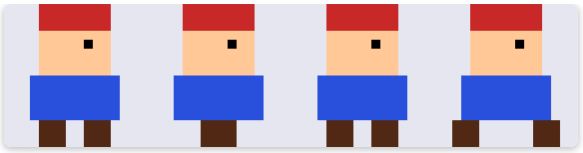


A palette: colour numbers 0-15.

Colour number 0 is special: it is **transparent**. Wherever a sprite has colour 0, you see whatever is behind it. This is how a character can have any shape instead of being a square.

Drawing your own sprite - step by step

1. Open a pixel-art program: **Aseprite**, **LibreSprite** (free), **Piskel** (free, in the browser), GIMP or even Paint.
2. Create a new image of **16 × 16 pixels** (or 32 × 16 for two animation frames side by side, 64 × 16 for four...). Zoom in a lot.
3. Give it a **transparent background**. If your program has no transparency, fill the background with pure magenta: red 255, green 0, blue 255 (#FF00FF). MegaBASIC treats magenta as transparent.
4. Draw with **15 colours or fewer** (the 16th is the transparent background). Fewer colours often look better anyway.
5. Save as **PNG** in the same folder as your `.bas` file. (Do not use JPG - it blurs pixels and adds hundreds of colours.)



hero.png, enlarged 8 times: four 16 × 16 frames of a walking boy

Frames

A **sprite sheet** is one PNG with several pictures - frames - in a row (or in several rows). MegaBASIC cuts it into frames and numbers them from 0, left to right, top to bottom. You say how big one frame is, in tiles: `SPRITES hero = "hero.png", 2, 2` means each frame is 2 tiles wide and 2 tiles high (16 × 16 pixels).

Frame size in pixels	Write
8 × 8	`1, 1`
16 × 16	`2, 2`
16 × 32 (tall)	`2, 4`
32 × 8 (wide, e.g. a bat)	`4, 1`
32 × 32 (biggest)	`4, 4`

Good to know The conversion from PNG to Mega Drive format happens when you **build** . The finished ROM contains the converted tiles - the PNG file is not needed to play the game.

15 Your first sprite

Let's put a character on the screen and move it with the joypad.

07_sprite.bas

```
1  ' A sprite that moves with the joypad
2  SPRITES hero = "hero.png", 2, 2   ' frames of 2x2 tiles = 16x16 pixels
3
4  DIM px, py
5
6  LOADTILES hero                    ' copy the picture to video memory
7  LOADPAL hero, 1                   ' its colours go to palette 1
8  SPRITEPAL 0, 1                    ' sprite 0 uses palette 1
9  px = 152
10 py = 100
11 DO
12     IF BUTTON(BTN_LEFT) THEN px -= 2
13     IF BUTTON(BTN_RIGHT) THEN px += 2
14     IF BUTTON(BTN_UP) THEN py -= 2
15     IF BUTTON(BTN_DOWN) THEN py += 2
16     SPRITE 0, px, py, hero, 0     ' sprite 0 shows frame 0 of hero
17     VSYNC
18 LOOP
```

Import the picture `hero.png` as a **sprite sheet** named `hero`. Frames are 2 × 2 tiles = 16 × 16 pixels. This line is not a command that "runs"; it tells the compiler to convert and include the picture.

The sprite's position on the screen in pixels: `px` (x) and `py` (y).

Copy all frames of `hero` into video memory. This must happen before the sprite can appear. It takes a split second, so do it once at the start - not every frame.

Copy the picture's colours into **palette 1**. We use palette 1 for sprites and keep palette 0 for backgrounds later.

Tell **sprite number 0** to use palette 1. The console has 80 sprites, numbered 0 to 79; we use number 0 for our hero.

Start in the middle: the screen is 320 wide, the sprite 16, so (320 - 16) / 2 = 152...

...and roughly in the middle vertically.

The game loop.

LEFT held: move 2 pixels left. Remember: x gets *smaller* to the left.

RIGHT held: 2 pixels right.

UP: y gets smaller, because y counts downwards from the top.

DOWN: y gets bigger.

The key command: `SPRITE number, x, y, sheet, frame`. Show sprite 0 at position (px, py) with frame 0 of the sheet `hero`. The sprite size (16 × 16) comes from the sheet automatically.

Wait for the next frame. The new sprite position is sent to the video chip now - so the sprite moves smoothly, 60 times a second.

And again.



Things to know about sprites

- A sprite stays where you last put it until you move it again or hide it with `SPRITEOFF number`. `CLEARSPRITES` hides all of them.
- When sprites overlap, the **lower number is in front**. Give important things - the player - low numbers.
- Sprites may be partly outside the screen; that is how things slide in from the edges. Sprites far outside are hidden automatically.
- At most 20 sprites can share one horizontal line of the screen. If more are on the same line, some vanish for that line - a famous limit of 16-bit consoles.

Exercise. Make the hero move twice as fast.
Answer: Change `2` into `4` in the four movement lines - or better, add `CONST SPEED = 4` and write `px -= SPEED`.

Exercise. Stop the hero from leaving the screen.
Answer: Add `px = CLAMP(px, 0, 304)` and `py = CLAMP(py, 0, 208)` before the SPRITE line (320-16 = 304, 224-16 = 208).

16 Animation

A drawing that moves is a series of slightly different pictures shown one after another. Our hero.png already contains four walking frames.

08_animation.bas

```
' Walking animation and flipping
SPRITES hero = "hero.png", 2, 2

DIM px, frame, walking, facingLeft

LOADTILES hero
LOADPAL hero, 1
SPRITEPAL 0, 1
SETCOLOR 0, 0, RGB(60, 120, 200)
BACKDROP 0, 0
px = 152
DO
    walking = FALSE
    IF BUTTON(BTN_LEFT) THEN px -= 2 : walking = TRUE : facingLeft = TRUE
    IF BUTTON(BTN_RIGHT) THEN px += 2 : walking = TRUE : facingLeft = FALSE
    IF walking THEN
        frame = (TIMER >> 3) AND 3    ' a new frame every 8 VSYNCs: 0,1,2,3,0...
    ELSE
        frame = 0
    END IF
    SPRITEFLIP 0, facingLeft, FALSE
    SPRITE 0, px, 120, hero, frame
    VSYNC
LOOP
```

The same sheet as before, with frames 0, 1, 2 and 3.

`frame`: which picture to show. `walking`: TRUE while a direction is held. `facingLeft`: which way the hero looks.

Graphics into VRAM...

...colours into palette 1...

...sprite 0 uses palette 1.

`SETCOLOR palette, colourNumber, colour` changes one colour of a palette. `RGB(red, green, blue)` mixes a colour from three amounts between 0 and 255 - here a medium blue.

The **backdrop** is the colour behind everything. We choose colour 0 of palette 0 - the blue we just set.

Start in the middle.

Game loop.

At the start of every frame we assume the hero stands still...

...if LEFT is held, move left, and remember that we are walking and facing left. Three statements after THEN, joined with `:` - all three happen only when LEFT is held.

The same for RIGHT, facing right.

Choose the frame:

TIMER counts frames: 0, 1, 2, 3... `>> 3` divides by 8 (it shifts the number's bits 3 places), so the result grows by 1 every 8 frames. `AND 3` keeps only the values 0-3 and starts again: 0, 1, 2, 3, 0, 1... A new walking picture every 8/60 of a second.

Not walking...

...show the standing picture.

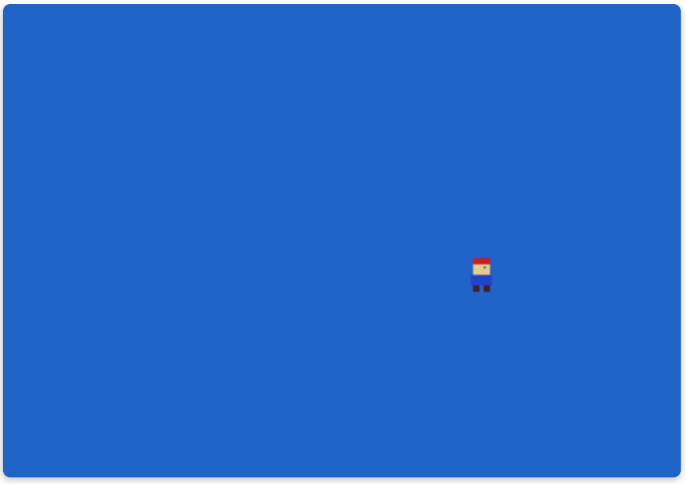
End of the frame choice.

`SPRITEFLIP number, horizontal, vertical` mirrors the sprite. Our frames all look right; when `facingLeft` is TRUE the console mirrors them for free - no need to draw left-facing frames.

Show the chosen frame at height 120.

Next frame.

Forever.



Try it Animation speed is easy to change: `>> 2` = new frame every 4 frames (fast), `>> 4` = every 16 frames (slow).

17 Many sprites with arrays

One sprite was fun. Now twenty - and thanks to arrays and loops, twenty is no more work than one.

10_arrays.bas

```
' Arrays: twenty bouncing stars
SPRITES gfx = "sprites.png", 2, 2
CONST COUNT = 20, F_STAR = 2

DIM sx(COUNT), sy(COUNT), vx(COUNT), vy(COUNT)

LOADTILES gfx
LOADPAL gfx, 1
FOR i = 1 TO COUNT
    sx(i) = RND(300)
    sy(i) = RND(200)
    vx(i) = RND(5) - 2       ' -2 .. 2
    vy(i) = RND(5) - 2
    IF vx(i) = 0 THEN vx(i) = 1
    SPRITEPAL i, 1
NEXT
DO
    FOR i = 1 TO COUNT
        sx(i) += vx(i)
        sy(i) += vy(i)
        IF sx(i) < 0 OR sx(i) > 304 THEN vx(i) = -vx(i)
        IF sy(i) < 0 OR sy(i) > 208 THEN vy(i) = -vy(i)
        SPRITE i, sx(i), sy(i), gfx, F_STAR
    NEXT
    VSYNC
LOOP
```

This sheet (from the Star Catcher game) has 8 frames: 0-1 ship, 2-3 star, 4-5 rock, 6-7 explosion.

Two constants: how many stars we want, and the frame number of the star picture. Names are easier to read than bare numbers.

Four arrays with boxes 0 to 20: x position, y position, x speed and y speed of every star. Star number i uses box i of every array.

Graphics...

...and colours.

Prepare stars 1 to 20 (we leave box 0 unused, so star numbers and sprite numbers match):

a random x between 0 and 299,

a random y,

a random x speed: RND(5) is 0-4, minus 2 gives -2 to 2,

the same for y,

but never 0 for x (the star would only move up and down),

and sprite number i uses palette 1.

Next star.

Game loop.

Every frame we go through all 20 stars:

move horizontally,

move vertically,

bounce off the left/right edges by reversing the x speed,

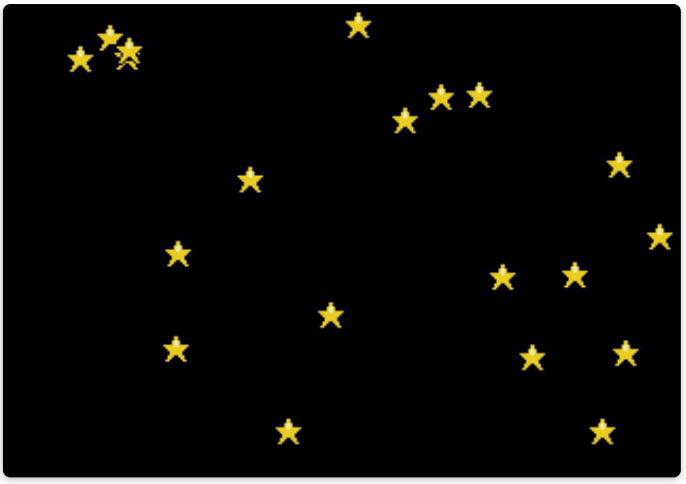
bounce off the top/bottom edges,

and draw star i with sprite number i.

Next star.

One frame done - all 20 stars appear at their new places together.

Forever.

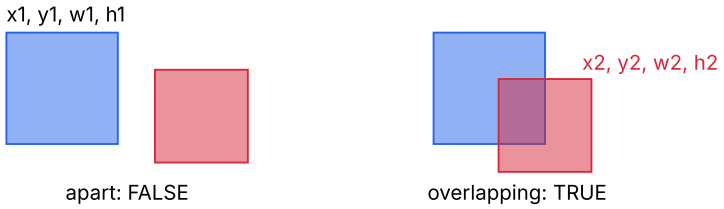


Try it Want 60 stars? Change `COUNT = 20` to 60. That is the power of constants and arrays. (Watch the 80-sprite and 20-per-line limits.)

18 Collisions and score

Games are about things touching: the ball hits the bat, the hero grabs a coin, a bullet hits an alien. Checking this is called collision detection.

MegaBASIC's `COLLIDE` function compares two rectangles. You give it the x, y, width and height of each, and it answers TRUE if they overlap:



COLLIDE(x1, y1, w1, h1, x2, y2, w2, h2)

11_collide.bas

```
' Collecting stars: collisions and score
SPRITES gfx = "sprites.png", 2, 2
```
The same sprite sheet: frame 0 is a ship, frame 2 a star.

```
CONST F_SHIP = 0, F_STAR = 2
```
Names for the two frames we use.

```

SFX ding
```
A sound effect called `ding` (chapter 22 explains sounds).

```
    PSG3 "@short v15 o6 l32 c g >c"
```
Three quick notes going up: C, G and the next C.

```
END SFX
```
End of the effect.

```

DIM px, py, starX, starY, score
```
Ship position, star position and the score.

```

SUB NewStar()
```
A SUB that moves the star to a new random place. A SUB without parameters still needs its brackets `()` here.

```
    starX = 16 + RND(288)
```
x between 16 and 303 - away from the screen edges...

```
    starY = 32 + RND(176)
```
...and y between 32 and 207 (the top rows are for the score).

```
END SUB
```
End of NewStar.

```

LOADTILES gfx
LOADPAL gfx, 1
SPRITEPAL 0, 1
SPRITEPAL 1, 1
px = 152 : py = 180
NewStar
DO
    IF BUTTON(BTN_LEFT) THEN px -= 3
    IF BUTTON(BTN_RIGHT) THEN px += 3
    IF BUTTON(BTN_UP) THEN py -= 3
    IF BUTTON(BTN_DOWN) THEN py += 3
    px = CLAMP(px, 0, 304)
```
Usual setup: graphics, colours, sprite 0 (ship) and...sprite 1 (star) use palette 1. Ship starts at the bottom centre. Place the first star by calling our SUB. Game loop. Steer the ship in four directions, 3 pixels per frame. Right... Up... Down. Keep the ship on screen horizontally...

28	`    py = CLAMP(py, 16, 208)`	...and vertically (below the score line).
29	`    IF COLLIDE(px, py, 16, 16, starX, starY, 16, 16) THEN`	The collision test: ship rectangle (px, py, 16 × 16) against star rectangle (starX, starY, 16 × 16).
30	`        score += 1`	Touching! One point...
31	`        PLAYSFX ding`	...a ding...
32	`        NewStar`	...and a new star somewhere else.
33	`    END IF`	End of the collision block.
34	`    SPRITE 0, px, py, gfx, F_SHIP`	Draw the ship...
35	`    SPRITE 1, starX, starY, gfx, F_STAR`	...and the star.
36	`    PRINT AT 1, 0, "STARS: "; score`	Show the score in the top row.
37	`    VSYNC`	Next frame.
38	`LOOP`	Forever.



Tip Pictures rarely fill their whole 16 × 16 square. For fair collisions use a slightly smaller rectangle: `COLLIDE(px + 2, py + 2, 12, 12, ...)`. Players forgive near misses - but never hits that "didn't touch".

19 Smooth movement and gravity

Moving 1 pixel per frame is 60 pixels per second; 2 pixels is 120. What about 1.5? And how do we make things fall like in real life?

The 1/16 pixel trick

MegaBASIC has only whole numbers. The trick: store positions in **sixteenths of a pixel**. A position of 160 means 160 / 16 = pixel 10. A speed of 24 sixteenths per frame is 1.5 pixels per frame. When drawing, divide by 16. The fastest way to divide by 16 is `>> 4` (shift the bits 4 places right) - you will see it in every action game in this book.

Stored value	Meaning
16	1 pixel
8	half a pixel
24	1.5 pixels
320 * 16 = 5120	pixel 320
`x >> 4`	x in whole pixels

Gravity

A thrown ball does not move at a constant speed: gravity pulls it down a bit more every moment. In a program: every frame, add a little to the downward speed. That one line makes jumps and falls look natural.

19_gravity.bas

```
' Smooth movement: a bouncing ball with gravity
SPRITES gfx = "sprites.png", 2, 2
CONST F_ROCK = 4
CONST GRAVITY = 4                ' added to the falling speed every frame
CONST FLOOR = 200                ' the ball lands here (in pixels)

DIM x, y, vx, vy                 ' position and speed, in 1/16 pixels

LOADTILES gfx
LOADPAL gfx, 1
SPRITEPAL 0, 1
PRINT AT 1, 1, "PRESS A TO THROW THE BALL AGAIN"
x = 20 * 16
y = 20 * 16
vx = 24                          ' 24/16 = 1.5 pixels per frame to the right
DO
    vy = vy + GRAVITY            ' gravity: fall a little faster every frame
```

We borrow the rock picture from the Star Catcher sheet as a ball.

Frame 4 = rock.

How much faster the ball falls each frame: 4/16 pixel per frame, every frame.

The ball bounces at y = 200.

Position (x, y) and speed (vx, vy), all in 1/16 pixels. *v* stands for velocity, the scientific word for speed with a direction.

Usual setup.

Colours into palette 1.

Sprite 0 uses palette 1.

Instructions.

Start at pixel (20, 20) - multiplied by 16 because we store sixteenths.

Same for y.

Moving right at 24/16 = 1.5 pixels per frame. vy starts at 0: not moving vertically yet.

Game loop.

Gravity! The downward speed grows every frame: 0, 4, 8, 12... the ball falls faster and faster.

18	`    x = x + vx`	Move horizontally...
19	`    y = y + vy`	...and vertically.
20	`    IF (y >> 4) > FLOOR THEN      ' hit the floor?`	Below the floor? `y >> 4` converts to pixels before comparing with 200.
21	`        y = FLOOR * 16`	Put the ball exactly on the floor...
22	`        vy = -vy * 3 / 4          ' bounce up, losing a quarter of the speed`	...and reverse the speed so it goes up again, but only with 3/4 of the speed. The ball loses energy and every bounce is lower - like a real ball.
23	`    END IF`	End of the floor check.
24	`    IF (x >> 4) > 304 OR (x >> 4) < 0 THEN vx = -vx`	Bounce off the side walls.
25	`    IF PRESSED(BTN_A) THEN`	Press A to lift the ball up again:
26	`        y = 20 * 16`	back to the top,
27	`        vy = 0`	with no vertical speed.
28	`    END IF`	End of the A button block.
29	`    SPRITE 0, x >> 4, y >> 4, gfx, F_ROCK`	Draw - converting from sixteenths to pixels.
30	`    VSYNC`	Next frame.
31	`LOOP`	Forever.



Four moments of the bouncing ball shown together

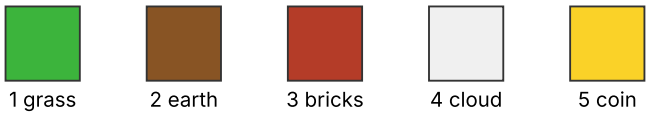
Good to know A **jump** in a platform game is the same physics: when the player presses jump, set `vy` to a negative number (upwards). Gravity slows the rise, stops it at the top and pulls the player back down. Coin Quest (chapter 27) does exactly this.

20 Backgrounds and levels with Tiled

Sprites are for moving things. Landscapes, walls and levels are drawn on the background planes from tiles - and the free Tiled map editor makes this easy.

The idea

A level is a grid. Each cell holds a *tile number*: 0 = nothing, 1 = grass, 2 = earth, 3 = bricks... The pictures for the numbers come from a **tileset**: one PNG containing all tile pictures side by side. Our `tiles.png` has five 16 × 16 tiles:



The five tiles of tiles.png and their Tiled numbers

Making a map in Tiled - step by step

1. Download Tiled from **mapeditor.org** (free) and start it.
2. **File > New > New Map**. Orientation: *Orthogonal*. Tile layer format: *CSV*. Tile size: **16 × 16** (the size of your tiles). Map size: for example 64 × 14 tiles. (The screen shows 20 × 14 tiles of 16 pixels.)
3. **Map > New Tileset**: choose your PNG (e.g. `tiles.png`), tile width/height 16, margin 0, spacing 0. Click *Save As* to keep the tileset with the map.
4. Click a tile in the Tileset panel, then paint in the map with the Stamp tool (B). Use the Eraser (E) for empty cells. X and Y flip the selected tile - MegaBASIC keeps flips.
5. When finished: **File > Export As**, choose the file type **CSV files (*.csv)**, and save it next to your `.bas` file, e.g. `level.csv`.

The CSV file is plain text - just numbers separated by commas, one line per row of the map. You can even open it in Notepad:

One line of level.csv (row 12: the ground)

```
1,1,1,1,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,1,0,0,0,1,1
...
```

12_background.bas

```
' A background drawn in Tiled
TILES ground = "tiles.png", 16      ' 16x16 tiles, same tileset as in Tiled
MAP level = "level.csv", 16         ' the map exported from Tiled as CSV

LOADTILES ground
LOADPAL ground, 0
SETCOLOR 0, 0, RGB(100, 160, 255)  ' sky blue
```

Import the tileset as tiles named `ground`. The `16` says the tiles are 16 × 16 pixels, like in Tiled. MegaBASIC cuts each one into four 8 × 8 hardware tiles. Without the 16, 8 × 8 tiles are assumed.

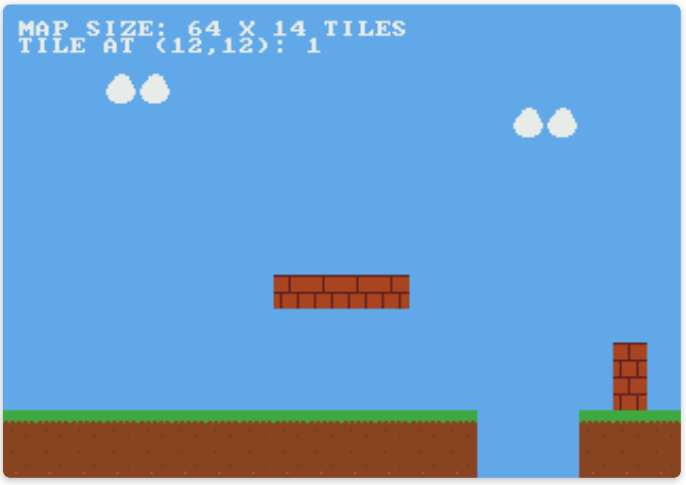
Import the map exported from Tiled, named `level`. Its tile size (16) must match the tileset.

Tiles into video memory. MegaBASIC remembers where they went in `ground.BASE` (you rarely need it).

Tile colours into palette 0.

Colour 0 is the transparent colour - where no tile is drawn we see the backdrop. Make it sky blue...

8	`BACKDROP 0, 0`	...and use it as the backdrop.
9	`DRAWMAP level, PLANE_B, ground, 0  ' draw the map on the back plane`	Draw the whole map onto plane B (the back plane) using the tiles `ground` and palette 0. One command!
10	`TEXTPAL 3`	Text is drawn with colour 15 of a palette. We replaced palette 0 with our tile colours, so for text we switch to palette 3, which still has the original colours (colour 15 = white).
11	`PRINT AT 1, 1, "MAP SIZE: "; level.WIDTH; " X "; level.HEIGHT; " TILES"`	`level.WIDTH` and `level.HEIGHT` are the map's size in Tiled tiles (64 and 14). Words after a dot are **properties** of an asset.
12	`PRINT AT 1, 2, "TILE AT (12,12): "; MAPTILE(level, 12, 12)`	`MAPTILE(map, x, y)` tells you which tile is at a position of the map, counted in Tiled tiles. 0 means empty. This is how a game finds out whether the player stands on ground or walks into a wall.



Planes and text

Plane B is behind plane A. Text is printed on plane A, so it appears in front of a map drawn on plane B. Sprites appear in front of both planes (unless you give tiles the PRIORITY flag). For a status bar that never moves even when both planes scroll, there is the **window**: `WINDOWROWS 2` shows the window over the top 2 rows, and `TEXTPLANE WINDOW` prints there.

> **Good to know** You can also put single tiles anywhere with `SETTILE plane, x, y, tile` - the Snake and Breakout games build their playfields this way, without Tiled.

21 Scrolling

Many games have levels wider than the screen. The view follows the player like a camera. On the Mega Drive this is called scrolling, and MegaBASIC needs only one command for it.

Imagine the level as a long painting and the screen as a window you slide along it. The position of the window's left edge in the level is the **camera position**, `camx`. Camera 0 shows the beginning of the level; camera 100 shows the level from pixel 100 onwards.

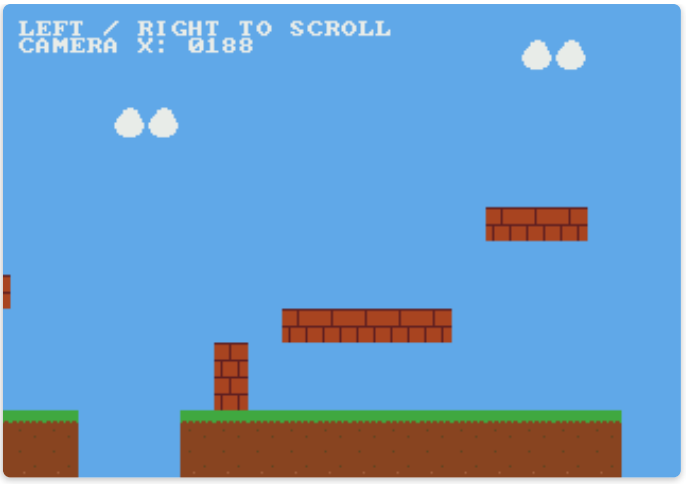
13_scrolling.bas

```
' Scrolling through a map that is wider than the screen
TILES ground = "tiles.png", 16
MAP level = "level.csv", 16

DIM camx, maxcam

LOADTILES ground
LOADPAL ground, 0
SETCOLOR 0, 0, RGB(100, 160, 255)
BACKDROP 0, 0
TEXTPAL 3
maxcam = level.WIDTH * 16 - 320    ' map width in pixels minus screen width
PRINT AT 1, 1, "LEFT / RIGHT TO SCROLL"
DO
    IF BUTTON(BTN_RIGHT) THEN camx += 4
    IF BUTTON(BTN_LEFT) THEN camx -= 4
    camx = CLAMP(camx, 0, maxcam)
    MAPSCROLL level, PLANE_B, ground, 0, camx, 0
    PRINT AT 1, 2, "CAMERA X: "; PAD$(camx, 4)
    VSYNC
LOOP
```

- Same tileset...
- ...and map. The map is 64 tiles x 16 pixels = 1024 pixels wide: more than three screens.
- The camera position, and the largest allowed camera position.
- Usual setup: tiles,
- colours,
- sky,
- backdrop,
- text colour.
- The camera must stop when the right edge of the screen reaches the end of the map: map width in pixels (64 × 16 = 1024) minus screen width (320) = 704.
- Instructions.
- Game loop.
- Move the camera 4 pixels per frame...
- ...in either direction.
- Never before the start or past the end of the map.
- The magic line: `MAPSCROLL map, plane, tiles, palette, cameraX, cameraY`. It scrolls the plane to the camera position and draws the new columns of the map that come into view. Call it once every frame.
- Show the camera position. Text on plane A does not scroll.
- Next frame.
- Forever.



Following the player

In a real game you don't steer the camera - the player moves, and the camera follows. Keep the player in the middle of the screen:

```
camx = CLAMP(playerX - 152, 0, maxcam)
MAPSCROLL level, PLANE_B, ground, 0, camx, 0
SPRITE 0, playerX - camx, playerY, hero, frame
```

Notice the last line: the player's position *in the level* is `playerX`, but on the *screen* it is `playerX - camx`. Everything that lives in the level - enemies, coins - is drawn at its level position minus the camera. Coin Quest (chapter 27) is built exactly like this.

22 Music and sound effects

No game feels alive without sound. The Mega Drive's sound is famous, and MegaBASIC lets you write music as text.

Music basics in two minutes

If you have never read music, here is all you need:

- **Notes** have letter names: C D E F G A B, then C again, one **octave** higher. On a piano the white keys are C D E F G A B C D E... The black keys are **sharps** (#, a little higher) or **flats** (a little lower): C# is between C and D.
- **Octaves** are numbered. Octave 4 is the middle of a piano. Octave 5 sounds higher, octave 2 is deep bass.
- **Lengths**: a whole note lasts 4 beats, a half note 2, a quarter note 1, an eighth note half a beat, a sixteenth a quarter beat. A **dot** after a note makes it one and a half times as long.
- **Tempo** is the speed in beats per minute (BPM). 120 BPM = 2 beats per second.
- A **rest** is a silence with a length, just like a note.

MML: music as text

MegaBASIC uses **MML** (Music Macro Language), invented for computers in the 1980s. You write notes as letters inside quotes:

You write	Means
`c d e f g a b`	the notes C D E F G A B (upper or lower case)
`c+` or `c#`, `b-`	C sharp, B flat
`c4 c8 c16 c2 c1`	quarter, eighth, sixteenth, half, whole note
`c4.`	dotted quarter note (1.5 beats)
`r4`	rest (silence) of a quarter note
`l8`	default length: from now on, a note without a number is an eighth
`o4`	octave 4
`>` `<`	one octave up / down
`v12`	volume 0 (silent) to 15 (loudest)
`@piano`	instrument (FM channels) or volume envelope (PSG channels)
`[c e g]4`	repeat the part in brackets 4 times
`L`	loop point: when the channel reaches its end it continues from here, forever

So `"o4 l4 c d e f g2 g2"` plays C D E F as quarter notes and two G half notes. `"o4 c8 e8 g8 >c4"` plays a rising C major chord, one note after another.

Channels

Each channel plays one note at a time; together they make the music. You write one line of MML per channel:

Channel	Sound	Good for
`FM1` ... `FM6`	FM synthesis (YM2612)	melody, bass, chords, drums - each can be any instrument

`PSG1` … `PSG3`	square waves	chiptune leads, arpeggios, echoes
`NOISE`	noise	hi-hats, snares, explosions

Writing a song: Ode to Joy

Let's write a real piece of music - Beethoven's famous melody, which is free for everyone to use - with a bass line, an accompaniment and drums.

14b_melody.bas

```
' Writing music: "Ode to Joy" (Beethoven, 1824)
MUSIC joy
    TEMPO 120
    FM1   "@flute v13 o4 l4 L e e f g g f e d c c d e e. d8 d2"
    FM1   "e e f g g f e d c c d e d. c8 c2"
    FM2   "@bass v12 o2 l1 L c <g> c <g> c <g> c l2 <g> c"
    PSG1  "@decay v8 o5 l2 L [c e <b> d]2 c e <b> d c e <b> c"
    NOISE "@hat v7 l8 L [c c d c]8"
END MUSIC

PRINT AT 8, 10, "ODE TO JOY"
PRINT AT 8, 12, "L. VAN BEETHOVEN"
PLAYMUSIC joy

DO
    PRINT AT 8, 16, "TIME: "; TIMER / 60; " SECONDS "
    VSYNC
LOOP
```

`MUSIC name` starts a song definition. Everything until `END MUSIC` belongs to the song `joy`.

120 beats per minute: each quarter note lasts half a second.

The melody on FM channel 1, with the `flute` instrument, volume 13, octave 4, quarter notes (`l4`). `L` = loop from here. Then the notes: *E E F G | G F E D | C C D E | E. D D.* `e.` is a dotted quarter, `d8` an eighth, `d2` a half note.

The melody continues on a second line - lines for the same channel are joined together. The second half ends on C, the home note.

The bass on FM2: whole notes (`l1`), very low (octave 2). `<g>` means: one octave down, play G, back up. The bass alternates C and G - the two main chords of the tune. The last bar has two half notes (`l2`): G, then C.

A soft accompaniment on a square-wave channel. `@decay` makes each note fade out. `[c e <b> d]2` plays C E B D twice.

Hi-hat drums on the noise channel: `c` is a bright noise, `d` a slightly darker one, in eighth notes, repeated 8 times = 8 bars.

End of the song.

Some text on screen...

...naming the composer.

Start the song. It plays **by itself** in the background - your program continues immediately with the next line.

A small loop to prove the program keeps running while the music plays:

TIMER / 60 = seconds since power-on.

Next frame.

Forever - and the music loops forever too, thanks to `L`.

> **Tip** Write music one channel at a time. Start with the melody, build, listen. Then add the bass, build, listen. Then drums. Use spaces and `|` to separate bars - they are ignored, but help you count.

Instruments

FM channels can sound like many instruments. Choose one with `@name`:

Instruments	Names
Keys	`piano epiano organ bell`
Bass	`bass slapbass synthbass`
Lead	`lead brass flute guitar square sine strings`
Drums & effects	`kick snare tom laser`

PSG and NOISE channels use **envelopes** that shape the volume of each note: `flat pluck decay short swell hat drum soft echo`. Advanced users can design their own FM instruments - see the Reference Manual.

Sound effects

A sound effect is written like a song, in an `SFX` block, usually with just one channel and very short notes. While an effect plays it *borrows* its channel from the music; the music continues on that channel when the effect ends. You never have to manage this yourself.

14_music.bas

```
1  ' Music and sound effects
2  MUSIC song
3      TEMPO 128
4      FM1   "@piano v12 o4 l8 L c e g >c< g e c e d f a >d< a f d f"
5      FM2   "@bass v13 o2 l4 L c c d d"
6      PSG1  "@decay v9 o5 l2 L e g f a"
7      NOISE "@hat v8 l8 L [c c d c]2"
8  END MUSIC
9
10 SFX laser
11     FM6 "@laser v15 o6 s-40 l8 c"
12 END SFX
13
14 SFX coin
15     PSG3 "@pluck v15 o6 l32 b >e4"
16 END SFX
17
18 PRINT AT 2, 2, "A: LASER   B: COIN"
19 PRINT AT 2, 3, "C: PAUSE / CONTINUE MUSIC"
20 PLAYMUSIC song
21 DO
22     IF PRESSED(BTN_A) THEN PLAYSFX laser
23     IF PRESSED(BTN_B) THEN PLAYSFX coin
24     IF PRESSED(BTN_C) THEN
25         IF MUSICPLAYING() THEN PAUSEMUSIC ELSE RESUMEMUSIC
26     END IF
27     VSYNC
28 LOOP
```

- Line 1: ' Music and sound effects
- Line 2: A short song...
- Line 3: ...at 128 beats per minute.
- Line 4: Piano melody in eighth notes (`l8`), looping. `>c<` = the C one octave higher, then back down.
- Line 5: Bass in quarter notes: C C D D.
- Line 6: Long soft notes on a square-wave channel.
- Line 7: Hi-hats: `[c c d c]2` = the pattern twice.
- Line 8: End of the song.
- Line 9:
- Line 10: A sound effect called `laser` ...
- Line 11: ...on FM channel 6. `s-40` is a **sweep**: the pitch falls by 40 steps every frame - the classic 'pew!'.
- Line 12: End of the laser.
- Line 13:
- Line 14: The classic coin sound...
- Line 15: ...a very short B, then a longer E higher up (`>e4`).
- Line 16: End of the coin sound.
- Line 17:
- Line 18: Instructions.
- Line 19: More instructions.
- Line 20: Start the music.
- Line 21: Game loop.
- Line 22: `PLAYSFX name` plays an effect. FM6 is borrowed from the music for a moment (this song doesn't use FM6 anyway).
- Line 23: The coin sound borrows PSG3.
- Line 24: Button C pauses the music, or continues it:
- Line 25: `MUSICPLAYING()` is TRUE while music plays. If it plays, pause it; otherwise continue.
- Line 26: End of the C button block.
- Line 27: Next frame.
- Line 28: Forever.

Tip Sound effects mostly use PSG3, NOISE and FM6. If your game has many effects, keep those channels quiet in your music - then the music never has holes when an effect plays.

Exercise. Make the hero of chapter 15 say 'boing' when A is pressed.
Answer: Add `SFX boing` / `PSG3 "@short v14 o4 s20 l8 c"` / `END SFX` at the top, and `IF PRESSED(BTN_A) THEN PLAYSFX boing` in the loop. `s20` makes the pitch rise.

Part IV — Six complete games

Each chapter builds a different kind of game: a sports game, a grid game, a bat-and-ball game, a shoot-'em-up, a platform game and an arcade catching game. Each starts with a plan, then explains every line.

All games are in the `examples/games` folder (Star Catcher in `examples/starcatcher`) together with their pictures. Open the folder of a game in VS Code and press `F5` to play it before you read its chapter - it is much easier to understand a program when you know what it does.

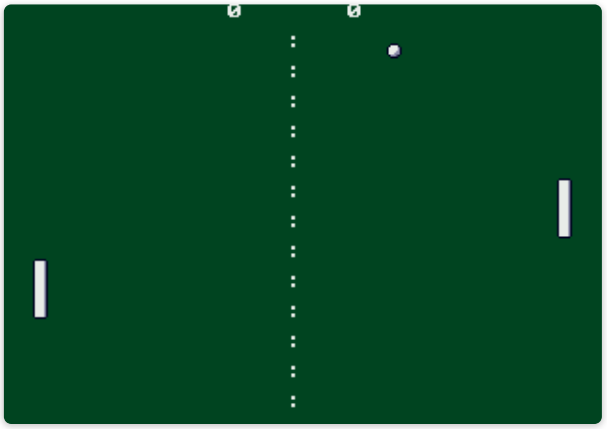
Chapter	Game	Type	New ideas
23	Pong	sports, 1 or 2 players	ball physics, a computer opponent, two joypads
24	Snake	grid / puzzle	drawing with tiles, 2-D arrays, a growing list
25	Breakout	bat and ball	destroying background tiles, bouncing off bricks, levels
26	Space Shooter	shoot-'em-up	bullets, enemy waves, aimed shots, explosions
27	Coin Quest	platform game	gravity, jumping, tile collisions, scrolling camera, DATA
28	Star Catcher	arcade	title screen, levels, lives, high score, full soundtrack

23 Game: Pong

Pong (1972) was the first video game to become a hit. Two bats, one ball, and a lot of fun - for one player against the computer or two players against each other.



Title screen



In the match

How to play

Each player moves a bat up and down. The ball bounces off the top and bottom of the court and off the bats. If the ball gets past your bat, the other player scores. The first to 7 points wins. Press START for a game against the computer, or A for two players (the second player uses joypad 2).

Planning the game

Thing	What we need to remember	Variables

left bat	its height on screen	`y1`
right bat	its height	`y2`
ball	position and speed in x and y	`bx, by, vx, vy`
score	points of both players	`score1, score2`
game mode	1 or 2 players	`twoPlayers`
serve pause	frames until the ball starts	`waitTimer`

The ball's position and speed are stored in **1/16 pixels** (chapter 19), so the ball can move at speeds like 2.5 pixels per frame and speed up a little after every hit.

Making the ball bounce

Bouncing is simpler than it sounds: the ball has a speed in x (`vx`) and in y (`vy`). To bounce off the top or bottom wall, make `vy` its opposite (`vy = -vy`). To bounce off a bat, flip `vx`. To make the game interesting, the new `vy` depends on *where* the ball hits the bat: near the middle it comes back flat, near the ends at a steep angle. That lets players aim.

Part 1: pictures, settings and sounds

pong.bas

```
5  TITLE "PONG"
6
7  SPRITES pad = "paddle.png", 1, 4      ' one frame, 1 tile wide, 4 tiles high (8x32)
8  SPRITES ball = "ball.png", 1, 1       ' one frame of 8x8 pixels
9
10 CONST TOP = 16, BOTTOM = 216          ' the ball bounces between these heights
11 CONST LEFT_X = 16, RIGHT_X = 296      ' x position of the two paddles
12 CONST PADDLE_H = 32                   ' paddle height in pixels
13 CONST WIN_SCORE = 7                   ' first to 7 points wins
14 CONST MAX_SPEED = 96                  ' fastest ball speed (in 1/16 pixels)
15
16 SFX hit
17     PSG3 "@short v14 o5 l32 g"
18 END SFX
19 SFX wall
20     PSG3 "@short v10 o4 l32 c"
21 END SFX
22 SFX point
23     PSG3 "@decay v14 o5 s-8 l4 c"
24 END SFX
25 MUSIC fanfare
26     TEMPO 140
```

	Name in the ROM header.
	The bat picture: one frame of 1 × 4 tiles = 8 pixels wide, 32 pixels high.
	The ball: one frame of 1 × 1 tile = 8 × 8 pixels.
	The court: the ball bounces between y = 16 (below the score line) and y = 216.
	Where the bats are on the x axis. The right bat is at 320 - 16 - 8 = 296.
	Bat height. Used in several calculations - one constant, so it can never get out of step.
	Points needed to win. Change it for longer matches.
	Speed limit for the ball: 96 / 16 = 6 pixels per frame.
	Sound for ball hitting a bat:
	one short high G.
	End of 'hit'.
	Sound for the walls:
	a quieter, lower C.
	End of 'wall'.
	Sound when a point is scored:
	a falling tone: `s-8` sweeps the pitch down every frame.
	End of 'point'.
	A short victory tune (no `L`, so it plays only once).
	Speed.

```
27      FM1 "@brass v13 o4 l8 c e g >c4 r8 <g >c2"
28      FM2 "@bass v12 o2 l4 c g c2"
29  END MUSIC
```

Brass melody climbing up a C major chord to a high C.

Bass under it.

End of the fanfare.

Part 2: variables and helper SUBs

pong.bas

```
31  DIM y1, y2                          ' paddle positions (top edge, in pixels)
32  DIM bx, by, vx, vy                  ' ball position and speed in 1/16 pixels
33  DIM score1, score2, twoPlayers, waitTimer
```

Bat positions: the y of each bat's top edge, in pixels.

Ball position and speed in 1/16 pixels.

Scores, the game mode and the serve pause.

pong.bas

```
36  SUB DrawCourt()
37      CLS
38      FOR row = 2 TO 27 STEP 2
39          PRINT AT 19, row, ":"
40      NEXT
41      DrawScore
42  END SUB
```

Draws the court at the start of a match.

Clear the screen.

Every second row from 2 to 27...

...a colon character in the middle column: a dashed net.

Next row.

SUBs may call other SUBs: show the score too.

End of DrawCourt.

pong.bas

```
44  SUB DrawScore()
45      PRINT AT 15, 0, score1
46      PRINT AT 23, 0, score2
47  END SUB
```

Prints both scores at the top, left and right of the net.

Player 1's score...

...and player 2's.

End of DrawScore.

pong.bas

```
50  SUB Serve(direction)
51      bx = 156 * 16
52      by = 112 * 16
53      vx = direction * 40
54      vy = RND(41) - 20
55      waitTimer = 45
56  END SUB
```

Puts the ball in the middle. `direction` is 1 (serve to the right) or -1 (to the left).

Centre of the court: x = 156 pixels (times 16 for 1/16 units)...

...y = 112.

Horizontal speed 40/16 = 2.5 pixels per frame, to the right or left.

A random vertical speed from -20 to +20, so every serve is different.

Wait 45 frames (3/4 second) before the ball moves - time to get ready.

End of Serve.

pong.bas

```
59  FUNCTION BounceAngle(paddleY)
60      LOCAL offset
```

Calculates the new vertical speed after the ball hits a bat whose top is at `paddleY`.

A local helper variable.

61	`    offset = (by >> 4) + 4 - (paddleY + PADDLE_H / 2)`	The distance between the ball's centre (`by >> 4` + 4 pixels) and the bat's centre (`paddleY` + 16). Negative = the ball hit the upper half, positive = the lower half. Range about -20 to +20.
62	`    RETURN offset * 3`	Turn the distance into a speed: hits near the edge send the ball off steeply.
63	`END FUNCTION`	End of BounceAngle.

pong.bas

66	`SUB ComputerMove()`	Moves the right bat when nobody plays player 2 - our first **artificial intelligence**!
67	`    LOCAL target`	Where the computer wants its bat to be.
68	`    target = (by >> 4) - PADDLE_H / 2 + 4`	It wants the bat's centre at the ball's height.
69	`    IF vx < 0 THEN target = 112 - PADDLE_H / 2  ' ball going away: return to centre`	But if the ball moves away (to the left), it goes back to the middle and waits - like a real player.
70	`    IF y2 < target - 3 THEN y2 += 3`	Move 3 pixels towards the target. The bat can only move 3 pixels per frame - that is why the computer can be beaten by fast, steep shots. The '- 3' / '+ 3' stops it from shaking up and down around the target.
71	`    IF y2 > target + 3 THEN y2 -= 3`	The same upwards.
72	`END SUB`	End of ComputerMove.

Try it Make the computer harder or easier by changing the 3s to 2 or 4. Game designers call this **tuning** - and it is half of what makes a game fun.

Part 3: setup and the title screen

pong.bas

75	`LOADTILES pad`	Bat graphics into video memory.
76	`LOADTILES ball`	Ball graphics.
77	`LOADPAL pad, 1`	Bat colours into palette 1...
78	`LOADPAL ball, 2`	...ball colours into palette 2 (each picture has its own colours).
79	`SPRITEPAL 0, 1`	Sprite 0 = left bat, palette 1.
80	`SPRITEPAL 1, 1`	Sprite 1 = right bat, palette 1.
81	`SPRITEPAL 2, 2`	Sprite 2 = ball, palette 2.
82	`SETCOLOR 0, 0, RGB(20, 70, 40)`	Dark green, like a table-tennis table...
83	`BACKDROP 0, 0`	...as background colour.

pong.bas

85	`DO`	The outer loop: title screen, match, result - forever.
86	`    ' ===== title screen =====`	
87	`    CLEARSPRITES`	Hide the bats and ball from the previous match.
88	`    CLS`	Clear the text.
89	`    PRINT AT 18, 6, "PONG"`	Title...
90	`    PRINT AT 10, 12, "START  = 1 PLAYER"`	...and the two choices.
91	`    PRINT AT 10, 14, "A      = 2 PLAYERS"`	A = two players.
92	`    PRINT AT 6, 20, "PAD 1 AND PAD 2: UP / DOWN"`	Controls.
93	`    DO`	Wait...
94	`        VSYNC`	...frame by frame...

95	`LOOP UNTIL PRESSED(BTN_START OR BTN_A)`	...until START or A is pressed.
96	`twoPlayers = PRESSED(BTN_A)`	Store the answer to the question "was it A?" - TRUE means two players, FALSE (it was START) means against the computer.
97	`RANDOMIZE TIMER`	Make the random serves different every game.

Part 4: the match - moving the bats

pong.bas

99	`' ===== a new match =====`	
100	`score1 = 0`	Reset the scores...
101	`score2 = 0`	...for both players.
102	`y1 = 96`	Bats in the middle.
103	`y2 = 96`	Both of them.
104	`DrawCourt`	Draw net and score.
105	`Serve 1`	First serve goes to the right.
106	`DO`	**The game loop of the match.**
107	`    ' --- move the paddles ---`	
108	`    IF BUTTON(BTN_UP) THEN y1 -= 4`	Player 1 (joypad 1): UP moves the bat 4 pixels up...
109	`    IF BUTTON(BTN_DOWN) THEN y1 += 4`	...DOWN moves it down.
110	`    IF twoPlayers THEN`	If there is a second human...
111	`        IF BUTTON(BTN_UP, 2) THEN y2 -= 4`	...read joypad 2 - note the `, 2` ...
112	`        IF BUTTON(BTN_DOWN, 2) THEN y2 += 4`	...for both directions.
113	`    ELSE`	Otherwise...
114	`        ComputerMove`	...the computer moves the right bat.
115	`    END IF`	End of the player-2 decision.
116	`    y1 = CLAMP(y1, TOP, BOTTOM - PADDLE_H)`	Keep both bats inside the court.
117	`    y2 = CLAMP(y2, TOP, BOTTOM - PADDLE_H)`	Same for the right bat.

Part 5: the match - moving and bouncing the ball

pong.bas

119	`    ' --- move the ball (after a short pause when serving) ---`	
120	`    IF waitTimer > 0 THEN`	During the serve pause...
121	`        waitTimer -= 1`	...just count down...
122	`    ELSE`	...otherwise...
123	`        bx += vx`	...move the ball by its speed...
124	`        by += vy`	...in both directions.
125	`    END IF`	End of the serve-pause check.
126		
127	`    ' --- bounce off the top and bottom ---`	
128	`    IF (by >> 4) < TOP THEN`	Ball above the top of the court?
129	`        by = TOP * 16`	Put it back exactly on the edge (so it cannot get stuck outside)...
130	`        vy = -vy`	...and reverse the vertical speed: bounce!
131	`        PLAYSFX wall`	Click.
132	`    ELSEIF (by >> 4) > BOTTOM - 8 THEN`	Ball below the bottom? (Its top is at by, so its bottom is 8 pixels lower.)

133	`            by = (BOTTOM - 8) * 16`	The same bounce at the bottom.
134	`            vy = -vy`	Reverse.
135	`            PLAYSFX wall`	Click.
136	`        END IF`	End of the wall bounces.
137		
138	`        ' --- bounce off the paddles ---`	
139	`        IF vx < 0 AND COLLIDE(bx >> 4, by >> 4, 8, 8, LEFT_X, y1, 8, PADDLE_H) THEN`	Moving left **and** touching the left bat? Checking the direction stops the ball from bouncing twice on the same bat.
140	`            vx = MIN(-vx + 4, MAX_SPEED)`	Reverse the horizontal speed and add 4 - the rally gets faster - but never faster than MAX_SPEED.
141	`            vy = BounceAngle(y1)`	New angle depending on where it hit.
142	`            PLAYSFX hit`	Pock!

pong.bas

143	`        END IF`	End of the left bat check.
144	`        IF vx > 0 AND COLLIDE(bx >> 4, by >> 4, 8, 8, RIGHT_X, y2, 8, PADDLE_H) THEN`	The same for the right bat, when the ball moves right.
145	`            vx = -MIN(vx + 4, MAX_SPEED)`	Faster, and now moving left (negative).
146	`            vy = BounceAngle(y2)`	Angle.
147	`            PLAYSFX hit`	Pock!
148	`        END IF`	End of the right bat check.
149		
150	`        ' --- a point is scored when the ball leaves the screen ---`	
151	`        IF (bx >> 4) < -8 THEN`	The ball left the screen on the left: player 1 missed it...
152	`            score2 += 1`	...player 2 scores...
153	`            PLAYSFX point`	...sound...
154	`            DrawScore`	...show the new score...
155	`            Serve 1`	...and serve towards player 2... er, to the right.
156	`        ELSEIF (bx >> 4) > 320 THEN`	Out on the right: player 1 scores.
157	`            score1 += 1`	Point for player 1.
158	`            PLAYSFX point`	Sound.
159	`            DrawScore`	Update the score.
160	`            Serve -1`	Serve to the left.
161	`        END IF`	End of the scoring check.
162		
163	`        ' --- draw everything ---`	
164	`        SPRITE 0, LEFT_X, y1, pad, 0`	Draw the left bat,
165	`        SPRITE 1, RIGHT_X, y2, pad, 0`	the right bat,
166	`        SPRITE 2, bx >> 4, by >> 4, ball, 0`	and the ball, converting 1/16 pixels to pixels.

Part 6: the end of the match

pong.bas

167	`        VSYNC`	One frame done.
168	`    LOOP UNTIL score1 = WIN_SCORE OR score2 = WIN_SCORE`	The match continues until someone has 7 points.
169		
170	`    ' ===== the match is over =====`	

```
    SPRITEOFF 2                          Hide the ball.
    PLAYMUSIC fanfare                    Victory music.
    IF score1 = WIN_SCORE THEN           Who won?
        PRINT AT 13, 12, "PLAYER 1 WINS!"  Player 1...
    ELSE                                 ...or...
        PRINT AT 13, 12, "PLAYER 2 WINS!"  ...player 2 (or the computer).
    END IF                               End of the winner decision.
    WAIT 240                             Show the result for 4 seconds.
LOOP                                     Back to the title screen.
```

Make it yours

- Add a "rally" counter that shows how many hits in a row, and a record.
- Make the bats smaller as the score goes up.
- Add a second ball when the score is 5:5.
- Let player 1 use the 6-button pad's X/Y/Z to make special shots.

24 Game: Snake

Snake is a classic of mobile phones and arcades: steer a growing snake to the apples - but never bite yourself or the wall.



The snake hunting an apple



Game over

How to play

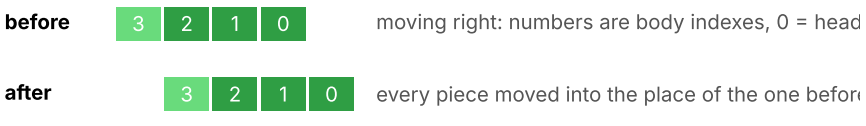
The snake keeps moving; you change its direction with the D-pad. Each apple makes it one piece longer and is worth 10 points. Every 50 points it gets faster. Hitting a wall or your own body ends the game.

Planning the game

Snake lives on a **grid** - the playfield is 40 × 26 cells of 8 × 8 pixels, exactly the size of one tile. This makes everything simple:

- The pictures are **tiles** on plane B, put in place with `SETTILE`. No sprites at all!
- A 2-D array `grid(x, y)` remembers what is in each cell: `EMPTY`, `SOLID` (wall or snake) or `FOOD`. To check if the snake may move into a cell, we just look at the array.
- The snake is a list of cells. `bodyX(0), bodyY(0)` is the head, `bodyX(1), bodyY(1)` the next piece, and so on up to `length - 1`, the tail.

Moving the snake: the head moves to a new cell, and every other piece moves into the place of the piece in front of it. The tail's old cell becomes empty - unless the snake just ate, then the tail stays and the snake grows.



How the snake moves

Part 1: pictures, constants, sounds and variables

snake.bas

```
5  TITLE "SNAKE"                                    ROM name.
6
```

7	`TILES pics = "snake.png"  ' tile 0 wall, 1 body, 2 head, 3 apple`	Four 8 × 8 tiles in one PNG: wall, body, head, apple. As TILES (not SPRITES), because we put them on a background plane.
8		
9	`CONST T_WALL = 0, T_BODY = 1, T_HEAD = 2, T_APPLE = 3`	Names for the four pictures: the first tile in the PNG is number 0.
10	`CONST W = 39, H = 25  ' grid size (columns 0..39, rows 0..25)`	The grid size: columns 0-39 and rows 0-25 (26 rows; the top 2 screen rows are for the score).
11	`CONST TOPROW = 2  ' the grid starts on screen row 2`	Grid row 0 is drawn on screen row 2.
12	`CONST MAXLEN = 500`	Longest possible snake - the size of the body arrays.
13	`CONST EMPTY = 0, SOLID = 1, FOOD = 2  ' what a grid cell contains`	What a grid cell can contain.
14		
15	`SFX eat`	Eating sound:
16	`    PSG3 "@short v15 o5 l32 c g >c"`	three rising notes.
17	`END SFX`	End of 'eat'.
18	`SFX crash`	Crash sound:
19	`    NOISE "@decay v15 w1 l2 e"`	white noise (`w1`) that fades out slowly.
20	`END SFX`	End of 'crash'.
21		
22	`DIM grid(W, H) AS BYTE  ' what is in each cell`	The playfield: 40 × 26 cells. `AS BYTE` because we only store 0, 1 or 2 - it saves memory.
23	`DIM bodyX(MAXLEN), bodyY(MAXLEN)  ' the snake: element 0 is the head`	The snake's cells. Element 0 is the head.
24	`DIM length, dir, newDir, score, best, delay, dead`	Snake length, current direction, the next direction, score, best score, speed (frames per step), and whether the game is over.

Part 2: drawing cells

snake.bas

27	`SUB DrawCell(x, y, picture)`	Puts one of our four pictures into grid cell (x, y).
28	`    SETTILE PLANE_B, x, y + TOPROW, pics.BASE + picture, 0`	`SETTILE plane, column, row, tile, palette`. `pics.BASE` is where LOADTILES put our tiles in video memory, so `pics.BASE + picture` is the tile we want. The grid starts 2 rows down.
29	`END SUB`	End of DrawCell.

snake.bas

31	`SUB EraseCell(x, y)`	Makes a cell empty again...
32	`    SETTILE PLANE_B, x, y + TOPROW, 0`	...by putting tile 0 there. Tile 0 is always empty (transparent).
33	`END SUB`	End of EraseCell.

Part 3: apples and a new game

snake.bas

36	`SUB PlaceApple()`	Puts an apple on a random **empty** cell.
37	`    LOCAL x, y`	Candidate position.

```
38      REPEAT                                  Try...
39          x = 1 + RND(W - 1)                  ...a random column inside the walls (1 to 38)...
40          y = 1 + RND(H - 1)                  ...and a random row (1 to 24)...
41      UNTIL grid(x, y) = EMPTY                ...until we find an empty cell (not on the snake!).
42      grid(x, y) = FOOD                       Remember the apple in the grid...
43      DrawCell x, y, T_APPLE                  ...and draw it.
44  END SUB                                     End of PlaceApple.
```

snake.bas

```
46  SUB NewGame()                               Sets up everything for a new game.
47      CLEARPLANE PLANE_B                      Erase the old playfield.
48      ' empty grid with a wall all around
49      FOR y = 0 TO H                          For every row...
50          FOR x = 0 TO W                      ...and every column:
51              IF x = 0 OR y = 0 OR x = W OR y = H THEN   On the border?
52                  grid(x, y) = SOLID          Then it is a wall: solid...
53                  DrawCell x, y, T_WALL       ...with a wall picture.
54              ELSE                            Inside the border...
55                  grid(x, y) = EMPTY          ...empty.
56              END IF                          End of border check.
57          NEXT                                Next column.
58      NEXT                                    Next row.
59      ' a snake of 4 pieces in the middle, moving right
60      length = 4                              The snake starts 4 pieces long...
61      FOR i = 0 TO length - 1                 ...for pieces 0 to 3:
62          bodyX(i) = 12 - i                   head at column 12, the others to its left (11, 10, 9)...
63          bodyY(i) = 12                       ...all on row 12.
64          grid(bodyX(i), bodyY(i)) = SOLID    Mark the cells as solid - the snake cannot move into itself.
65          DrawCell bodyX(i), bodyY(i), T_BODY Draw a body piece.
66      NEXT                                    Next piece.
67      DrawCell bodyX(0), bodyY(0), T_HEAD     Draw the head picture over piece 0.
68      dir = 1                                 Directions are numbered 0 = up, 1 = right, 2 = down, 3 = left. Start moving right.
69      newDir = 1                              The direction for the next step.
70      score = 0                               No points yet.
71      delay = 7                               The snake moves one cell every 7 frames (about 8.5 steps per second).
72      dead = FALSE                            Alive.
73      PlaceApple                              First apple.
74  END SUB                                     End of NewGame.
```

Part 4: steering and moving

snake.bas

```
77  SUB ReadPad()                               Reads the D-pad. We store the wish in newDir and apply it at the next step.
78      IF BUTTON(BTN_UP) AND dir <> 2 THEN newDir = 0   UP turns the snake up - but not if it is moving down (2): it would bite its own neck!
79      IF BUTTON(BTN_RIGHT) AND dir <> 3 THEN newDir = 1   RIGHT, unless moving left.
```

```
80      IF BUTTON(BTN_DOWN) AND dir <> 0 THEN newDir = 2     DOWN, unless moving up.
81      IF BUTTON(BTN_LEFT) AND dir <> 1 THEN newDir = 3     LEFT, unless moving right.
82  END SUB                                                  End of ReadPad.
```

snake.bas

```
85  SUB MoveSnake()                                          Moves the snake one cell. The heart of the game.
86      LOCAL nx, ny, ate                                    The new head position, and whether we ate an apple.
87      dir = newDir                                         Apply the steering wish.
88      nx = bodyX(0)                                        Start from the current head position...
89      ny = bodyY(0)                                        ...both x and y.
90      SELECT CASE dir                                      ...and move it one cell in the current direction:
91          CASE 0                                           up:
92              ny -= 1                                      one row up,
93          CASE 1                                           right:
94              nx += 1                                      one column right,
95          CASE 2                                           down:
96              ny += 1                                      one row down,
97          CASE 3                                           left:
98              nx -= 1                                      one column left.
99      END SELECT                                           End of the direction choice.
100     ate = (grid(nx, ny) = FOOD)                          Is there an apple in the new cell? The answer (TRUE/FALSE) goes into ate.
101     IF grid(nx, ny) = SOLID THEN                         Is the new cell a wall or the snake?
102         dead = TRUE                                      Then the game is over...
103         EXIT SUB                                         ...and we leave the SUB immediately.
104     END IF                                               End of the crash check.
105     IF ate THEN                                          Ate an apple:
106         IF length < MAXLEN THEN length += 1              grow by one (the tail stays where it is),
107         score += 10                                      10 points,
108         IF delay > 2 AND score MOD 50 = 0 THEN delay -= 1   every 50 points, speed up (fewer frames per step), but never below 2,
109         PLAYSFX eat                                      and crunch.
110     ELSE                                                 Did not eat:
111         ' the tail moves: free its cell
112         EraseCell bodyX(length - 1), bodyY(length - 1)   the tail leaves its cell - erase the picture...
113         grid(bodyX(length - 1), bodyY(length - 1)) = EMPTY   ...and mark the cell as empty.
114     END IF                                               End of the eating decision.
115     ' every piece takes the place of the piece in front of it
116     FOR i = length - 1 TO 1 STEP -1                      Now every piece moves into the place of the piece in front of it. We go backwards from the tail to the neck, so no position is overwritten before it was copied.
117         bodyX(i) = bodyX(i - 1)                          Piece i takes the x of piece i - 1...
118         bodyY(i) = bodyY(i - 1)                          ...and its y.
119     NEXT                                                 Next piece.
120     DrawCell bodyX(1), bodyY(1), T_BODY                  Where the head was, there is now a body piece.
121     bodyX(0) = nx                                        The head goes to the new cell...
122     bodyY(0) = ny                                        ...x and y.
```

```
    grid(nx, ny) = SOLID
    DrawCell nx, ny, T_HEAD
    IF ate THEN PlaceApple
END SUB
```

	...which is now solid...
	...and shows the head picture.
	The eaten apple needs a replacement.
	End of MoveSnake.

Part 5: the main program

snake.bas

```
LOADTILES pics
LOADPAL pics, 0
SETCOLOR 0, 0, RGB(30, 30, 50)
BACKDROP 0, 0
TEXTPAL 3
DO
    NewGame
    PRINT AT 1, 0, "SNAKE"
    DO
        FOR t = 1 TO delay
            ReadPad
            VSYNC
        NEXT
        MoveSnake
        PRINT AT 12, 0, "SCORE "; PAD$(score, 4); "
BEST "; PAD$(best, 4)
    LOOP UNTIL dead
    PLAYSFX crash
    IF score > best THEN best = score
    PRINT AT 15, 13, " GAME OVER "
    WAIT 150
    PRINT AT 15, 13, "           "
LOOP
```

	Tiles into video memory.
	Their colours into palette 0.
	Dark blue-grey background...
	...as backdrop.
	White text from palette 3.
	Outer loop: one game per round.
	Set up a new game.
	Title in the corner.
	The game loop.
	Wait `delay` frames before the next step...
	...but read the joypad in every one of them, so no button press is missed...
	...one frame at a time.
	End of the waiting loop.
	Now move one cell.
	Update the score line.
	Until the snake crashes.
	Crash!
	New best score?
	Game over message...
	...for 2.5 seconds...
	...then erase it.
	Next game.

Make it yours

- Add a golden apple (a new tile) worth 50 points that disappears after a few seconds.
- Add walls inside the playfield for level 2.
- Make the snake wrap around: leaving the right edge brings it back on the left (remove the side walls, use `MOD`).
- Two snakes for two players!

25 Game: Breakout

In Breakout (1976) you bounce a ball against a wall of bricks. Every brick you hit disappears. Clear the wall to reach the next level.



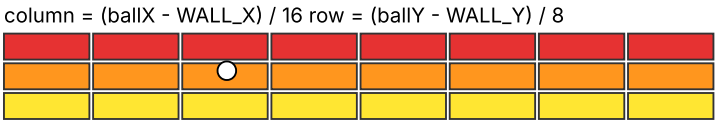
Breakout: some bricks are already gone

How to play

Move the bat with LEFT and RIGHT. The ball starts on the bat; press A to serve. Higher rows give more points. If the ball falls past the bat you lose one of 3 balls.

Planning the game

- The **bricks are background tiles** on plane B. Each brick is 16 × 8 pixels: a left-half tile and a right-half tile. The PNG has six colours of brick, each as two tiles: left half, right half.
- The array `wall(column, row)` remembers which bricks still exist (1) or are gone (0).
- The **bat and ball are sprites**.
- To find out whether the ball touches a brick, we turn the ball's pixel position into a brick column and row - like finding which square of a chess board a finger is on.



ball centre at column 2, row 1 → wall(2, 1)

From pixels to brick numbers

Bouncing off bricks uses a neat trick: move the ball in x first. If it is now inside a brick, undo the x move and flip `vx`. Then move in y. If it is now inside a brick, undo and flip `vy`. This way the ball bounces correctly whether it hits a brick from the side or from below.

Part 1: pictures, settings, sounds, variables

breakout.bas

```
5  TITLE "BREAKOUT"                                    ROM name.
```

```
6
7   TILES bricks = "bricks.png"          ' 6 colours x (left half, right half)
8   SPRITES bat = "bat.png", 4, 1        ' 32 x 8 pixels
9   SPRITES ball = "ball.png", 1, 1      ' 8 x 8 pixels
10
11  CONST COLS = 18, ROWS = 6            ' size of the brick wall
12  CONST WALL_X = 16, WALL_Y = 32       ' top-left corner of the wall in pixels
13  CONST BAT_Y = 200
14  CONST LEFT_EDGE = 16, RIGHT_EDGE = 304, TOP_EDGE = 16
15
16  SFX bonk
17      PSG3 "@short v13 o5 l32 e"
18  END SFX
19  SFX smash
20      PSG2 "@short v15 o6 l32 c g"
21  END SFX
22  SFX lost
23      PSG3 "@decay v15 o4 s-5 l2 c"
24  END SFX
25
26  DIM wall(COLS - 1, ROWS - 1) AS BYTE  ' 1 = brick still there
27  DIM batX, bx, by, vx, vy             ' ball position and speed in 1/16 pixels
28  DIM score, lives, level, left, onBat
```

The brick tiles: 6 colours x 2 halves = 12 tiles of 8 x 8.

The bat: 4 × 1 tiles = 32 × 8 pixels.

The ball: 8 × 8.

The wall has 18 columns and 6 rows of bricks.

The wall's top-left corner in pixels.

The bat's height on screen.

The playing area: the ball bounces at these pixel positions.

Sound for bat and walls.

A short E.

End of 'bonk'.

Brick smash sound, on PSG2 so it can play together with 'bonk' on PSG3.

Two high notes.

End of 'smash'.

Ball lost:

a slowly falling tone.

End of 'lost'.

One box per brick: `wall(c, r)` is 1 if the brick in column c, row r still exists. Columns 0-17, rows 0-5.

Bat x position (pixels); ball position and speed (1/16 pixels).

Score, balls left, level, bricks left, and whether the ball sits on the bat.

Part 2: drawing the wall

breakout.bas

```
31  SUB DrawBrick(c, r, colour)
32      LOCAL x, y
33      x = WALL_X / 8 + c * 2
34      y = WALL_Y / 8 + r
35      IF colour < 0 THEN
36          SETTILE PLANE_B, x, y, 0
37          SETTILE PLANE_B, x + 1, y, 0
38      ELSE
39          SETTILE PLANE_B, x, y, bricks.BASE + colour * 2, 0
40          SETTILE PLANE_B, x + 1, y, bricks.BASE + colour * 2 + 1, 0
41      END IF
```

Draws the brick at column c, row r in a colour 0-5 - or erases it if colour is -1.

Screen cell of the brick's left half.

Pixels to cells: the wall starts at cell 2 (16 / 8); each brick is 2 cells wide.

Each brick is 1 cell high; the wall starts at cell row 4.

Erase?

Put empty tiles in both cells...

...left and right.

Draw:

the left half: colour c's tiles are at positions c*2 (left) and c*2 + 1 (right)...

...and the right half.

End of draw/erase.

42	`END SUB`	End of DrawBrick.

breakout.bas

44	`SUB BuildWall()`	Builds a complete new wall (new game or new level).
45	`    CLEARPLANE PLANE_B`	Remove everything from plane B.
46	`    FOR r = 0 TO ROWS - 1`	For every row...
47	`        FOR c = 0 TO COLS - 1`	...and column:
48	`            wall(c, r) = 1`	the brick exists,
49	`            DrawBrick c, r, r`	and is drawn - in the colour of its row (row 0 red, row 1 orange...).
50	`        NEXT`	Next column.
51	`    NEXT`	Next row.
52	`    left = COLS * ROWS`	108 bricks to destroy.
53	`    ' grey side walls and roof made of font characters`	
54	`    FOR y = 1 TO 27`	Draw the side walls with text characters:
55	`        PRINT AT 1, y, "!"`	a column of ! on the left...
56	`        PRINT AT 38, y, "!"`	...and on the right.
57	`    NEXT`	Next row.
58	`    FOR x = 1 TO 38`	And a roof...
59	`        PRINT AT x, 1, "="`	...of = signs.
60	`    NEXT`	Next column.
61	`END SUB`	End of BuildWall.

Part 3: hitting bricks

breakout.bas

64	`FUNCTION HitBrick()`	Checks whether the ball's centre is inside a brick. If yes: removes it, scores, and returns TRUE.
65	`    LOCAL c, r`	Brick column and row.
66	`    c = ((bx >> 4) + 4 - WALL_X) >> 4     ' bricks are 16 pixels wide`	Ball x in pixels (`bx >> 4`), plus 4 for its centre, minus the wall's left edge; divided by 16 (`>> 4`), the brick width.
67	`    r = ((by >> 4) + 4 - WALL_Y) >> 3     ' and 8 pixels high`	The same for y; bricks are 8 high, so divide by 8 (`>> 3`).
68	`    IF c < 0 OR c >= COLS OR r < 0 OR r >= ROWS THEN RETURN FALSE`	Outside the wall area: certainly no brick. (This check also protects the array from wrong indexes!)
69	`    IF wall(c, r) = 0 THEN RETURN FALSE`	That brick is already gone.
70	`    wall(c, r) = 0`	Hit! The brick is gone...
71	`    DrawBrick c, r, -1`	...from the screen too...
72	`    score += (ROWS - r) * 10`	...points: top row 60, bottom row 10...
73	`    left -= 1`	...one brick fewer...
74	`    PLAYSFX smash`	...smash...
75	`    RETURN TRUE`	...and tell the caller that a brick was hit.
76	`END FUNCTION`	End of HitBrick.

breakout.bas

78	`SUB DrawStatus()`	The status line at the top.
79	`    PRINT AT 3, 0, "SCORE "; PAD$(score, 5)`	Score with 5 digits,
80	`    PRINT AT 18, 0, "LEVEL "; level`	level,

81	`    PRINT AT 29, 0, "BALLS "; lives`	balls left.
82	`END SUB`	End of DrawStatus.

Part 4: setup and a new game

breakout.bas		
85	`LOADTILES bricks`	Bricks, bat and ball into video memory...
86	`LOADTILES bat`	...the bat...
87	`LOADTILES ball`	...and the ball.
88	`LOADPAL bricks, 0`	Three palettes for three pictures:
89	`LOADPAL bat, 1`	the bat's colours in palette 1,
90	`LOADPAL ball, 2`	the ball's in palette 2.
91	`SPRITEPAL 0, 1`	Sprite 0 = bat...
92	`SPRITEPAL 1, 2`	...sprite 1 = ball.
93	`TEXTPAL 3`	White text.
94	`SETCOLOR 0, 0, RGB(10, 10, 40)`	Very dark blue...
95	`BACKDROP 0, 0`	...background.

breakout.bas		
97	`DO`	Outer loop: one game per round.
98	`    ' ===== start a game =====`	
99	`    score = 0`	New game: no points,
100	`    lives = 3`	three balls,
101	`    level = 1`	level 1,
102	`    BuildWall`	a full wall,
103	`    batX = 144`	bat in the middle,
104	`    onBat = TRUE`	ball waiting on the bat,
105	`    DrawStatus`	status line,
106	`    PRINT AT 13, 16, "PRESS A TO SERVE"`	and a hint.
107	`    DO`	**The game loop.**

Part 5: the game loop

breakout.bas		
108	`        ' --- the bat ---`	
109	`        IF BUTTON(BTN_LEFT) THEN batX -= 5`	The bat moves fast: 5 pixels per frame.
110	`        IF BUTTON(BTN_RIGHT) THEN batX += 5`	Right.
111	`        batX = CLAMP(batX, LEFT_EDGE, RIGHT_EDGE - 32)`	Between the side walls (the bat is 32 wide).
112		
113	`        IF onBat THEN`	While the ball waits on the bat:
114	`            ' the ball sits on the bat until A is pressed`	
115	`            bx = (batX + 12) * 16`	it rides along with the bat, in the middle...
116	`            by = (BAT_Y - 8) * 16`	...just above it.
117	`            IF PRESSED(BTN_A) THEN`	A serves:
118	`                onBat = FALSE`	the ball is free,
119	`                vx = 24`	flies a little to the right...
120	`                vy = -(36 + level * 6)`	...and upwards (negative y), faster in higher levels.

```
                PRINT AT 13, 16, "                "     Erase the hint.
            END IF                                      End of serve.
        ELSE                                            The ball is flying:
```

breakout.bas

```
            ' --- move across, bounce on side walls and bricks ---
            bx += vx                                    Step 1: move in x only.
            IF (bx >> 4) < LEFT_EDGE OR (bx >> 4) > RIGHT_EDGE - 8 THEN
                bx -= vx                                Undo the move...
                vx = -vx                                ...and bounce.
                PLAYSFX bonk                            Bonk.
            ELSEIF HitBrick() THEN                      Or moved into a brick? (HitBrick removes it.)
                bx -= vx                                Undo the move...
                vx = -vx                                ...and bounce sideways.
            END IF                                      End of x checks.
            ' --- move up/down, bounce on the roof and bricks ---
            by += vy                                    Step 2: move in y only.
            IF (by >> 4) < TOP_EDGE THEN                Hit the roof?
                by -= vy                                Undo...
                vy = -vy                                ...bounce down...
                PLAYSFX bonk                            ...bonk.
            ELSEIF HitBrick() THEN                      Moved into a brick from above or below?
                by -= vy                                Undo...
                vy = -vy                                ...and bounce vertically.
            END IF                                      End of y checks.
            ' --- the bat sends the ball back up ---
            IF vy > 0 AND COLLIDE(bx >> 4, by >> 4, 8, 8, batX, BAT_Y, 32, 8) THEN
                                                        Falling and touching the bat?
                vy = -ABS(vy)                           Send it up. -ABS(vy) is always negative = upwards.
                vx = ((bx >> 4) + 4 - (batX + 16)) * 4  Like in Pong: the further from the bat's centre, the steeper the angle. Hitting with the left end sends it left.
                PLAYSFX bonk                            Bonk.
            END IF                                      End of bat check.
            ' --- missed it ---
            IF (by >> 4) > 224 THEN                     Fell off the bottom:
                lives -= 1                              one ball lost,
                PLAYSFX lost                            sad sound,
                onBat = TRUE                            the next ball waits on the bat,
                IF lives > 0 THEN PRINT AT 13, 16, "PRESS A TO SERVE"
                                                        and if there are balls left, show the hint.
            END IF                                      End of ball-lost check.
            ' --- wall cleared: next level ---
            IF left = 0 THEN                            All bricks gone?
                level += 1                              Next level...
                BuildWall                               ...new wall...
                onBat = TRUE                            ...ball back on the bat...
```

```
162                 PRINT AT 13, 16, "PRESS A TO SERVE"     ...hint.
163             END IF                                      End of level check.
164             DrawStatus                                  Update the status line.
165         END IF                                          End of 'ball on bat / flying'.
166
167         SPRITE 0, batX, BAT_Y, bat, 0                   Draw the bat...
168         SPRITE 1, bx >> 4, by >> 4, ball, 0             ...and the ball.
169         VSYNC                                           Next frame.
170     LOOP UNTIL lives = 0                                Play until all balls are lost.
```

breakout.bas

```
171     PRINT AT 15, 16, "GAME OVER"                        Game over...
172     WAIT 180                                            ...for 3 seconds...
173     PRINT AT 15, 16, "          "                       ...then erase the message...
174 LOOP                                                    ...and start a new game.
```

Make it yours

- Bricks that need two hits (store 2 in `wall` and count down).
- A bonus that makes the bat wider (needs a second, wider bat picture).
- Different wall shapes per level, stored in DATA lines.
- A high-score line, like in Snake.

26 Game: Space Shooter

Shoot-'em-ups were the Mega Drive's speciality. Fly a ship, shoot waves of aliens, dodge their shots. This game juggles dozens of moving objects at once.



Title screen over the scrolling starfield



Shooting aliens

How to play

Fly in all directions with the D-pad and hold A, B or C to fire. Aliens weave down the screen in waves; each one is worth 50 points. Some aliens shoot at you. Three hits and the game is over.

Planning: object pools

At any moment there may be 5 bullets, 8 aliens and 6 alien shots on the screen. Each kind of object gets a set of arrays - a **pool**. For bullets:

Array	Meaning
`bOn(i)`	is bullet i flying? (TRUE/FALSE)
`bx(i), by(i)`	its position

To fire, we look for a bullet that is not flying, switch it on and place it at the ship. When it leaves the screen we switch it off again, so it can be reused. Aliens are similar, but they have three **states**: `FREE` (not in use), `FLYING` and `EXPLODING`. Each pool also owns a range of sprite numbers:

Sprites	Used for
0	player ship
1 - 5	bullets (sprite = SPR_B + i - 1)
6 - 13	aliens
14 - 19	alien shots

Two pieces of maths

- **Wavy flight.** `SIN(angle)` goes smoothly up and down between -256 and 256 as the angle grows (a full wave every 256 steps). Adding `SIN(...) * 40 >> 8` to an alien's x makes it swing 40 pixels left and right as it flies down.

- **Aiming.** `ATAN2(dy, dx)` gives the direction (0-255) from one point to another. `COS(direction)` and `SIN(direction)` then give the x and y parts of a speed in that direction - so the alien's shot flies straight at the player.

Part 1: pictures, constants, music, sounds and variables

shooter.bas	
`TITLE "SPACE SHOOTER"`	ROM name.
`SPRITES gfx = "shooter.png", 2, 2  ' 16x16 frames, see the constants below`	One sheet with all sprites, 16 × 16 frames.
`TILES space = "space.png"`	The star and planet tiles from Star Catcher...
`MAP sky = "space.csv"`	...and its starry sky map, which scrolls endlessly.
`CONST F_SHIP = 0, F_BULLET = 2, F_ALIEN = 3, F_BOOM = 5, F_SHOT = 7`	Frame numbers: ship (0 and 1, with flickering flame), bullet (2), alien (3 and 4, two animation frames), explosion (5 and 6), alien shot (7).
`CONST MAXB = 5  ' player bullets`	Pool sizes: up to 5 bullets...
`CONST MAXE = 8  ' aliens`	...8 aliens...
`CONST MAXS = 6  ' alien shots`	...and 6 alien shots.
`CONST SPR_B = 1, SPR_E = 6, SPR_S = 14  ' first sprite number of each group`	The first sprite number of each pool.
`CONST FREE = 0, FLYING = 1, EXPLODING = 2`	Names for the alien states.
`MUSIC battle`	Driving battle music:
`    TEMPO 160`	fast,
`    FM1  "@lead v11 o4 l8 L [e e >e< e d e g e]2 [c c >c< c d d >d< d]2"`	a lead melody,
`    FM2  "@synthbass v13 o2 l8 L [e e e e e e e e]2 [c c c c d d d d]2"`	pumping synth bass,
`    FM5  "@snare v10 o3 l4 L [r c r c]4"`	snare drum,
`    FM6  "@kick v13 o2 l4 L [c r c c]4"`	kick drum,
`    NOISE "@hat v8 l8 L [c c c c c c c c]4"`	and hi-hats.
`END MUSIC`	End of the music.
`SFX shoot`	Laser shot:
`    PSG3 "@short v11 o6 s-20 l16 c"`	a high note whose pitch falls quickly.
`END SFX`	End of 'shoot'.
`SFX explode`	Alien explosion:
`    NOISE "@drum v15 w1 l4 d"`	a burst of white noise.
`END SFX`	End of 'explode'.
`SFX ouch`	Player hit - on FM6, so it can sound together with the others...
`    FM6 "@laser v15 o3 s-10 l4 c"`	...a low, falling laser sound.
`END SFX`	End of 'ouch'.
`DIM px, py, score, lives, cooldown, hurtTimer, spawnTimer, t, scrollY`	The player's position, score, lives, the fire cooldown, the invincibility timer after a hit, the alien spawn timer, a frame counter t and the background scroll.
`DIM bx(MAXB), by(MAXB), bOn(MAXB) AS BYTE`	The bullet pool.

39	`DIM ex(MAXE), ey(MAXE), eBase(MAXE), ePhase(MAXE), eState(MAXE), eTimer(MAXE)`	The alien pool: position, the centre of its wave (`eBase`), the wave's starting point (`ePhase`), its state and an explosion timer.
40	`DIM sx(MAXS), sy(MAXS), svx(MAXS), svy(MAXS), sOn(MAXS) AS BYTE`	The alien-shot pool: position and speed in 1/16 pixels.

Part 2: the player

shooter.bas

43	`SUB MovePlayer()`	Steering, firing and drawing the ship.
44	`    IF BUTTON(BTN_LEFT) THEN px -= 3`	Fly 3 pixels per frame in all directions...
45	`    IF BUTTON(BTN_RIGHT) THEN px += 3`	right...
46	`    IF BUTTON(BTN_UP) THEN py -= 3`	up...
47	`    IF BUTTON(BTN_DOWN) THEN py += 3`	down.
48	`    px = CLAMP(px, 0, 304)`	Stay on screen...
49	`    py = CLAMP(py, 24, 208)`	...below the status line.
50	`    IF cooldown > 0 THEN cooldown -= 1`	The **cooldown** counts frames until the gun may fire again.
51	`    IF BUTTON(BTN_A OR BTN_B OR BTN_C) AND cooldown = 0 THEN Fire`	Holding a fire button fires automatically, but at most every 8 frames.
52	`    IF hurtTimer > 0 THEN hurtTimer -= 1`	After a hit the player is invincible for a while.
53	`    IF hurtTimer AND 4 THEN`	While invincible the ship blinks: `hurtTimer AND 4` switches between 0 and 4 every 4 frames.
54	`        SPRITEOFF 0`	Hidden...
55	`    ELSE`	...or...
56	`        SPRITE 0, px, py, gfx, F_SHIP + ((t >> 2) AND 1)`	...visible, with the flame flickering between frames 0 and 1.
57	`    END IF`	End of blink.
58	`END SUB`	End of MovePlayer.

shooter.bas

60	`SUB Fire()`	Fires one bullet, if a free one exists.
61	`    FOR i = 1 TO MAXB`	Look through the bullet pool:
62	`        IF bOn(i) = FALSE THEN`	a free bullet?
63	`            bOn(i) = TRUE`	Use it:
64	`            bx(i) = px`	start at the ship's x...
65	`            by(i) = py - 10`	...just above the ship.
66	`            cooldown = 8`	Next shot in 8 frames.
67	`            PLAYSFX shoot`	Pew.
68	`            EXIT SUB`	One bullet is enough - stop searching.
69	`        END IF`	End of 'free bullet'.
70	`    NEXT`	Next bullet. (If all 5 are flying, nothing happens.)
71	`END SUB`	End of Fire.

shooter.bas

73	`SUB MoveBullets()`	Moves all flying bullets.
74	`    FOR i = 1 TO MAXB`	For each bullet...
75	`        IF bOn(i) THEN`	...that is flying:
76	`            by(i) -= 7`	up 7 pixels per frame - fast!
77	`            IF by(i) < -16 THEN`	Left the top of the screen?

```
78                  bOn(i) = FALSE                  Free it...
79                  SPRITEOFF SPR_B + i - 1         ...and hide its sprite. Bullet 1 uses sprite 1, bullet 5 sprite 5.
80              ELSE                                Still on screen:
81                  SPRITE SPR_B + i - 1, bx(i), by(i), gfx, F_BULLET   draw it.
82              END IF                              End of 'left the screen'.
83          END IF                                  End of 'flying'.
84      NEXT                                        Next bullet.
85  END SUB                                         End of MoveBullets.
```

shooter.bas

```
87  SUB HurtPlayer()                                The player was hit.
88      IF hurtTimer > 0 THEN EXIT SUB              Still invincible from the last hit? Then nothing happens.
89      lives -= 1                                  One life less,
90      hurtTimer = 100                             100 frames of invincibility (and blinking),
91      PLAYSFX ouch                                ouch,
92      DrawStatus                                  update the status line.
93  END SUB                                         End of HurtPlayer.
```

Part 3: the aliens

shooter.bas

```
96  SUB SpawnAlien()                                From time to time a new alien appears at the top.
97      spawnTimer -= 1                             Count down...
98      IF spawnTimer > 0 THEN EXIT SUB             ...not yet.
99      spawnTimer = MAX(15, 45 - score / 200)      Time until the next alien: 45 frames at first, shorter as the score grows, but never less than 15. The game gets harder by itself.
100     FOR e = 1 TO MAXE                           Find a free alien slot:
101         IF eState(e) = FREE THEN                found one:
102             eState(e) = FLYING                  it starts flying,
103             eBase(e) = 40 + RND(224)            around a random x position (the centre of its wave),
104             ey(e) = -16                         just above the screen,
105             ePhase(e) = RND(256)                starting at a random point of the wave, so not all aliens swing together.
106             EXIT SUB                            One alien at a time.
107         END IF                                  End of 'free slot'.
108     NEXT                                        Next slot.
109 END SUB                                         End of SpawnAlien.
```

shooter.bas

```
111 SUB AlienShoots(e)                              Alien number e fires a shot aimed at the player.
112     LOCAL angle                                 The direction of the shot.
113     FOR s = 1 TO MAXS                           Find a free shot:
114         IF sOn(s) = FALSE THEN                  found one:
115             angle = ATAN2(py - ey(e), px - ex(e))   ATAN2(dy, dx): the direction from the alien to the player, as a number from 0 to 255 (0 = right, 64 = down, 128 = left, 192 = up).
    ' direction to the player
```

```
116             sx(s) = ex(e) * 16                          The shot starts at the alien (in 1/16 pixels)...
117             sy(s) = ey(e) * 16                          ...both coordinates.
118             svx(s) = COS(angle) * 40 >> 8               Speed in x: COS gives -256...256; times 40,
    ' 2.5 pixels per frame                                  divided by 256 ( >> 8 ) gives -40...40 sixteenths
                                                            = 2.5 pixels per frame in total.
119             svy(s) = SIN(angle) * 40 >> 8               Speed in y, the same way. Together they point
                                                            exactly at the player.
120             sOn(s) = TRUE                               The shot is flying.
121             EXIT SUB                                    Done.
122         END IF                                          End of 'free shot'.
123     NEXT                                                Next shot slot.
124 END SUB                                                 End of AlienShoots.
```

shooter.bas

```
126 SUB MoveAliens()                                        The biggest SUB: moves aliens, checks hits,
                                                            animates explosions.
127     FOR e = 1 TO MAXE                                   For each alien:
128         IF eState(e) = FLYING THEN                      if it is flying...
129             ey(e) += 1                                  ...it moves down 1 pixel per frame...
130             ex(e) = eBase(e) + (SIN(ePhase(e) + t * 3)  ...and swings around its base x: the angle grows
* 40 >> 8)   ' wavy flight                                 with time t, SIN goes up and down, times 40 / 256
                                                            = up to 40 pixels left or right.
131             IF ey(e) > 224 THEN                         Left the bottom of the screen?
132                 eState(e) = FREE                        Free the slot...
133                 SPRITEOFF SPR_E + e - 1                 ...hide the sprite...
134                 CONTINUE FOR                            ...and go on with the next alien (skip the rest of
                                                            the loop body).
135             END IF                                      End of 'left the screen'.
136             ' hit by one of our bullets?
137             FOR i = 1 TO MAXB                           Hit by a bullet? Check every bullet against this
                                                            alien:
138                 IF bOn(i) AND COLLIDE(bx(i) + 6,        A flying bullet whose rectangle (the thin middle
by(i), 4, 12, ex(e) + 1, ey(e) + 3, 14, 10) THEN            part of its frame) overlaps the alien's body?
139                     bOn(i) = FALSE                      The bullet is used up...
140                     SPRITEOFF SPR_B + i - 1             ...and hidden.
141                     eState(e) = EXPLODING               The alien explodes...
142                     eTimer(e) = 16                      ...for 16 frames.
143                     score += 50                         50 points!
144                     PLAYSFX explode                     Boom.
145                     DrawStatus                          Show the new score.
146                     EXIT FOR                            No need to check the other bullets.
147                 END IF                                  End of 'bullet hit'.
148             NEXT                                        Next bullet.
149             ' crashed into the player?
150             IF eState(e) = FLYING AND COLLIDE(px + 3,    Still flying and crashing into the player?
py + 3, 10, 10, ex(e) + 2, ey(e) + 3, 12, 10) THEN
151                 HurtPlayer                              Ouch.
152                 eState(e) = EXPLODING                   The alien explodes too...
153                 eTimer(e) = 16                          ...for 16 frames.
154             END IF                                      End of crash check.
155             ' sometimes shoot at the player
```

```
156             IF eState(e) = FLYING AND ey(e) > 0 AND ey(e) < 150 AND RND(120) = 0 THEN AlienShoots e   Sometimes it shoots: only in the upper part of the screen, and on average once every 120 frames (RND(120) is 0 one time in 120).
157             IF eState(e) = FLYING THEN SPRITE SPR_E + e - 1, ex(e), ey(e), gfx, F_ALIEN + ((t >> 3) AND 1)   Draw it, alternating frames 3 and 4 every 8 frames.
158         ELSEIF eState(e) = EXPLODING THEN   An exploding alien:
159             eTimer(e) -= 1   count down,
160             IF eTimer(e) >= 8 THEN   First half of the explosion (timer 15 down to 8)...
161                 SPRITE SPR_E + e - 1, ex(e), ey(e), gfx, F_BOOM   ...the small fireball (frame 5)...
162             ELSE   ...second half...
163                 SPRITE SPR_E + e - 1, ex(e), ey(e), gfx, F_BOOM + 1   ...the big fireball (frame 6).
164             END IF   End of the fireball choice.
165             IF eTimer(e) = 0 THEN   Explosion over?
166                 eState(e) = FREE   Free the slot...
167                 SPRITEOFF SPR_E + e - 1   ...hide the sprite.
168             END IF   End of 'explosion over'.
169         END IF   End of the state decision.
170     NEXT   Next alien.
171 END SUB   End of MoveAliens.
```

Part 4: alien shots, status, cleaning up

shooter.bas

```
173 SUB MoveShots()   Moves the alien shots.
174     LOCAL x, y   Shot position in pixels.
175     FOR s = 1 TO MAXS   For each shot...
176         IF sOn(s) THEN   ...that is flying:
177             sx(s) += svx(s)   move it in x...
178             sy(s) += svy(s)   ...and y,
179             x = sx(s) >> 4   convert to pixels...
180             y = sy(s) >> 4   ...for both coordinates.
181             IF x < -16 OR x > 320 OR y < -16 OR y > 224 THEN   Left the screen in any direction?
182                 sOn(s) = FALSE   Free it,
183                 SPRITEOFF SPR_S + s - 1   hide it.
184             ELSEIF COLLIDE(x + 5, y + 5, 6, 6, px + 4, py + 4, 8, 8) THEN   Hit the player? (Small rectangles: shots are tiny.)
185                 sOn(s) = FALSE   The shot disappears...
186                 SPRITEOFF SPR_S + s - 1   ...from the screen...
187                 HurtPlayer   ...and hurts the player.
188             ELSE   Otherwise...
189                 SPRITE SPR_S + s - 1, x, y, gfx, F_SHOT   ...draw it.
190             END IF   End of checks.
191         END IF   End of 'flying'.
192     NEXT   Next shot.
193 END SUB   End of MoveShots.
```

shooter.bas

```
195 SUB DrawStatus()                                  The status line.
196     PRINT AT 1, 1, "SCORE "; PAD$(score, 6)       Score with 6 digits...
197     PRINT AT 30, 1, "LIVES "; lives               ...and lives.
198 END SUB                                           End of DrawStatus.
```

shooter.bas

```
200 SUB ClearAll()                                    Resets all pools before a new game.
201     CLEARSPRITES                                  Hide all sprites.
202     FOR i = 1 TO MAXB                             All bullets...
203         bOn(i) = FALSE                            ...off.
204     NEXT                                          Next.
205     FOR e = 1 TO MAXE                             All aliens...
206         eState(e) = FREE                          ...free.
207     NEXT                                          Next.
208     FOR s = 1 TO MAXS                             All alien shots...
209         sOn(s) = FALSE                            ...off.
210     NEXT                                          Next.
211 END SUB                                           End of ClearAll.
```

Part 5: the main program

shooter.bas

```
214 LOADTILES gfx                                     Sprites...
215 LOADTILES space                                   ...and background tiles into video memory.
216 LOADPAL space, 0                                  Background colours in palette 0...
217 LOADPAL gfx, 1                                    ...sprite colours in palette 1.
218 FOR n = 0 TO 19                                   All 20 sprites we use...
219     SPRITEPAL n, 1                                ...get palette 1.
220 NEXT                                              Next sprite.
221 SETCOLOR 0, 0, RGB(0, 0, 40)                      Deep space blue...
222 BACKDROP 0, 0                                     ...as backdrop.
223 TEXTPAL 3                                         White text.
224 DRAWMAP sky, PLANE_B, space, 0                    Draw the starfield on plane B.
225
226 DO                                                Outer loop: title, game, game over.
227     ' ----- title -----
228     ClearAll                                      Clean up from the last game.
229     CLS                                           Clear text.
230     PRINT AT 13, 9, "SPACE  SHOOTER"              Title...
231     PRINT AT 9, 13, "D-PAD: FLY   A,B,C: FIRE"    ...controls...
232     PRINT AT 14, 18, "PRESS START"                ...and how to start.
233     DO                                            Title loop:
234         t += 1                                    count frames,
235         scrollY -= 2                              scroll the stars down 2 pixels per frame...
236         SCROLL PLANE_B, 0, scrollY                ...(moving the plane's view up makes the stars
                                                      move down: we fly forwards),
237         VSYNC                                     next frame,
238     LOOP UNTIL PRESSED(BTN_START)                 until START.
239     RANDOMIZE TIMER                               Different game every time.
```

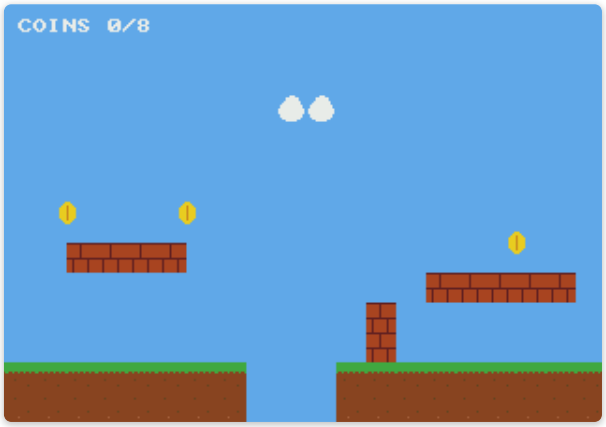
```
240
241     ' ----- play -----                    
242     CLS                                  Clear the title text.
243     score = 0                            New game: no points...
244     lives = 3                            ...three lives...
245     px = 152                             ...ship at the bottom centre...
246     py = 190                             ...near the bottom...
247     hurtTimer = 0                        ...not invincible...
248     spawnTimer = 60                      ...first alien after one second.
249     DrawStatus                           Status line.
250     PLAYMUSIC battle                     Battle music!
251     DO                                   The game loop - a list of SUB calls:
252         t += 1                           count frames (the aliens' waves use t),
253         scrollY -= 2                     scroll...
254         SCROLL PLANE_B, 0, scrollY       ...the stars,
255         MovePlayer                       player,
256         MoveBullets                      bullets,
257         SpawnAlien                       new aliens,
258         MoveAliens                       aliens and hits,
259         MoveShots                        alien shots,
260         VSYNC                            show the frame.
261     LOOP UNTIL lives = 0                 Until all lives are lost.
262
263     ' ----- game over -----              
264     STOPMUSIC                            Silence...
265     PRINT AT 15, 13, "GAME OVER"         ...game over...
266     WAIT 180                             ...3 seconds...
267 LOOP                                     ...back to the title.
```

Make it yours

- Power-ups: an alien sometimes drops an item that gives a triple shot.
- A boss alien made of four 32 × 32 sprites, with an energy bar.
- Different alien movements: straight dives, circles (SIN for x and COS for y), formations.
- A high score, and lives shown as small ship icons.

27 Game: Coin Quest

Platform games - jumping over pits, running along a level - defined the 16-bit era. Coin Quest has a scrolling level made in Tiled, gravity, jumping, solid ground and walls, and coins to collect.



Running and jumping through the level



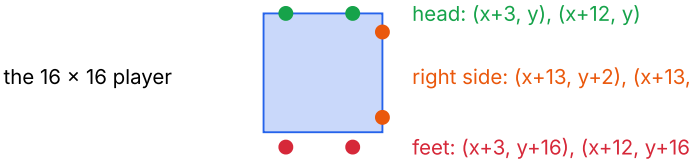
Reaching the end

How to play

Run with LEFT and RIGHT, jump with A, B or C. Collect the 8 coins and reach the right end of the level. Falling into a pit sends you back to the start.

Planning: how a platform game works

- The level is a **Tiled map** (64 × 14 tiles of 16 × 16 pixels) on plane B. `MAPSCROLL` scrolls it so that the player stays in the middle of the screen.
- The player's position is a position **in the level** (not on the screen), in 1/16 pixels. On screen the player appears at `levelX - camx`.
- To know if the player may move somewhere, we ask the map: `MAPTILE` tells us which tile is at a pixel position. Grass, earth and bricks are **solid**.
- We move **first sideways, then up/down**, and check the map after each, exactly like in Breakout. Sideways into a wall: don't move. Down onto ground: stand on it. Up into a ceiling: stop rising.
- Coins are sprites. Their level positions come from `DATA` lines at the end of the program.



We ask "solid?" at these test points around the player.

Collision test points (enlarged 4 times)

Part 1: assets, constants, sounds, variables

platformer.bas

```
6  TITLE "COIN QUEST"                                    ROM name.
```

7		
8	`TILES ground = "tiles.png", 16 ' 16x16 metatiles`	The level tiles, 16 × 16 like in Tiled.
9	`SPRITES hero = "hero.png", 2, 2 ' 4 walking frames`	The walking boy, 4 frames.
10	`SPRITES items = "tiles.png", 2, 2 ' same picture as sprites: frame 4 = coin`	The same tileset picture loaded a second time, now as a sprite sheet. Frame 4 is the coin. (Sprites need their tiles in a different order than backgrounds, so the picture is converted twice.)
11	`MAP level = "level.csv", 16 ' 64 x 14 tiles made in Tiled`	The level from Tiled.
12		
13	`' tile numbers in the Tiled tileset (0 = empty)`	
14	`CONST T_GRASS = 1, T_DIRT = 2, T_BRICK = 3`	The Tiled numbers of the solid tiles.
15		
16	`' physics in 1/16 pixel units`	
17	`CONST GRAVITY = 5, JUMP_SPEED = -92, WALK = 22, MAX_FALL = 100`	Physics, all in 1/16 pixels: gravity adds 5 per frame to the falling speed, a jump starts with speed -92 (upwards), walking speed is 22 (1.4 pixels), and falling is limited to 100 (6 pixels per frame).
18	`CONST NCOINS = 8`	Number of coins.
19		
20	`SFX jump`	Jump sound:
21	`    PSG3 "@short v13 o4 s12 l16 c"`	a short note whose pitch rises (`s12`) - boing!
22	`END SFX`	End of 'jump'.
23	`SFX coin`	Coin sound:
24	`    PSG3 "@pluck v14 o6 l32 b >e4"`	B, then a high E.
25	`END SFX`	End of 'coin'.
26	`SFX fall`	Falling into a pit:
27	`    PSG3 "@decay v14 o5 s-6 l2 c"`	a long falling tone.
28	`END SFX`	End of 'fall'.
29		
30	`MUSIC tune`	A cheerful tune:
31	`    TEMPO 132`	speed,
32	`    FM1  "@square v10 o5 l8 L e g >c< g a f d f g e c e d4 g4"`	a square-wave melody,
33	`    FM2  "@bass v13 o2 l8 L [c >c<]4 [f >f<]2 [g >g<]2"`	bouncing octave bass,
34	`    NOISE "@hat v8 l8 L [c d]8"`	hi-hats.
35	`END MUSIC`	End of the tune.
36		
37	`DIM px, py, vx, vy, onGround, facing, camx, frame, coins`	Player position and speed in 1/16 pixels (level coordinates!), standing on ground?, facing left?, camera x, a frame counter and the coin count.
38	`DIM coinX(NCOINS), coinY(NCOINS), coinGot(NCOINS) AS BYTE`	Coin positions in level pixels, and which coins are collected.

Part 2: asking the map

platformer.bas

41	`FUNCTION Solid(x, y)`	Answers TRUE if the level pixel (x, y) is inside a solid tile.
42	`    LOCAL t`	The tile number.

43	`    t = MAPTILE(level, x >> 4, y >> 4)`	Pixel to tile: tiles are 16 pixels, so divide by 16 (`>> 4`). MAPTILE returns the Tiled number there (0 = empty, also outside the map).
44	`    RETURN t = T_GRASS OR t = T_DIRT OR t = T_BRICK`	Solid if grass, earth or bricks. Clouds and empty sky are not solid - you can jump through clouds.
45	`END FUNCTION`	End of Solid.

platformer.bas

47	`SUB StartLevel()`	Puts the player at the start.
48	`    px = 32 * 16`	x = 32 pixels...
49	`    py = 160 * 16`	...y = 160 pixels, standing on the ground.
50	`    vx = 0`	Not moving...
51	`    vy = 0`	...in any direction.
52	`    MAPRESET PLANE_B ' force a full redraw of the map`	Tell MAPSCROLL to redraw the whole visible map next time (the camera jumps back to the start).
53	`END SUB`	End of StartLevel.

platformer.bas

55	`SUB ReadCoins()`	Reads the coin positions from the DATA lines.
56	`    RESTORE coindata`	`RESTORE label` sets the reading position to the DATA after the label `coindata:` (at the end of the program).
57	`    FOR i = 1 TO NCOINS`	For each coin...
58	`        READ coinX(i), coinY(i)`	... `READ` takes the next two numbers from DATA...
59	`        coinGot(i) = FALSE`	...and the coin is not collected yet.
60	`    NEXT`	Next coin.
61	`END SUB`	End of ReadCoins.

Part 3: moving the player

platformer.bas

63	`SUB MovePlayer()`	Walking, falling, jumping and collisions.
64	`    LOCAL nx, ny, x, y`	nx, ny: the *new* position we would like to move to (1/16 pixels); x, y: pixel versions for the map tests.
65	`    ' ---- walking ----`	
66	`    vx = 0`	No button, no walking.
67	`    IF BUTTON(BTN_LEFT) THEN vx = -WALK : facing = TRUE`	LEFT: walk left and face left...
68	`    IF BUTTON(BTN_RIGHT) THEN vx = WALK : facing = FALSE`	...RIGHT: walk right, face right.
69	`    nx = px + vx`	Where we want to go in x.
70	`    x = nx >> 4`	In pixels.
71	`    y = py >> 4`	The current height in pixels.
72	`    IF vx > 0 THEN`	Walking right:
73	`        IF Solid(x + 13, y + 2) OR Solid(x + 13, y + 14) THEN nx = px`	test two points on the right side of the body (near the head and near the feet). Wall? Then stay where we are.
74	`    ELSEIF vx < 0 THEN`	Walking left:
75	`        IF Solid(x + 2, y + 2) OR Solid(x + 2, y + 14) THEN nx = px`	test the left side.
76	`    END IF`	End of wall checks.
77	`    px = MAX(0, nx)`	Accept the new x, but never left of the level start.

platformer.bas

```
    ' ---- gravity and jumping ----
    vy = MIN(vy + GRAVITY, MAX_FALL)
    ny = py + vy
    x = px >> 4
    y = ny >> 4
    onGround = FALSE
    IF vy > 0 THEN
        IF Solid(x + 3, y + 16) OR Solid(x + 12, y + 16) THEN
            ny = (((y + 16) >> 4) * 16 - 16) << 4
' stand exactly on the tile
            vy = 0
            onGround = TRUE
        END IF
    ELSEIF vy < 0 THEN
        IF Solid(x + 3, y) OR Solid(x + 12, y) THEN
            ny = py
            vy = 0
        END IF
    END IF
    py = ny
    IF onGround AND PRESSED(BTN_A OR BTN_B OR BTN_C) THEN
        vy = JUMP_SPEED
        PLAYSFX jump
```

Gravity makes us fall faster every frame - up to the maximum falling speed.

Where we want to go in y.

Pixel x...

...and the new pixel y.

Assume we are in the air until we find ground.

Moving down:

is there something solid just below either foot?

Yes: put the player exactly on top of that tile. (y + 16) >> 4 is the tile row under the feet; * 16 is its top in pixels; - 16 the player's height; << 4 back to 1/16 pixels.

Stop falling.

We are standing.

End of ground check.

Moving up (jumping):

bumped the head on something?

Then don't move up...

...and start falling.

End of ceiling check.

End of up/down.

Accept the new y.

Jump - but only when standing on something (no jumping in mid-air!), and only when the button was just pressed.

Shoot upwards. Gravity will slow us down and pull us back: a perfect arc.

Boing.

platformer.bas

```
    END IF
    ' fell into a pit?
    IF (py >> 4) > 232 THEN
        PLAYSFX fall
        WAIT 30
        StartLevel
    END IF
    ' reached the end of the level?
    IF (px >> 4) > level.WIDTH * 16 - 40 THEN
        PRINT AT 12, 12, "LEVEL COMPLETE!"
        PRINT AT 12, 14, "COINS: "; coins; " OF "; NCOINS
        WAIT 240
        PRINT AT 12, 12, "               "
        PRINT AT 12, 14, "               "
        coins = 0
        ReadCoins
        StartLevel
```

End of jump.

Below the bottom of the level - fell into a pit:

sad sound,

half a second pause,

back to the start (collected coins stay collected).

End of pit check.

Near the right end of the level (level width in pixels minus 40):

congratulations,

how many coins,

4 seconds to enjoy it,

erase the messages...

...both lines,

and restart: no coins,

all coins back,

player at the start.

```
117     END IF                                                  End of level-complete check.
118 END SUB                                                     End of MovePlayer.
```

Part 4: drawing the world

platformer.bas

```
120 SUB DrawWorld()                                             Camera, player sprite and coins.
121     LOCAL sx, sy, x, anim                                   Helper variables.
122     x = px >> 4                                             Player x in level pixels.
123     camx = CLAMP(x - 152, 0, level.WIDTH * 16 - 320)        The camera keeps the player in the middle of the screen (152 = 160 - half the player), but never shows anything before the start or after the end of the level.
124     MAPSCROLL level, PLANE_B, ground, 0, camx, 0            Scroll plane B to the camera and draw new map columns.
125     ' player
126     frame += 1                                              Animation counter.
127     IF vx <> 0 AND onGround THEN anim = (frame >> 2) AND 3 ELSE anim = 0   Walking on the ground: cycle frames 0-3 every 4 frames. Standing or jumping: frame 0.
128     SPRITEFLIP 0, facing, FALSE                             Mirror when facing left.
129     SPRITE 0, x - camx, py >> 4, hero, anim                 Draw the player at **level x minus camera** = screen x.
130     ' coins are sprites (frame 4 of the items sheet)
131     FOR i = 1 TO NCOINS                                     For each coin:
132         IF coinGot(i) THEN                                  collected?
133             SPRITEOFF i                                     Then hide it.
134         ELSE                                                Otherwise:
135             sx = coinX(i) - camx                            screen position = level position minus camera...
136             sy = coinY(i)                                   ...(y does not scroll in this game),
137             SPRITE i, sx, sy, items, 4                      draw it (sprites outside the screen are hidden automatically). The coin uses palette 0, the same colours as the tiles, because it comes from the same picture.
138             IF COLLIDE(x + 2, py >> 4, 12, 16, coinX(i), coinY(i), 16, 16) THEN   Touching the player? (Both rectangles in level coordinates.)
139                 coinGot(i) = TRUE                           Collected!
140                 coins += 1                                  Count it.
141                 PLAYSFX coin                                Ding.
142             END IF                                          End of touch check.
143         END IF                                              End of 'collected?'.
144     NEXT                                                    Next coin.
145     PRINT AT 1, 1, "COINS "; coins; "/"; NCOINS             Coin counter in the corner.
146 END SUB                                                     End of DrawWorld.
```

Part 5: the main program and the coin DATA

platformer.bas

```
149 LOADTILES ground                                            Level tiles...
150 LOADTILES hero                                              ...player...
151 LOADTILES items                                             ...and coin sprites into video memory.
152 LOADPAL ground, 0                                           Tile colours (also used by the coins) in palette 0...
```

153	`LOADPAL hero, 1`	...player colours in palette 1.
154	`SETCOLOR 0, 0, RGB(100, 160, 255)`	Sky blue...
155	`BACKDROP 0, 0`	...as backdrop.
156	`TEXTPAL 3`	White text.
157	`SPRITEPAL 0, 1`	The player uses palette 1.
158	`ReadCoins`	Coins from DATA.
159	`StartLevel`	Player to the start.
160	`PLAYMUSIC tune`	Music.
161	`DO`	The game loop - short, because the SUBs do the work:
162	`    MovePlayer`	move,
163	`    DrawWorld`	draw,
164	`    VSYNC`	show.
165	`LOOP`	Forever.
166		
167	`coindata:`	A **label**: a name followed by a colon. RESTORE uses it to find the DATA below.
168	`DATA 120, 104,  184, 104,  360, 120,  432, 72`	Coin positions as pairs of numbers: x, y, x, y... The first coin is at (120, 104).
169	`DATA 600, 104,  664, 56,  872, 120,  960, 104`	The other four coins. DATA lines are never run - they are a table that READ takes numbers from.

Make it yours

- Draw a second level in Tiled and switch to it at the end of the first.
- Add enemies that walk left and right on platforms (use `Solid` to turn them around at walls).
- Make the jump higher when the button is held longer (reduce gravity while held and moving up).
- Remove a coin from the map itself with `SETTILE` instead of using sprites.

28 Game: Star Catcher

The last game brings everything together into a polished arcade game: title screen with music, increasing difficulty, lives, explosions, a high score and a full 10-channel soundtrack.



Title screen



Catching stars, dodging rocks

How to play

Steer the ship left and right at the bottom of the screen. Catch the falling stars, avoid the rocks. Every 10 stars the level goes up: more rocks, faster falling, more points per star. You have 3 ships; after a crash you blink and cannot be hit for a moment.

Planning

- Falling objects (stars and rocks) live in a pool of 8 slots, like the aliens in the Space Shooter. `objKind(i)` says what is in slot i: nothing, a star or a rock.
- Object y positions are in 1/16 pixels, so each object can fall at its own smooth speed.
- The main program is only four lines: load graphics, then forever title screen, play, game over.

> **Try it** This chapter shows the complete program. You already know every command in it - try to predict what each part does before you read the explanation.

Part 1: assets and settings

starcatcher.bas

```
5  TITLE "STAR CATCHER"                                  ROM name.
6
7  ' ---------- graphics ----------
8  SPRITES gfx = "sprites.png", 2, 2      ' sprite sheet,   8 frames of 16 × 16: ship x 2, star x 2, rock x 2,
   frames are 2x2 tiles (16x16)                             explosion x 2.
9  TILES space = "space.png"              ' 8x8             Star and planet tiles (8 × 8).
   background tiles
10 MAP sky = "space.csv"                  ' background      The night sky, drawn in Tiled: 40 × 32 tiles -
   drawn in Tiled, saved as CSV                             exactly as high as plane B, so it can scroll
                                                            vertically forever without a seam.
11
```

```
12  ' frame numbers inside sprites.png (left to right)
13  CONST F_SHIP = 0, F_STAR = 2, F_ROCK = 4, F_BOOM = 6
14
15  ' ---------- game settings ----------
16  CONST MAXOBJ = 8                    ' falling objects on screen at once
17  CONST SHIP_Y = 196                  ' the ship's fixed height on screen
18  CONST KIND_NONE = 0, KIND_STAR = 1, KIND_ROCK = 2
19  CONST BOOM_SPR = 9                  ' sprite number used for the explosion
```

Frame numbers of the first picture of each pair.	
Up to 8 falling objects.	
The ship's height (it only moves sideways).	
What an object slot contains.	
Sprite 9 shows the explosion; 0 is the ship and 1-8 the objects.	

Part 2: music and effects

starcatcher.bas

```
22  MUSIC titlesong
23      TEMPO 100
24      FM1  "@epiano v13 o5 l4 L g e- c e- f a- >c< a- g b >d< b >c2< g2"
25      FM2  "@bass v12 o2 l2 L c c f f g g c c"
26      PSG1 "@decay v8 o5 l8 L [c g e- g]2 [c a- f a-]2 [d b g b]2 [c g e- g]2"
27  END MUSIC
28
29  MUSIC gamesong
30      TEMPO 150
31      FM1   "@lead v12 o5 l8 L c4 e-8 g8 >c4< b-8 g8 a-4 g8 f8 e-4 c4"
32      FM1   "d4 f8 a-8 g4 f8 e-8 d2 r4 g4 c4 e-8 g8 >c4 d8 e-8"
33      FM1   "d4 c8< b-8 a-4 g4 f8 g8 a-8 b-8 >c8 d8 e-8 d8 c2 r2"
34      FM2   "@bass v13 o2 l8 L [c c >c< c]2 [a- a- >a-< a-]2 [b- b- >b-< b-]2 [g g >g< g]2"
35      FM2   "[c c >c< c]2 [a- a- >a-< a-]2 [f f >f< f] [g g >g< g] [c c >c< c]2"
36      FM3   "@strings v8 o4 l1 L e- c d <b> e- c l2 c d l1 e-"
37      FM4   "@strings v7 o4 l1 L g e- f d g e- l2 c d l1 g"
38      FM5   "@snare v11 o3 l4 L [r c r c]8"
39      FM6   "@kick v14 o2 l4 L [c r c r]8"
40      PSG1  "@pluck v9 o4 l16 L [c e- g >c<]4 [c e- a- >c<]4 [d f b- >d<]4 [d g b >d<]4"
41      PSG1  "[c e- g >c<]4 [c e- a- >c<]4 [c f a- >c<]2 [d g b >d<]2 [c e- g >c<]4"
42      NOISE "@hat v9 l8 L [c c c c c c c c]8"
43  END MUSIC
44
45  MUSIC overtune
46      TEMPO 110
```

A calm tune for the title, 100 BPM, in C minor.	
Slow.	
Electric piano melody in quarter notes, looping (`L`). `e-` and `a-` are flats.	
Bass in half notes: C, F, G, C - the chords of the melody.	
A soft square-wave arpeggio.	
End of the title tune.	
The game music uses all 10 channels:	
fast,	
a lead melody that continues...	
...over...	
...three lines,	
a pumping bass line...	
...in two lines,	
two string channels holding long chord notes...	
...(whole notes, l1),	
snare drum on beats 2 and 4,	
kick drum on beats 1 and 3,	
fast arpeggios following the chords...	
...over two lines,	
and hi-hats on every eighth note.	
End of the game music.	
The game-over jingle...	
...a bit slower...	

```
47      FM1 "@brass v13 o4 l8 g f e- d c4 <g4 >c2"
```
...brass falling down the scale - no L, so it plays only once,

```
48      FM2 "@bass v13 o2 l4 c <g >c c1"
```
with a bass.

```
49  END MUSIC
```
End of the jingle.

```
50
```

```
51  ' ---------- sound effects ----------
```

```
52  SFX catch
```
Star caught:

```
53      PSG3 "@short v15 o6 l32 c e g >c"
```
four fast rising notes.

```
54  END SFX
```
End of 'catch'.

```
55
```

```
56  SFX boom
```
Crash:

```
57      NOISE "@decay v15 w1 l2 e"
```
white noise slowly fading.

```
58  END SFX
```
End of 'boom'.

```
59
```

```
60  SFX levelup
```
Level up:

```
61      PSG3 "@pluck v14 o5 l32 c e g >c e g >c"
```
a fast run over two octaves.

```
62  END SFX
```
End of 'levelup'.

Part 3: variables and the main program

starcatcher.bas

```
64  ' ---------- variables ----------
```

```
65  DIM shipx, score, hiscore, lives, level, caught
```
Ship position, score, high score, lives, level, stars caught.

```
66  DIM spawnTimer, boomTimer, scrollY
```
Frames until the next object, explosion/invincibility timer, sky scroll position.

```
67  DIM objKind(MAXOBJ), objX(MAXOBJ), objY(MAXOBJ),
    objSpeed(MAXOBJ)
```
The object pool: kind, x, y (1/16 pixels) and falling speed.

```
68
```

```
69  '
    ============================================================
```

```
70  '  MAIN PROGRAM
```

```
71  '
    ============================================================
```

```
72  LoadGraphics
```
The main program: load graphics once...

```
73  DO
```
...then forever:

```
74      TitleScreen
```
title screen,

```
75      PlayGame
```
play,

```
76      GameOver
```
game over.

```
77  LOOP
```
And again.

Part 4: loading graphics

starcatcher.bas

```
80  SUB LoadGraphics()
```
Runs once.

```
81      LOADTILES gfx                    ' copy the
    sprite pictures into video memory
```
Sprites into video memory...

```
82      LOADTILES space                  ' ...and the
    background tiles
```
...followed by the background tiles.

```
83      LOADPAL space, 0                ' background colours -> palette 0
84      LOADPAL gfx, 1                  ' sprite colours     -> palette 1
85      SETCOLOR 0, 0, RGB(0, 0, 50)    ' colour 0 of palette 0 = deep blue
86      BACKDROP 0, 0                   ' use it as the screen background
87      TEXTPAL 3                       ' text uses palette 3 (white by default)
88      DRAWMAP sky, PLANE_B, space, 0  ' draw the Tiled map on plane B
89      FOR i = 0 TO BOOM_SPR
90          SPRITEPAL i, 1              ' all our sprites use palette 1
91      NEXT
92  END SUB
```

Background palette 0...

...sprite palette 1.

Deep blue...

...night sky.

White text.

Draw the stars and the planet.

Sprites 0-9...

...all use palette 1.

Next sprite.

End of LoadGraphics.

Part 5: the title screen

starcatcher.bas

```
95  SUB TitleScreen()
96      LOCAL t
97      CLS
98      CLEARSPRITES
99      PLAYMUSIC titlesong
100     PRINT AT 13, 7, "S T A R   C A T C H E R"
101     PRINT AT 12, 11, "CATCH THE STARS,"
102     PRINT AT 12, 12, "DODGE THE ROCKS!"
103     PRINT AT 13, 16, "HI-SCORE "; PAD$(hiscore, 5)
104     SPRITE 0, 152, 146, gfx, F_SHIP
105     FADEIN 2
106     DO
107         t += 1
108         IF t AND 32 THEN
109             PRINT AT 14, 21, "PRESS START"
110         ELSE
111             PRINT AT 14, 21, "           "
112         END IF
113         SPRITE 0, 152, 146, gfx, F_SHIP + ((t >> 2) AND 1)
114         ScrollStars
115         VSYNC
116     LOOP UNTIL PRESSED(BTN_START)
117     RANDOMIZE TIMER                 ' different game every time
118     PLAYSFX levelup
119 END SUB
```

Shows the title until START is pressed.

Frame counter for blinking.

Clear text...

...and sprites.

Title music.

Title, spaced out for style.

The goal...

...in two lines.

The best score so far.

Show the ship as decoration.

Fade in from black (2 frames per step). After a game over the screen is black; at power-on it is already bright.

Title loop:

count,

for 32 frames...

...show PRESS START,

for the next 32...

...hide it: blinking text.

End of blink.

Flickering engine flame.

Moving sky.

Next frame.

Until START.

Every game different.

Start jingle.

End of TitleScreen.

Part 6: playing

starcatcher.bas	
`SUB PlayGame()`	One complete game.
`    CLS`	Remove the title text.
`    score = 0`	Reset everything:
`    lives = 3`	3 ships,
`    level = 1`	level 1,
`    caught = 0`	no stars caught,
`    shipx = 152`	ship in the centre,
`    spawnTimer = 30`	first object after half a second,
`    boomTimer = 0`	no explosion.
`    FOR i = 1 TO MAXOBJ`	Empty the object pool...
`        objKind(i) = KIND_NONE`	...every slot...
`        SPRITEOFF i`	...and hide its sprite.
`    NEXT`	Next slot.
`    PLAYMUSIC gamesong`	Game music.
`    DrawHud`	Status line.
`    DO`	**The game loop:**
`        MoveShip`	ship,
`        SpawnObjects`	new objects,
`        MoveObjects`	falling and collisions,
`        ScrollStars`	sky,
`        VSYNC`	show.
`    LOOP UNTIL lives = 0`	Until all ships are lost.
`END SUB`	End of PlayGame.

Part 7: the ship

starcatcher.bas	
`SUB MoveShip()`	Steering and drawing the ship and its explosion.
`    LOCAL frame`	Which ship picture.
`    IF BUTTON(BTN_LEFT) THEN shipx -= 3`	Left...
`    IF BUTTON(BTN_RIGHT) THEN shipx += 3`	...right, 3 pixels per frame.
`    shipx = CLAMP(shipx, 0, 304)`	Stay on screen.
`    frame = F_SHIP + ((TIMER >> 2) AND 1)     ' flicker the engine flame`	Frame 0 or 1, switching every 4 frames: a flickering flame.
`    IF boomTimer > 0 THEN`	Recently hit?
`        boomTimer -= 1`	Count down the 90 frames...
`        IF boomTimer > 70 THEN`	...in the first 20 frames...
`            SPRITE BOOM_SPR, shipx, SHIP_Y, gfx, F_BOOM + ((boomTimer >> 2) AND 1)`	...show the explosion over the ship, alternating its two frames,
`        ELSE`	after that...
`            SPRITEOFF BOOM_SPR`	...hide it.
`        END IF`	End of explosion.
`        IF boomTimer AND 4 THEN              ' blink while invincible`	Blink while invincible:
`            SPRITEOFF 0`	hidden for 4 frames...
`        ELSE`	...then...
`            SPRITE 0, shipx, SHIP_Y, gfx, frame`	...visible for 4.

```
164         END IF                                          End of blink.
165     ELSE                                                Not hit recently:
166         SPRITE 0, shipx, SHIP_Y, gfx, frame             just draw the ship.
167     END IF                                              End of 'recently hit'.
168 END SUB                                                 End of MoveShip.
```

Part 8: new objects

starcatcher.bas

```
171 SUB SpawnObjects()                                      Sometimes drops a new star or rock.
172     spawnTimer -= 1                                     Count down...
173     IF spawnTimer > 0 THEN EXIT SUB                     ...not yet.
174     spawnTimer = MAX(12, 50 - level * 4)                Time to the next object: 46 frames at level 1, 4 fewer each level, at least 12.
175     FOR i = 1 TO MAXOBJ                                 Find a free slot:
176         IF objKind(i) = KIND_NONE THEN                  this one is free:
177             IF RND(100) < 20 + level * 3 THEN           a 23% chance of a rock at level 1, 3% more each level...
178                 objKind(i) = KIND_ROCK                  ...a rock...
179             ELSE                                        ...otherwise...
180                 objKind(i) = KIND_STAR                  ...a star.
181             END IF                                      End of kind choice.
182             objX(i) = RND(304)                          Random column.
183             objY(i) = -16 * 16                  '       Just above the screen (-16 pixels, in 1/16 units).
    1/16 pixel units
184             objSpeed(i) = 16 + RND(16) + level * 3      Falling speed 1-2 pixels per frame, faster each level.
185             EXIT SUB                                    One object at a time.
186         END IF                                          End of 'free slot'.
187     NEXT                                                Next slot.
188 END SUB                                                 End of SpawnObjects.
```

Part 9: falling and collisions

starcatcher.bas

```
191 SUB MoveObjects()                                       Moves every object and checks if it touches the ship.
192     LOCAL y, frame                                      Pixel y and picture.
193     FOR i = 1 TO MAXOBJ                                 For each slot...
194         IF objKind(i) <> KIND_NONE THEN                 ...that contains an object:
195             objY(i) += objSpeed(i)                      fall,
196             y = objY(i) >> 4                    '       convert to pixels.
    back to whole pixels
197             IF y > 224 THEN                             Fell off the bottom:
198                 objKind(i) = KIND_NONE                  free the slot...
199                 SPRITEOFF i                             ...hide the sprite.
200             ELSEIF COLLIDE(shipx + 2, SHIP_Y + 2, 12,   Touching the ship (12 × 12 boxes, a bit smaller than the pictures, for fairness)?
    12, objX(i) + 2, y + 2, 12, 12) THEN
201                 HitObject i                             Handle it in its own SUB.
202             ELSE                                        Otherwise draw it:
```

```
203                 IF objKind(i) = KIND_STAR THEN frame = F_STAR ELSE frame = F_ROCK   star or rock picture,
204                 frame += (TIMER >> 3) AND 1                                         plus 0 or 1 to animate: twinkling stars, tumbling rocks,
205                 SPRITE i, objX(i), y, gfx, frame                                    draw with sprite number i.
206             END IF                                                                  End of fall/hit/draw.
207         END IF                                                                      End of 'contains an object'.
208     NEXT                                                                            Next slot.
209 END SUB                                                                             End of MoveObjects.
```

Part 10: catching and crashing

starcatcher.bas

```
212 SUB HitObject(n)                                    Object n touched the ship.
213     IF objKind(n) = KIND_STAR THEN                  A star:
214         score += 10 * level                         points, more at higher levels,
215         caught += 1                                 count it.
216         IF caught MOD 10 = 0 THEN                   Every 10th star...
217             level += 1                              ...next level...
218             PLAYSFX levelup                         ...with a jingle.
219         ELSE                                        Otherwise...
220             PLAYSFX catch                           ...the normal catch sound.
221         END IF                                      End of level check.
222     ELSEIF boomTimer = 0 THEN                       A rock - only if the ship is not invincible:
223         lives -= 1                                  lose a ship,
224         boomTimer = 90                              explosion and invincibility for 90 frames,
225         PLAYSFX boom                                boom.
226     END IF                                          End of star/rock.
227     objKind(n) = KIND_NONE                          The object is gone either way...
228     SPRITEOFF n                                     ...from the screen too.
229     DrawHud                                         Update the status line.
230 END SUB                                            End of HitObject.
```

Part 11: sky, status line and game over

starcatcher.bas

```
233 SUB ScrollStars()                                   Moves the sky.
234     scrollY -= 1                                    One step per frame...
235     SCROLL PLANE_B, 0, scrollY >> 1                 ...but the plane moves only half as fast (>> 1): a calm drift.
236 END SUB                                            End of ScrollStars.
237
238 SUB DrawHud()                                      The status line ('HUD' = head-up display).
239     PRINT AT 1, 1, "SCORE "; PAD$(score, 5)         Score,
240     PRINT AT 17, 1, "LEVEL "; level                 level,
241     PRINT AT 31, 1, "SHIPS "; lives                 ships.
242 END SUB                                            End of DrawHud.
243
244 SUB GameOver()                                     After the last ship is lost.
```

245	`    LOCAL t`	Frame counter.
246	`    SPRITEOFF BOOM_SPR`	Remove a leftover explosion.
247	`    PLAYMUSIC overtune`	Game-over jingle.
248	`    IF score > hiscore THEN hiscore = score`	New high score? Remember it (until the console is switched off).
249	`    PRINT AT 15, 12, "GAME  OVER"`	The message.
250	`    DO`	Wait:
251	`        t += 1`	count,
252	`        ScrollStars`	keep the sky moving,
253	`        VSYNC`	next frame,
254	`    LOOP UNTIL t > 240 OR (t > 60 AND PRESSED(BTN_START))`	4 seconds - or START after at least 1 second. The brackets make the order clear.
255	`    FADEOUT 2`	Fade to black,
256	`    CLEARSPRITES`	hide everything. The main loop then shows the title again.
257	`END SUB`	End of GameOver.

Make it yours

- A heart that gives an extra ship (draw a ninth frame).
- Let the ship shoot rocks with button A.
- Show "LEVEL 2!" in the middle of the screen for two seconds when the level changes.
- Two-player mode with a second ship on joypad 2.

Part V — Help

What to do when something goes wrong, a glossary of words used in this book, and a one-page list of commands.

29 When something goes wrong

Every programmer makes mistakes - all the time. The skill is not to avoid mistakes but to find them quickly. Here is how.

Errors from the compiler

When the compiler does not understand something, it stops and prints a message with the file name and line number. In VS Code the line is underlined in red and listed in the *Problems* panel (`Ctrl+Shift+M`). Click the message to jump to the line.

Example

```
game.bas:42: error: unknown variable 'scroe' - give it a value first or declare it with DIM at the top of the program
```

Read the message slowly - it usually says exactly what is wrong. Then look at the line *and the line before it*: sometimes the real mistake is a little earlier.

Message	What it means	How to fix it
unknown variable 'x'	You read a variable that was never given a value - often a typo (`scroe` instead of `score`).	Check the spelling. If a SUB uses a global, declare it with DIM in the main program.
unknown command or SUB 'PRNT'	A misspelt command, or a SUB that doesn't exist.	Check spelling; press `Ctrl+Space` to see the correct names.
missing END IF	A block IF (THEN at the end of the line) was never closed.	Add END IF. Indentation helps you see where.
FOR i without NEXT / WHILE without WEND / DO without LOOP	A loop was not closed.	Add the closing word.
expected ... but found ...	Something is missing, e.g. a comma between values or THEN after IF.	Look at the place the message mentions.
X needs 3 value(s), got 2	A command or SUB got too few or too many values.	Hover over the name in VS Code to see what it expects.
... is a constant and cannot be used as a variable	A variable has the same name as a CONST or built-in constant - remember that upper/lower case do not matter.	Rename the variable.

'x' is an asset name	A variable has the same name as a picture, map or sound.	Rename one of them.
numbers are integers	You wrote a decimal number like 1.5.	Use whole numbers; for fractions use 1/16 pixel units (chapter 19).
file not found: hero.png	The picture is not in the same folder as the .bas file, or the name is different.	Check the file name - including upper/lower case on macOS/Linux.
image uses 23 colours ... limit is 16	The PNG has too many colours.	Reduce colours (GIMP: Image > Mode > Indexed, 16 colours).
image size ... is not a multiple of ...	The picture's width/height doesn't fit the frame or tile size.	Resize the canvas, e.g. to a multiple of 16.
unknown FM instrument '@pian'	A misspelt instrument name in music.	Check the list in chapter 22.

The game builds but does not work

Symptom	Likely cause
Nothing moves; the picture is frozen	The game loop has no `VSYNC` (or WAIT), so sprites are never sent to the screen.
A sprite does not appear	Missing `LOADTILES`, wrong sprite number, or it is placed outside the screen.
A sprite has wrong colours	Its palette was not loaded or the sprite uses another palette: check `LOADPAL` and `SPRITEPAL`.
Text is invisible	Text uses colour 15 of the text palette. After loading your own colours into palette 0, use `TEXTPAL 3`.
The game is far too fast	The loop has no VSYNC; or VSYNC is inside an inner loop that runs many times.
The game is too slow / jerky	More than one VSYNC per loop, or a very big loop (thousands of steps) every frame.
A button press is ignored	PRESSED is only true in one frame; if your loop skips frames (WAIT 5), presses can be missed. Use VSYNC in game loops.
A value is "wrong"	Whole-number division (7 / 2 = 3), or a local variable used where a global was meant.
The screen turns one solid colour and everything stops	The program crashed - usually an array index far outside its size, or POKE to a wrong address.

Detective tricks

- **Print the values.** Add `PRINT AT 0, 27, "X="; x; " Y="; y; " "` to the game loop and watch the numbers while you play. This is the oldest and best debugging trick.
- **Make it smaller.** Comment out parts of the program (put `'` in front of lines) until the problem disappears. The last thing you removed contains the problem.
- **Change one thing at a time.** Build and test after every small change. If something breaks, you know what did it.

- **Explain it to someone.** Explaining your program line by line - even to a cat or a rubber duck - often reveals the mistake. Programmers really call this *rubber duck debugging*.

30 Glossary

Words used in this book, in alphabetical order.

Array
A variable with many numbered boxes: `scores(3)`.

Assignment
Putting a value into a variable: `x = 5`.

Backdrop
The colour shown where nothing else is drawn.

BASIC
A programming language designed in 1964 for beginners.

Bug
A mistake in a program. Removing bugs is called debugging.

Build
Turning your program into a ROM with the compiler.

Collision
Two objects touching. Tested with `COLLIDE` or by asking the map with `MAPTILE`.

Comment
Text after `'` that the compiler ignores; notes for humans.

Compiler
The program that translates your text into machine code (`mbc.py`).

Condition
A question that is true or false: `lives = 0`.

Constant
A named value that never changes: `CONST SPEED = 3`.

Coordinates
The x (right) and y (down) numbers of a position on the screen.

CSV
A text file of numbers separated by commas. Tiled exports maps in this format.

DATA
Numbers stored in the program, read with `READ`.

Emulator
A program that imitates a game console on a PC.

Frame
One picture on the TV - 60 per second. Also: one picture of an animation in a sprite sheet.

Function
A named calculation that gives back a value: `ABS(x)`, or your own FUNCTION.

Game loop
The loop that runs once per frame: input, update, draw, VSYNC.

Global variable
A variable that can be used everywhere (DIM in the main program).

Index
The number that selects a box of an array.

Local variable
A variable that only exists inside one SUB or FUNCTION.

Loop
Code that repeats: FOR, WHILE, DO, REPEAT.

MML
Music Macro Language: notes written as letters, e.g. `"c d e"`.

Palette
A list of 16 colours. Tiles and sprites store colour numbers that refer to a palette.

Parameter
A value given to a SUB or FUNCTION when it is called.

Pixel
One dot of the picture. The screen is 320 × 224 pixels.

Plane
A big background layer made of tiles (A, B, and the window).

PNG
A picture file format that keeps pixels exact. Use it for all game graphics.

Pool
A set of array slots that are reused for objects like bullets.

Random
Unpredictable; `RND(n)` gives random numbers.

ROM
The game file (`.md`) - the same thing that is inside a cartridge.

Scrolling
Moving a plane smoothly so the view follows the action.

SFX
Sound effect.

Sprite
A small picture that can be placed at any pixel position; the console shows up to 80.

Sprite sheet
One PNG containing several sprite frames.

String
Text between quotes: `"HELLO"`.

SUB
A named block of code you can call like a command.

Tile
An 8 × 8 pixel picture - the building block of all Mega Drive graphics.

Tiled
A free map editor (mapeditor.org) for drawing levels from tiles.

Tileset
A PNG containing all the tiles for a map.

Transparent
Colour 0 of every palette: you see through it.

Variable
A named box that holds one number.

VRAM
The video chip's memory, where tiles and maps are stored.

VSYNC
Waiting for the next frame and sending changes to the video chip.

31 Command cheat sheet

The most important MegaBASIC words on one page. The Reference Manual describes all of them in detail.

Program structure

`DIM a, b(10)`	create variables / arrays
`CONST NAME = 5`	constant
`SUB Name(p) ... END SUB`	own command
`FUNCTION F(p) ... RETURN x`	own function
`LOCAL x`	local variable
`' text`	comment

Decisions and loops

`IF c THEN ... ELSEIF ... ELSE ... END IF`	decide
`SELECT CASE v / CASE 1, 2 / CASE ELSE`	choose
`FOR i = 1 TO 10 STEP 1 ... NEXT`	count
`WHILE c ... WEND`	repeat while
`DO ... LOOP UNTIL c`	repeat until
`EXIT FOR / CONTINUE FOR`	leave / skip

Text and timing

`PRINT AT x, y, "T"; v`	write text
`PAD$(v, 5)`	zero-padded number
`CLS`	clear text
`TEXTPAL p`	text palette
`VSYNC`	wait one frame
`WAIT n`	wait n frames
`TIMER`	frame counter

Joypad

`BUTTON(BTN_A)`	held
`PRESSED(BTN_START)`	just pressed
`BUTTON(BTN_UP, 2)`	joypad 2
`BTN_UP DOWN LEFT RIGHT A B C START`	buttons

Graphics

`SPRITES s = "f.png", 2, 2`	sprite sh
`TILES t = "f.png", 16`	tileset
`MAP m = "f.csv", 16`	Tiled ma
`LOADTILES s`	to video memory
`LOADPAL s, 1`	colours t palette 1
`SPRITE n, x, y, s, frame`	show spr
`SPRITEPAL n, p / SPRITEFLIP n, h, v`	sprite palette / mirror
`SPRITEOFF n / CLEARSPRITES`	hide
`SETCOLOR p, i, RGB(r, g, b)`	change a colour
`BACKDROP p, i`	backgrou colour
`DRAWMAP m, PLANE_B, t, 0`	draw a m
`MAPSCROLL m, PLANE_B, t, 0, camx, camy`	scroll a map
`MAPTILE(m, tx, ty)`	tile at ma position
`SETTILE plane, x, y, tile, pal`	one tile
`SCROLL plane, x, y`	scroll a plane
`FADEIN / FADEOUT`	fades

Maths

`RND(n)`	random 0..n-1
`ABS MIN MAX CLAMP`	useful functions
`x >> 4`	divide by 16
`a MOD b`	remainder
`COLLIDE(x1,y1,w1,h1, x2,y2,w2,h2)`	rectangles overlap?
`SIN(a) COS(a) ATAN2(dy, dx)`	angles 0-255

Sound

`MUSIC n ... END MUSIC`	song
`SFX n ... END SFX`	effect

`FM1 "@piano o4 l8 c d e"`	a channel
`PLAYMUSIC n / STOPMUSIC`	music
`PLAYSFX n`	effect
`PAUSEMUSIC / RESUMEMUSIC`	pause

32 Where to go next

You have learned to program and built six games. What now?

- **Change the games.** Every game chapter ends with ideas. Pick one and try it - this is where real learning happens.
- **Make your own small game.** Ideas for first projects: a reaction tester, a 'whack-a-mole', Space Invaders, a maze game, Flappy Bird, a top-down racing game. Start with the smallest version that works, then improve it.
- **Read the Reference Manual.** It describes every command, custom FM instruments, all music commands and the hardware limits.
- **Draw your own graphics and music.** A game with your own art and tunes feels completely different.
- **Play it on real hardware.** Copy your `.md` file onto a flash cartridge and play your game on a real Mega Drive and television.
- **Share it.** Show your game to friends and family. Ask them to play without your help, watch where they get stuck, and improve it.

Thousands of the developers who made the games you love started exactly where you are now: with a BASIC program that printed HELLO.

Have fun - and welcome to Mega Drive game development!